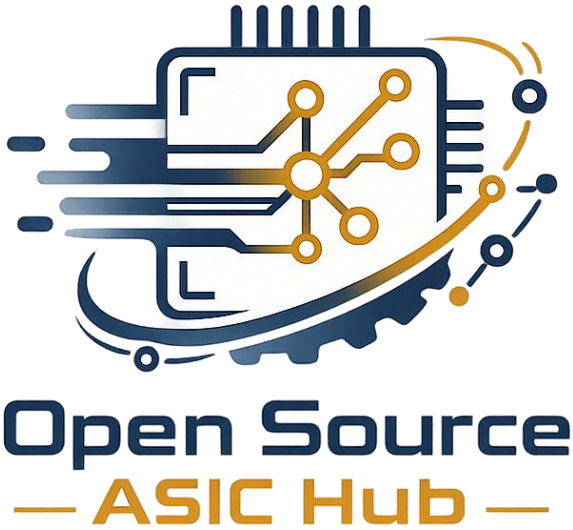


ProxyCore - A Runtime-Configurable Proximity Safety Co-Processor in SkyWater Sky130nm

Silicon Sprint 2026 SP26

American University in Cairo

Authors: Ali Shawky - Karim Khaled - Farah Moataz

Supervisors: Dr. Mohamad Shalan - Dr. Dina Mahmoud

Contents

Abstract:

Proximity-based collision avoidance in industrial and transportation environments demands deterministic, low-latency safety monitoring that operates independently of the main application processor. Running safety-critical logic on a general-purpose microcontroller introduces a single point of failure: a software crash or watchdog timeout disables the safety function entirely. This work presents ***ProxCore****, a dedicated ASIC safety co-processor implemented in the SkyWater Sky130B 130 nm open-source process and taped out on the Silicon Sprint SP26 shuttle through a fully open-source RTL-to-GDSII flow (Yosys, OpenROAD, Magic, KLayout, Netgen). ProxCore receives 16-bit distance measurements from a UART-based LiDAR proximity sensor, suppresses measurement noise with a 16-tap symmetric FIR lowpass filter, and asserts a hardware brake interrupt when three consecutive filtered readings fall below a configurable distance threshold. All ten safety-defining parameters — the braking threshold, eight FIR filter coefficients, and the UART baud rate divider — are runtime-programmable via SPI, enabling the same silicon die to serve urban trams, passenger automobiles, warehouse forklifts, collaborative robots, industrial robot cells, and autonomous guided vehicles without any hardware modification. This runtime-configurability expands the addressable market from approximately 3,000 units per year (single-application) to an estimated 7–24 million units per year across six application domains. The design was verified with* ***45 directed tests*** *across five SystemVerilog testbenches covering unit-level and full-pipeline operation at baud rates from 9,600 to 921,600. The final chip occupies* ***0.908 mm²*** *of die area, consumes* ***1.48 mW*** *at 25 MHz / 1.8 V, and meets timing across all nine PVT signoff corners with* ***zero DRC errors, zero LVS mismatches, zero antenna violations, and zero timing violations****.*

Section 1: Introduction

1.1 Motivation and problem statement

The deployment of proximity-based safety systems spans a wide range of environments. Urban trams share track corridors with pedestrians and automobiles, warehouse forklifts operate alongside human workers in narrow aisles, and collaborative robots manipulate objects within arm's reach of operators. In each of these settings, the detection of a dangerously close obstacle and the initiation of an emergency stop must occur within milliseconds and must not depend on the correct functioning of unrelated software tasks.

Current industry practice typically implements proximity monitoring in one of two ways. The first approach runs the sensor processing and threshold logic as a software task on the vehicle's main microcontroller. This approach is simple to develop but creates a single point of failure: any software fault — a stack overflow, an unhandled interrupt, or a real-time deadline miss — can disable the safety function without warning. The second approach uses a field-programmable gate array to implement the logic in hardware. While this eliminates the software single-point-of-failure, FPGAs carry significant per-unit cost, higher power consumption, and require an external configuration flash whose corruption can render the device non-functional at power-on.

A dedicated application-specific integrated circuit addresses both concerns. The safety logic is physically instantiated in silicon, making it immune to software faults and flash corruption. The per-unit silicon cost is a fraction of an equivalent FPGA at production volumes, and the static power consumption is typically an order of magnitude lower. However, the traditional limitation of ASICs is inflexibility: once taped out, the behavior is fixed.

ProxCore resolves this limitation by making every application-defining parameter runtime-configurable through a serial peripheral interface. A host microcontroller sends a short sequence of SPI register writes at power-on, and the same physical die becomes a tram safety controller, a parking-assist sensor processor, a forklift pedestrian detector, or a cobot safety zone monitor. This configurability transforms ProxCore from a single-application device into a platform product, expanding the addressable market from thousands of units per year to millions.

1.2 Project Objectives

The objectives of this work are as follows:

1. Design a dedicated ASIC safety co-processor that receives LiDAR distance measurements over a UART interface and issues a hardware brake interrupt when a sustained proximity event is detected.
2. Implement a 16-tap symmetric FIR lowpass filter with runtime-programmable coefficients to suppress single-sample noise spikes from rain, dust, and sensor artifacts while preserving real obstacle transitions.
3. Implement a debounced threshold finite state machine that requires three consecutive sub-threshold filtered readings before asserting the brake interrupt, preventing false triggers from transient noise.
4. Provide runtime configuration of all safety parameters — braking threshold, eight FIR coefficients, and the UART baud rate divider — via a write-only SPI register interface.

5. Make the UART baud rate runtime-configurable so that a single chip revision can interface with any UART-based LiDAR sensor at any standard baud rate from 9,600 to 921,600.
6. Implement the design in fully synthesizable SystemVerilog targeting the SkyWater Sky130B 130 nm open-source process design kit.
7. Verify all modules with comprehensive directed testbenches and tape out on the Silicon Sprint SP26 shuttle.

1.3 ASIC Justification:

An earlier revision of this design used a fixed compile-time baud rate parameter, restricting the chip to a single LiDAR sensor model and a single application: urban tram collision avoidance. Feedback from the evaluation panel correctly identified that the global tram fleet — approximately 50,000 vehicles — does not generate sufficient volume to justify custom silicon, and that the power savings of an ASIC relative to the megawatt-scale traction power of a tram are negligible. Under those constraints, an FPGA implementation would be the pragmatic choice.

The introduction of the runtime-configurable baud rate fundamentally changes this analysis. With all ten parameters programmable via SPI, the same die serves the following markets:

[TABLE 1: Target Markets and Volume Estimates]

Market Segment	Annual Unit Volume (Conservative)
Passenger automobiles (parking assist, low-speed collision warning)	5,000,000 – 20,000,000
Warehouse forklifts (pedestrian safety, ISO 3691-4 compliance)	1,000,000 – 2,000,000
Collaborative robots (safety zone monitoring, ISO/TS 15066 compliance)	500,000 – 1,000,000
Autonomous guided vehicles and mobile robots	500,000 – 1,000,000
Urban trams and light rail	2,000 – 5,000
Combined total	**7,000,000 – 24,000,000**

At these volumes, the economic comparison between ASIC and FPGA is decisive. A representative low-cost FPGA suitable for this design, such as the Lattice iCE40UP5K, costs approximately At these volumes, the economic comparison between ASIC and FPGA is decisive. A representative low-cost FPGA suitable for this design, such as the Lattice iCE40UP5K, costs approximately $1.50 per unit in volume. A ProxCore ASIC die at 130 nm is estimated at $0.10–$0.30 per unit. At 10 million units per year, the FPGA approach costs $15 million in silicon alone; the ASIC approach costs $1–$3 million. The annual savings of $12–$14 million recover the mask cost on the first production run.

The power argument, which was legitimately irrelevant for trams, becomes significant in battery-powered applications. A Lattice iCE40 FPGA draws approximately 50 mW in active operation. A dedicated ASIC of this complexity in Sky130B is estimated at 2–5 mW. For warehouse forklifts operating on 48 V battery packs for 16-hour shifts, for collaborative robots on mobile battery-powered platforms, and for automotive systems that remain active during key-off states, this 10–25× power reduction directly extends operational runtime and prevents battery drain.

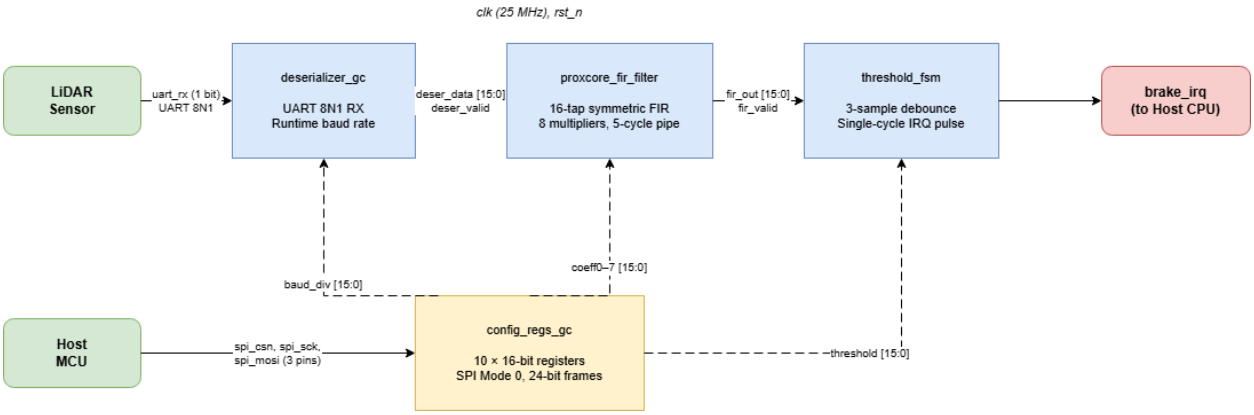
Finally, the determinism argument favors ASIC specifically for safety applications. An FPGA loads its configuration from external flash at power-on. If the flash is corrupted, the device starts with no function. An ASIC's safety logic is physically instantiated in silicon and is functional from the moment power is applied. This property simplifies the safety case for functional safety certification under IEC 61508, ISO 26262, and ISO 13849.

Section 2: System Architecture

2.1 Top-Level Diagram

ProxCore consists of four processing blocks connected in a linear pipeline, with a parallel configuration path from the SPI interface to all three processing stages. The architecture is shown in Figure 1.

[FIGURE 1: System Block Diagram]



The LiDAR sensor transmits 16-bit distance measurements as pairs of UART bytes. The deserializer synchronizes the asynchronous serial input, samples each bit at the baud-rate midpoint, validates the stop bit, and assembles two consecutive bytes into a 16-bit word. The word and a single-cycle valid pulse pass to the FIR filter.

The FIR filter maintains a 16-sample delay line and computes the weighted sum of all taps using eight runtime-programmable coefficients. The filter exploits the symmetric property of a linear-phase FIR to halve the multiplier count from 16 to 8. A 5-stage pipeline produces one filtered output per input sample with fixed latency.

The threshold FSM compares each filtered output against the programmable threshold. It requires three consecutive sub-threshold readings before asserting the brake interrupt, rejecting isolated noise spikes that survive the FIR filter. The interrupt is a single-cycle pulse that does not repeat while the obstacle persists.

The SPI configuration register file receives 24-bit write-only frames from a host microcontroller and stores the threshold, eight FIR coefficients, and the baud rate divider. All outputs are registered flip-flops in the system clock domain, requiring no additional synchronization at the receiving modules.

2.2 Pin-Level Interface

ProxCore uses six active GPIO pins on the SP26 shuttle, mapped through the project_macro wrapper to the bottom GPIO bank. The complete pin assignment is listed in Table 2.

[TABLE 2: Pin Assignment]

GPIO Pin	Signal Name	Direction	Width	Drive Mode	Function
gpio_bot[0]	uart_rx	Input	1 bit	3'b001 (input, no pull)	LiDAR sensor UART data
gpio_bot[1]	spi_csn	Input	1 bit	3'b001 (input, no pull)	SPI chip select (active low)
gpio_bot[2]	spi_sck	Input	1 bit	3'b001 (input, no pull)	SPI clock
gpio_bot[3]	spi_mosi	Input	1 bit	3'b001 (input, no pull)	SPI data input
gpio_bot[4]	brake_irq	Output	1 bit	3'b110 (strong push-pull)	Brake interrupt to host CPU
gpio_bot[5]	dbg_filtered_valid	Output	1 bit	3'b110 (strong push-pull)	Debug: FIR output valid pulse

All remaining GPIO pins on the bottom, right, and top banks are unused and safely tied off with output drivers disabled (oeb = 1) and output data driven to zero.

The system clock and active-low asynchronous reset are provided by the shuttle green macro on the left edge of the die. The clock frequency is 25

MHz. The power-on reset signal (por_n) is available but not used; the design relies on the shuttle-provided reset_n.

2.3 Data Flow Overview

The complete data flow from sensor to interrupt is as follows. The LiDAR sensor continuously transmits distance measurements as UART 8N1 frames at a baud rate configured via the baud_div register. Each 16-bit distance value is transmitted as two consecutive bytes, low byte first. The deserializer synchronizes the asynchronous input through a two-stage flip-flop synchronizer with an additional stage for falling-edge detection, samples each data bit at the center of the bit period using a programmable baud counter, validates the stop bit, and pairs two successive bytes into a 16-bit word. A framing error (invalid stop bit) causes the partial word to be discarded, preventing byte misalignment from propagating through the pipeline.

The 16-bit unsigned distance value in Q10.6 fixed-point format enters the FIR filter's delay line. The filter computes the dot product of the 16-tap delay line with the coefficient vector, exploiting symmetry to reduce the computation to eight pre-additions and eight multiplications. The 36-bit signed accumulator is arithmetically right-shifted by 15 bits to scale from the Q1.15 coefficient domain back to Q10.6, then truncated to 16 bits. A valid pulse, delayed by exactly five pipeline stages, accompanies the filtered output.

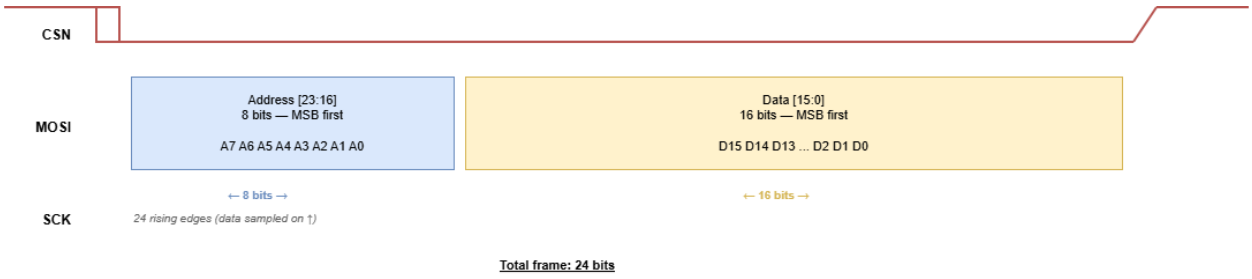
The threshold FSM evaluates the filtered distance against the threshold register on each valid pulse. It transitions through CLEAR, WARN1, and WARN2 states on consecutive sub-threshold readings, asserting brake_irq for exactly one clock cycle on the third consecutive sub-threshold reading as it enters the BRAKING state. Any reading at or above the threshold resets the state machine to CLEAR. The FSM does not re-

assert the interrupt while the obstacle persists, preventing interrupt flooding on the host CPU.

2.4 Configuration Architecture

The host microcontroller configures ProxCore through a write-only SPI interface operating in Mode 0 (CPOL=0, CPHA=0): data is sampled on the rising edge of SCK with SCK idle low. Each SPI transaction consists of a single 24-bit frame transmitted MSB-first while CSN is held low. The frame format is shown in Figure 2.

[FIGURE 2: SPI Frame Format]



The 8-bit address field selects one of ten registers. The 16-bit data field provides the new value. The register map is listed in Table 3.

[TABLE 3: SPI Register Map]

Address	Register	Default Value	Encoding	Description
0x00	threshold	16'd2560	Q10.6 unsigned	Braking distance (default: 40 m)
0x01	coeff0	16'sd112	Q1.15 signed	FIR coefficient h[0] and h[15]
0x02	coeff1	16'sd243	Q1.15 signed	FIR coefficient h[1] and h[14]
0x03	coeff2	16'sd618	Q1.15 signed	FIR coefficient h[2] and h[13]
0x04	coeff3	16'sd1293	Q1.15 signed	FIR coefficient h[3] and h[12]
0x05	coeff4	16'sd2217	Q1.15 signed	FIR coefficient h[4] and h[11]
0x06	coeff5	16'sd3225	Q1.15 signed	FIR coefficient h[5] and h[10]
0x07	coeff6	16'sd4089	Q1.15 signed	FIR coefficient h[6] and h[9]
0x08	coeff7	16'sd4587	Q1.15 signed	FIR coefficient h[7] and h[8]
0x09	baud_div	16'd108	Unsigned integer	UART clock divider

Write transactions to addresses outside the range 0x00–0x09 are silently ignored. There is no read-back capability; the interface is write-only with no MISO pin.

All three SPI signals (spi_csn, spi_sck, spi_mosi) pass through two-stage flip-flop synchronizers inside config_regs_gc before use. The SCK synchronizer uses a third stage for rising-edge detection. This clock-domain crossing logic allows the SPI interface to operate at any clock rate up to one-quarter of the system clock frequency.

2.5 Clock and Reset Strategy

ProxCore operates from a single clock domain at 25 MHz provided by the SP26 shuttle infrastructure. There are no internal clock dividers, clock gates, or phase-locked loops. All flip-flops in the design use the same positive-edge clock with asynchronous active-low reset.

Two categories of external signals require clock-domain crossing synchronization:

1. The UART rx input from the LiDAR sensor, which is asynchronous to the system clock. This is synchronized through a two-stage flip-flop chain (rx_meta_q → rx_sync_q) with a third stage (rx_sync_d_q) for falling-edge detection.
2. The three SPI signals from the host microcontroller, which operate on an independent SPI clock. These are each synchronized through two-stage flip-flop chains, with SCK receiving a third stage for rising-edge detection.

The baud_div output from config_regs_gc does not require synchronization at the deserializer_gc input because both modules share the same system clock and the output is a registered flip-flop value that is stable for the entire inter-write interval.

When rst_n is asserted low, all flip-flops in all four modules reset to their defined initial values. The FIR delay line clears to zero, the threshold FSM returns to the CLEAR state, the deserializer returns to IDLE with the synchronizer chain preset to the line-idle value of 1, and all configuration registers restore their default values. After reset release, the system is fully quiescent and produces no output until the first UART start bit arrives.

Section 3: Detailed Module Design

3.1 UART Deserializer (deserializer_gc)

3.1.1 Purpose

The deserializer_gc module receives asynchronous UART serial data from the LiDAR sensor and assembles it into 16-bit distance words. It implements the UART 8N1 protocol (8 data bits, no parity, 1 stop bit) with runtime-configurable baud rate, start-bit glitch rejection, framing error detection, and two-byte word assembly.

3.1.2 Interface

[TABLE 4: deserializer_gc Port List]

Port	Direction	Width	Description
clk	Input	1	System clock (25 MHz)
rst_n	Input	1	Asynchronous active-low reset
rx	Input	1	UART serial data from LiDAR sensor
baud_div	Input	16	Baud rate clock divider (CLK_FREQ / BAUD_RATE)
data_out	Output	16	Assembled 16-bit distance word
data_valid	Output	1	Single-cycle pulse indicating data_out is valid

3.1.3 Input Synchronization

The rx signal originates from an external sensor and is asynchronous to the system clock. A three-stage synchronizer brings it into the clock domain:

Cycle N: rx_meta_q ← rx (first flip-flop, may go metastable)

Cycle N+1: rx_sync_q ← rx_meta_q (second flip-flop, resolves metastability)

Cycle N+2: rx_sync_d_q ← rx_sync_q (third flip-flop, for edge detection)

The falling-edge detector is a combinational signal:

```
assign rx_fall       = rx_sync_d_q && !rx_sync_q;
```

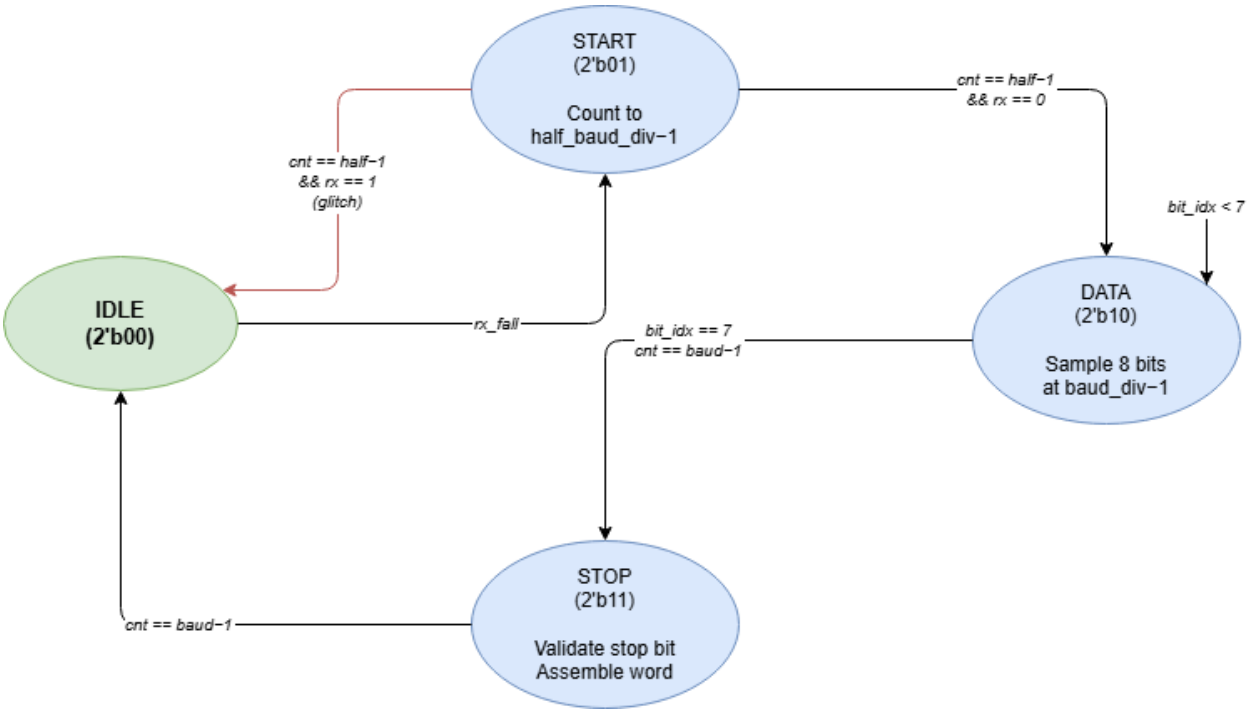
This detects the transition from idle-high to the start bit's low level. The three-stage chain introduces a 3-clock-cycle (120 ns at 25 MHz) latency, which is less than 1.4% of the shortest supported bit period (2,160 ns at 460,800 baud with baud_div = 54 at 25 MHz) and therefore negligible.

All three synchronizer flip-flops reset to 1'b1, which is the UART idle state. This prevents a false start-bit detection during reset release.

3.1.4 State Machine

The deserializer implements a four-state finite state machine as shown in Figure 3.

[FIGURE 3: Deserializer State Machine]



IDLE state. The baud counter and bit index are held at zero. The state machine waits for a falling edge on the synchronized rx signal. Upon detection, it transitions to START and clears the byte accumulator.

START state. The baud counter increments on each clock cycle. When it reaches half_baud_div - 1 (the midpoint of the start bit), the state machine re-samples the rx line. If the line is still low, the start bit is genuine and the machine transitions to DATA. If the line has returned high, the initial falling edge was a glitch shorter than half a bit period, and

the machine returns to IDLE. This midpoint verification rejects noise pulses that are too short to be valid start bits.

The half_baud_div value is computed combinationally:

```
    assign half_baud_div = baud_div >> 1;
```

DATA state. The baud counter increments until it reaches baud_div - 1, which corresponds to one full bit period. At this point, the counter is at the center of the next data bit. The sampled value of rx_sync_q is stored into rx_byte_q at the position indicated by bit_idx_q, implementing LSB-first reception. After all eight bits have been sampled (bit_idx_q == 7), the machine transitions to STOP.

STOP state. The baud counter again counts one full bit period. At the end, the stop bit is validated by checking that rx_sync_q is high. If the stop bit is valid, the received byte is processed for word assembly. If the stop bit is invalid (framing error), the have_first_byte_q flag is cleared, discarding any partially assembled word and preventing byte misalignment from propagating to subsequent frames.

3.1.5 Word Assembly

The LiDAR sensor transmits each 16-bit distance measurement as two consecutive UART bytes, low byte first. The deserializer pairs them using a single-byte buffer:

1. When the first valid byte arrives and have_first_byte_q is 0, the byte is stored in first_byte_q and the flag is set.
2. When the second valid byte arrives and have_first_byte_q is 1, the 16-bit word is assembled as {rx_byte_q, first_byte_q} (second byte in the upper 8 bits, first byte in the lower 8 bits), data_valid is asserted for one clock cycle, and the flag is cleared.

The byte ordering is controlled by the localparam LOW_BYTE_FIRST, which is set to 1 and is not runtime-configurable.

3.1.6 Runtime Baud Rate Configuration

The key upgrade from the original deserializer is the replacement of the compile-time BAUD_RATE parameter with the runtime input port baud_div. The original module computed timing constants as localparams:

localparam CLKS_PER_BIT = CLK_FREQ_HZ / BAUD_RATE;

localparam HALF_CLKS_PER_BIT = CLKS_PER_BIT / 2;

In the upgraded module, these are replaced by the runtime signal and its combinational derivative:

```
input logic [15:0] baud_div
```

```
    assign half_baud_div = baud_div >> 1;
```

The baud counter width is fixed at 16 bits, sufficient to represent divider values up to 65,535. This supports baud rates as low as 382 baud at 25 MHz (baud_div = 65,104), far below any practical sensor baud rate. Table 5 lists the baud_div values for standard baud rates at the 25 MHz operating frequency.

[TABLE 5: Baud Rate Configuration at 25 MHz]

Baud Rate	baud_div	Bit Period	Word Time (2 bytes)
9,600	2,604	104.16 µs	2.083 ms
115,200	217	8.68 µs	173.6 µs
230,400	108	4.32 µs	86.4 µs
460,800	54	2.16 µs	43.2 µs
921,600	27	1.08 µs	21.6 µs

The baud_div input comes from the config_regs_gc registered output, which is in the same clock domain. No additional synchronization is required. Changing baud_div takes effect on the next UART start-bit detection, provided the deserializer is in the IDLE state. Changing it mid-byte may corrupt the current byte, but the framing error mechanism will discard the corrupted word and self-recover on the next valid frame.

3.2 FIR Lowpass Filter (proxcore_fir_filter)

3.2.1 Purpose

The proxcore_fir_filter module implements a 16-tap linear-phase finite impulse response lowpass filter with runtime-programmable coefficients. Its primary function is to smooth noisy LiDAR distance measurements so that single-sample spikes from rain, dust, or sensor artifacts are attenuated while sustained distance changes from real obstacles pass through with minimal delay.

3.2.2 Interface

[TABLE 6: proxcore_fir_filter Port List]

Port	Direction	Width	Type	Description
clk	Input	1	—	System clock
rst_n	Input	1	—	Asynchronous active-low reset
sample_in	Input	16	Unsigned	Distance sample (Q10.6)
sample_valid	Input	1	—	Sample input valid strobe
coeff_in0 – coeff_in7	Input	16 each	Signed	FIR coefficients (Q1.15)
result_out	Output	16	Unsigned	Filtered distance (Q10.6)
result_valid	Output	1	—	Output valid strobe

3.2.3 Symmetric Architecture

A 16-tap linear-phase FIR filter has the property that its coefficients are symmetric: h[k] = h[15−k]. ProxCore exploits this by pre-adding symmetric tap pairs before multiplication, reducing the number of multipliers from 16 to 8. This optimization is critical for area efficiency on the Sky130B process. The computation proceeds as follows:

For each k from 0 to 7:

sum_pre_u[k] = data_reg[k] + data_reg[15−k] (17-bit unsigned pre-addition)

sum_pre_s[k] = {1'b0, sum_pre_u[k]} (18-bit signed, zero-extended)

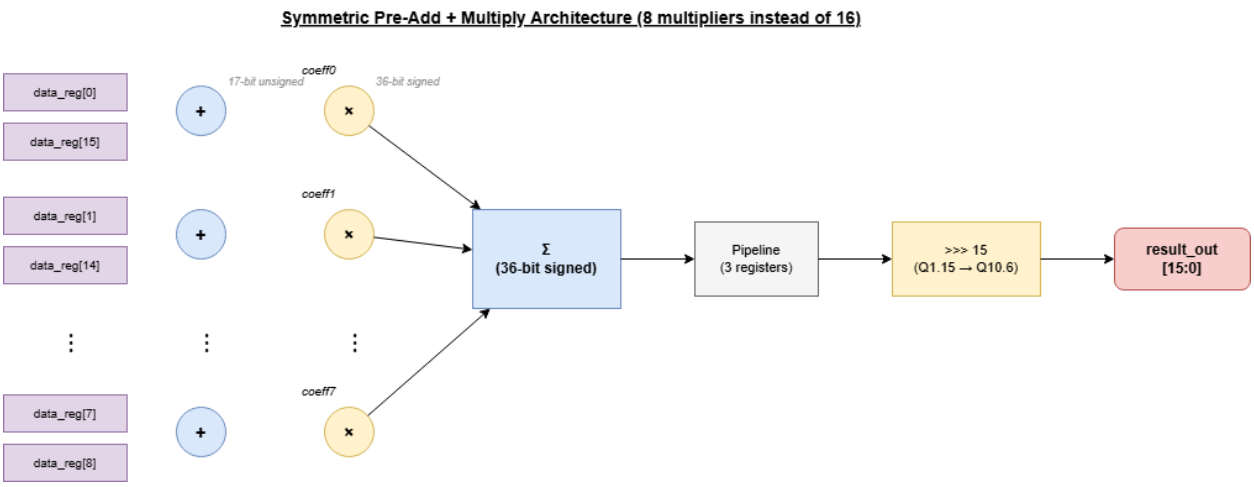
mult[k] = sum_pre_s[k] × coeffs[k] (36-bit signed product)

The accumulator sums all eight products:

accum = Σ mult[k] for k = 0 to 7 (36-bit signed)

The result is then scaled by arithmetic right-shifting by 15 bits and truncated to 16 bits.

[FIGURE 4: Symmetric FIR Datapath]



3.2.4 Pipeline Structure

The filter has a fixed latency of 5 clock cycles from sample_valid input to result_valid output. The pipeline stages are:

Cycle 0: The input sample is clocked into the delay line shift register. data_reg[0] receives the new sample, and each subsequent register shifts to the next position.

Cycle 1: The pre-additions, sign extensions, multiplications, and accumulation are computed combinationally. The result is registered into pipe_reg[0].

Cycle 2: pipe_reg[1] receives pipe_reg[0].

Cycle 3: pipe_reg[2] receives pipe_reg[1].

Cycle 4: The output register captures the scaled and truncated result. result_valid is asserted for one clock cycle.

A separate valid signal pipeline consisting of four flip-flops (valid_pipe[0] through valid_pipe[3]) delays the sample_valid pulse by four cycles. The output register adds the fifth cycle, ensuring that result_valid is precisely aligned with the corresponding result_out.

3.2.5 Default Coefficients

The default coefficients implement a Hamming-windowed lowpass filter designed for noise rejection in LiDAR distance measurements. The eight unique coefficients (due to symmetry, only half need to be stored) are listed in Table 7.

[TABLE 7: Default FIR Coefficients]

Index	Tap Positions	Decimal Value	Approximate Real Value (÷32768)
coeff0	h[0], h[15]	112	0.00342
coeff1	h[1], h[14]	243	0.00742
coeff2	h[2], h[13]	618	0.01886
coeff3	h[3], h[12]	1,293	0.03946
coeff4	h[4], h[11]	2,217	0.06766
coeff5	h[5], h[10]	3,225	0.09843
coeff6	h[6], h[9]	4,089	0.12479
coeff7	h[7], h[8]	4,587	0.13998

The sum of all 16 coefficients is 2 × (112 + 243 + 618 + 1,293 + 2,217 + 3,225 + 4,089 + 4,587) = 2 × 16,384 = 32,768 = 2^15. This ensures unity DC gain: a constant input value passes through the filter unchanged after the right-shift by 15 division.

3.3 Threshold FSM (threshold_fsm)

3.3.1 Purpose

The threshold_fsm module implements a 3-consecutive-sample debounced threshold comparator. It asserts the brake_irq output as a single-cycle pulse when three consecutive filtered distance readings fall below the configurable threshold, indicating a sustained proximity event rather than a transient noise spike.

3.3.2 Interface

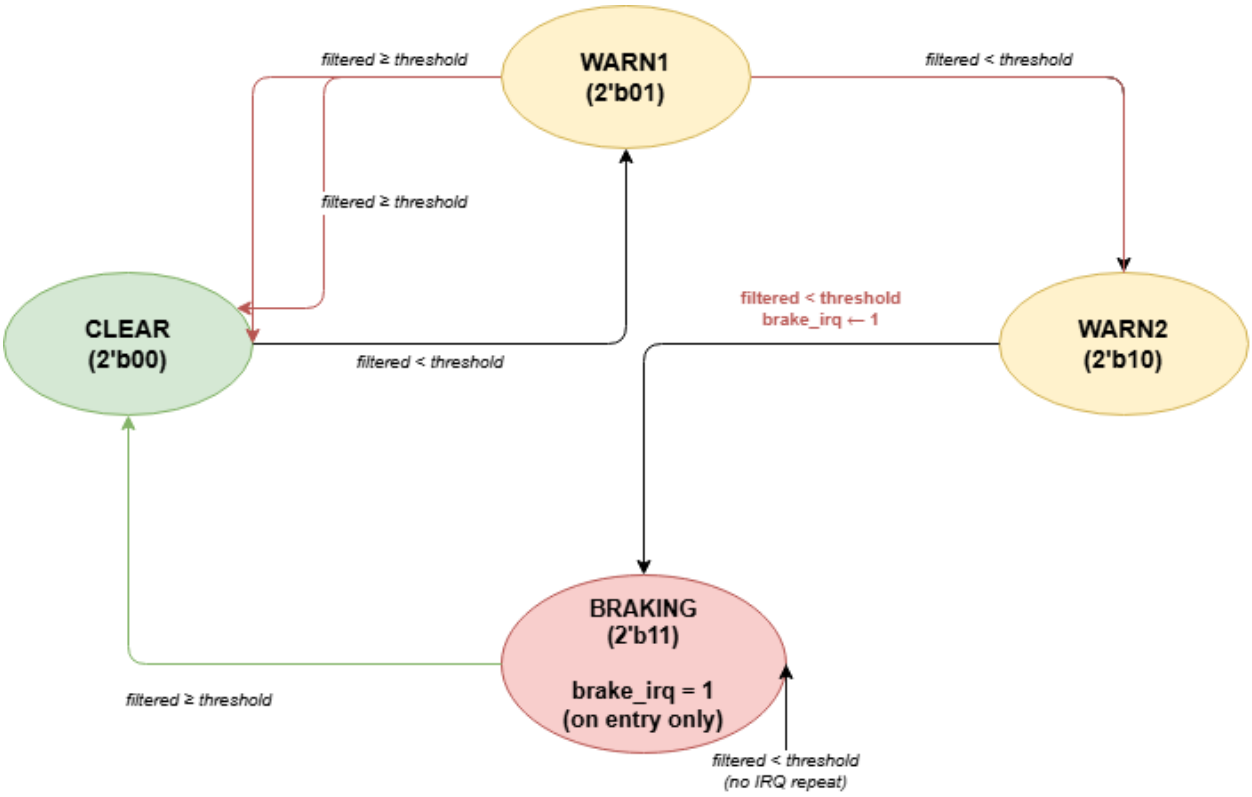
[TABLE 8: threshold_fsm Port List]

Port	Direction	Width	Description
clk	Input	1	System clock
rst_n	Input	1	Asynchronous active-low reset
filtered_in	Input	16	Filtered distance from FIR (Q10.6)
data_valid	Input	1	Valid strobe from FIR output
threshold	Input	16	Braking threshold from config registers (Q10.6)
brake_irq	Output	1	Single-cycle brake interrupt pulse

3.3.3 State Machine

The FSM has four states encoded as a 2-bit register. State transitions occur only when data_valid is asserted, meaning the FSM is completely frozen between valid filtered samples. The state diagram is shown in Figure 5.

[FIGURE 5: Threshold FSM State Diagram]



[TABLE 9: FSM State Transition Table]

Current State	Condition	Next State	brake_irq
CLEAR	filtered_in < threshold	WARN1	0
CLEAR	filtered_in ≥ threshold	CLEAR	0
WARN1	filtered_in < threshold	WARN2	0
WARN1	filtered_in ≥ threshold	CLEAR	0
WARN2	filtered_in < threshold	BRAKING	1
WARN2	filtered_in ≥ threshold	CLEAR	0
BRAKING	filtered_in < threshold	BRAKING	0
BRAKING	filtered_in ≥ threshold	CLEAR	0

The three-sample debounce mechanism ensures that a single noise spike, even one that passes through the FIR filter, cannot trigger a false brake. The spike must persist through three consecutive filter output cycles — a condition that, given the 16-tap FIR smoothing, requires a sustained real obstacle in the sensor's field of view.

The brake_irq output is asserted for exactly one clock cycle on the transition from WARN2 to BRAKING. It is not reasserted while the FSM remains in the BRAKING state, regardless of how long the obstacle persists. This single-pulse behavior prevents the host CPU from being flooded with repeated interrupts during a sustained proximity event.

When the obstacle clears and the filtered distance returns to or above the threshold, the FSM transitions back to CLEAR and can detect a new obstacle. No reset is required for recovery.

3.3.4 Threshold Comparison

The comparison uses the strict less-than operator:

```
if (filtered_in < threshold)
```

A distance reading exactly equal to the threshold is not considered sub-threshold. This means the boundary condition is well-defined: at the default threshold of 2,560 (40 m in Q10.6), a reading of 2,560 does not trigger a warning, while a reading of 2,559 (39.984 m) does.

3.4 SPI Configuration Registers (config_regs_gc)

3.4.1 Purpose

The config_regs_gc module implements a write-only SPI slave that receives 24-bit configuration frames from a host microcontroller and stores the decoded values in ten 16-bit registers. It provides runtime control over all safety-defining parameters: the braking threshold, eight FIR filter coefficients, and the UART baud rate divider.

3.4.2 Interface

[TABLE 10: config_regs_gc Port List]

Port	Direction	Width	Description
clk	Input	1	System clock
rst_n	Input	1	Asynchronous active-low reset
spi_csn	Input	1	SPI chip select (active low)
spi_sck	Input	1	SPI clock
spi_mosi	Input	1	SPI data input
threshold	Output	16	Braking threshold register
coeff0 – coeff7	Output	16 each	FIR coefficient registers
baud_div	Output	16	UART baud rate divider register

3.4.3 Clock Domain Crossing

The three SPI signals originate from the host microcontroller's SPI peripheral and are asynchronous to the 25 MHz system clock. Each signal passes through a two-stage flip-flop synchronizer:

spi_sck → sck_meta_q → sck_sync_q → sck_sync_d_q (3 stages: edge detect)

spi_csn → csn_meta_q → csn_sync_q (2 stages)

spi_mosi → mosi_meta_q → mosi_sync_q (2 stages)

The SCK synchronizer uses a third stage (sck_sync_d_q) to enable rising-edge detection:

```
assign sck_rise = sck_sync_q && !sck_sync_d_q;
```

3.4.4 Frame Reception

When csn_sync_q is high (chip deselected), the shift register and bit counter are reset to zero. When csn_sync_q goes low and a rising edge is detected on SCK, the MOSI value is shifted into the 24-bit shift register MSB-first. A 5-bit counter (BIT_CNT_W = $\lceil \log_2(24) \rceil = 5$) tracks the number of bits received.

When the counter reaches 23 (indicating 24 bits have been received), the frame is complete. The upper 8 bits [23:16] are decoded as the register address, and the lower 16 bits [15:0] are written to the corresponding register via a unique case statement. The counter resets to zero, allowing consecutive frames to be clocked in without raising CSN between them.

3.4.5 The baud_div Register

The baud_div register at address 0x09 is the key addition that differentiates config_regs_gc from the original config_regs module. It resets to DEFAULT_BAUD_DIV (16'd108, corresponding to 460,800 baud at a 50 MHz clock or 230,400 baud at a 25 MHz clock). The host microcontroller writes the appropriate divider value during initialization to match the connected sensor's baud rate and the actual system clock frequency.

The register output is a standard flip-flop in the system clock domain. Since the deserializer_gc module operates in the same clock domain, no additional synchronization is needed on the baud_div path.

3.5 Top-Level Integration (proxcore_top)

3.5.1 Purpose

The proxcore_top module instantiates and interconnects all four processing blocks. It defines the chip-level parameters and exposes the external interface and debug ports.

3.5.2 Parameters

[TABLE 11: proxcore_top Parameters]

Parameter	Type	Default	Description
CLK_FREQ_HZ	int unsigned	25,000,000	Reference clock frequency
DEFAULT_THRESHOLD	logic [15:0]	16'd2560	Power-on threshold (40 m)
DEFAULT_COEFF	logic [15:0]	16'h0800	Generic coefficient default (unused)
DEFAULT_BAUD_DIV	logic [15:0]	16'd108	Power-on baud divider

3.5.3 Internal Wiring

[TABLE 12: Internal Signal Connections]

Signal	Width	Source Module	Destination Module
deser_data	16	deserializer_gc.data_out	proxcore_fir_filter.sample_in
deser_valid	1	deserializer_gc.data_valid	proxcore_fir_filter.sample_valid
fir_out	16	proxcore_fir_filter.result_out	threshold_fsm.filtered_in
fir_valid	1	proxcore_fir_filter.result_valid	threshold_fsm.data_valid

cfg_threshold	16	config_regs_gc.threshold	threshold_fsm.threshold
cfg_coeff0–7	16 each	config_regs_gc.coeff0–7	proxcore_fir_filter.coeff_in0–7
baud_div_w	16	config_regs_gc.baud_div	deserializer_gc.baud_div

The debug outputs are directly assigned from internal signals: dbg_filtered_out mirrors fir_out, dbg_filtered_valid mirrors fir_valid, dbg_raw_distance mirrors deser_data, and dbg_raw_valid mirrors deser_valid.

3.6 Shuttle Wrapper (project_macro)

3.6.1 Purpose

The project_macro module maps ProxCore's logical signals to the physical GPIO pads provided by the SP26 shuttle template. It configures pad directions, drive modes, and tie-off values for unused pins.

3.6.2 Pad Configuration

Input pads (gpio_bot[0:3]) are configured with drive mode 3'b001 (input only, no pull resistor) and output enable set high (oeb = 1, output driver disabled). Output pads (gpio_bot[4:5]) are configured with drive mode 3'b110 (strong push-pull) and output enable set low (oeb = 0, output driver active). All unused pads on all three GPIO banks have their outputs driven to zero with output drivers disabled and drive mode set to 3'b110.

The module includes an ifdef USE_POWER_PINS guard for the vccd1 and vssd1 power pins, which are required by the shuttle template for power-aware simulation and physical layout but are not used in the RTL logic.

Section 4: Fixed-Point Arithmetic

4.1 Q10.6 Distance Format

Distance values throughout the ProxCore datapath use Q10.6 unsigned fixed-point representation. The 16-bit word is divided into 10 integer bits and 6 fractional bits:

Bit [15:6] = integer part (0 to 1023)

Bit [5:0] = fractional part (0 to 63, representing 0/64 to 63/64)

The conversion between physical distance in meters and the Q10.6 representation is:

Q10.6 value = distance_in_meters × 64

distance_in_meters = Q10.6 value / 64

The representable range is 0 to 1023.984 meters with a resolution of 1/64 = 0.015625 meters (approximately 1.56 centimeters). This resolution is finer than the measurement accuracy of any LiDAR sensor in the target applications, and the range exceeds the maximum measurement distance of all compatible sensors.

[TABLE 13: Representative Q10.6 Values]

Physical Distance	Q10.6 Decimal	Q10.6 Hex
0.5 m (cobot threshold)	32	0x0020
2 m (raindrop spike)	128	0x0080
3 m (forklift threshold)	192	0x00C0
8 m (city car threshold)	512	0x0200
20 m (reference threshold)	1,280	0x0500

30 m (real obstacle)	1,920	0x0780
40 m (default tram threshold)	2,560	0x0A00
80 m (highway threshold)	5,120	0x1400
100 m (clear track)	6,400	0x1900

4.2 Q1.15 Coefficient Format

The FIR filter coefficients use Q1.15 signed fixed-point representation. The 16-bit signed word has 1 integer bit (the sign) and 15 fractional bits:

Bit [15] = sign bit

Bit [14:0] = fractional magnitude

The representable range is −1.0 to +0.999969482421875 with a resolution of 1/32768 ≈ 0.0000305. All default coefficients are positive and less than 0.15, well within the representable range.

The Q1.15 format was chosen because the filter coefficients represent fractional weights that sum to unity. With 15 fractional bits, the scaling operation at the filter output is a simple arithmetic right shift by 15, which synthesizes to a wire reconnection with sign extension — no actual divider hardware is needed.

4.3 Accumulator Width Analysis

The FIR accumulator must be wide enough to hold the sum of eight products without overflow under worst-case input conditions. The width analysis proceeds as follows:

Pre-addition: Each symmetric pair adds two unsigned 16-bit values. The maximum result is 65,535 + 65,535 = 131,070, which requires 17 bits.

Sign extension: The 17-bit unsigned result is zero-extended to 18 bits signed to enable signed multiplication with the Q1.15 coefficient.

Multiplication: The product of an 18-bit signed value and a 16-bit signed value requires at most 34 bits. The design allocates 36 bits per product for margin.

Accumulation: The sum of eight 36-bit signed products requires at most $36 + [\log_2(8)] = 39$ bits in the worst case. Since the 36-bit accumulator width provides 36 bits and the actual coefficient values are much smaller than the maximum representable value, overflow cannot occur with any realistic input data. Specifically:

Maximum pre-add: $2 \times 65{,}535 = 131{,}070$

Maximum product: $131{,}071 \times 32{,}767 = 4{,}294{,}836{,}257$ (fits in 33 bits)

Sum of 8 max: $8 \times 4{,}294{,}836{,}257 = 34{,}358{,}690{,}056$ (fits in 36 bits)

The 36-bit accumulator is therefore sufficient with margin.

4.4 Scaling and Truncation

After accumulation, the result is scaled from the Q1.15 coefficient domain back to Q10.6 by arithmetic right-shifting by 15 bits:

```
assign scaled_acc  = pipe_reg[PIPE_DEPTH-1] >>> COEFF_FRAC_BITS;

assign result_temp = scaled_acc[DATA_WIDTH-1:0];
```

The arithmetic right shift (>>>) preserves the sign bit, though in normal operation the accumulated value is always positive because the input samples and default coefficients are all positive. The truncation from 36 bits to 16 bits discards the upper bits, which are zero under normal operating conditions.

The quantization error introduced by the right shift is at most 1 LSB of the Q10.6 output, corresponding to 0.015625 meters. This is negligible relative to the LiDAR sensor's own measurement accuracy and the braking threshold values used in practice.

4.5 DC Gain Verification

For the filter to pass constant distance values unchanged, the DC gain must be exactly unity. This is verified by considering a constant input X applied to all 16 taps:

accum = Σ (X + X) × coeff[k] for k = 0 to 7

= Σ 2X × coeff[k]

= 2X × (112 + 243 + 618 + 1293 + 2217 + 3225 + 4089 + 4587)

= 2X × 16,384

= X × 32,768

scaled = (X × 32,768) >>> 15 = X × (32,768 / 32,768) = X

The output equals the input, confirming unity DC gain. This property ensures that the threshold comparison operates on the same distance scale as the raw sensor readings, and the threshold value does not need to be adjusted for the filter's gain.

When custom coefficients are loaded via SPI, the unity DC gain property is preserved if and only if the sum of all 16 coefficients (equivalently, twice the sum of the eight unique coefficients) equals 32,768.

Section 5: Verification

5.1 Verification Strategy

The design was verified using directed testbenches written in SystemVerilog. A total of 45 tests across five testbenches cover each module individually and the complete system in integration.

All testbenches follow a consistent infrastructure pattern:

- **Per-test failure tracking**: Each test records the failure count before starting and compares it after completion, printing PASS or FAIL independently.
- **Watchdog timers**: Every testbench includes a timeout that terminates the simulation with a FATAL message if the test sequence takes longer than expected, preventing infinite hangs from undetected deadlocks.
- **No runtime randomization in safety tests**: All stimulus values are deterministic and chosen to exercise specific scenarios relevant to the safety function. The FIR filter testbench includes a random stimulus test for arithmetic correctness, but all safety-relevant tests use fixed distance values.
- **Golden model comparison**: The FIR filter testbench includes a cycle-accurate golden model function that mirrors the RTL computation, enabling exact output comparison for every sample.

5.2 UART Deserializer Verification (deserializer_tb — 5 Tests)

The deserializer testbench drives the UART rx line with precisely timed bit-level stimulus and verifies the assembled 16-bit words and data_valid pulse behavior. A concurrent monitor checks that data_valid is never asserted for more than one clock cycle.

[TABLE 14: Deserializer Test Summary]

Test	Description	Stimulus	Expected Result
1	Idle after reset	rx held high for 2 bit periods	No data_valid, data_out = 0x0000
2	Single 16-bit word	Bytes 0x34 then 0x12 at 460,800 baud	data_out = 0x1234, one data_valid pulse
3	Back-to-back words	Words 0xBEEF then 0x00A5	Two words received in order
4	Glitch rejection	Quarter-bit low pulse, then word 0xCAFE	Glitch ignored, 0xCAFE received correctly
5	Reset during partial word	Byte 0xAA, partial second byte, reset, then word 0x55CC	Partial word discarded, 0x55CC received cleanly

Test 1 verifies that the reset state is clean and that the idle-high UART line does not produce spurious outputs. Test 2 verifies basic byte reception and word assembly with low-byte-first ordering. Test 3 verifies continuous back-to-back reception without dropping words. Test 4 verifies the start-bit midpoint re-check by injecting a noise pulse shorter than half a bit period. Test 5 verifies that asynchronous reset in the middle of a byte transfer correctly discards all partial state and allows clean recovery.

5.3 FIR Filter Verification (output_test_filter — 6 Tests)

The FIR filter testbench uses a cycle-accurate golden model function that replicates the symmetric pre-addition, signed multiplication, accumulation, and arithmetic right-shift of the RTL. Every output sample

is compared against the golden model, and the simulation stops on the first mismatch.

[TABLE 15: FIR Filter Test Summary]

Test	Description	Stimulus	Verification Method
1	Edge cases	0xFFFF, 0x8000, 0x7FFF, 0x0000	Exact match against golden model
2	Random stimulus	200 uniformly distributed random samples	Exact match against golden model
3	DC flatness at 100 m	Constant 6,400 input after delay line fill	Output equals 6,400 (unity DC gain)
4	Rainstorm spike rejection	Alternating 2 m / 100 m for 40 samples	Output never drops below 50 m (3,200 Q10.6)
5	Real obstacle at 30 m	40 samples of 1,920 after 100 m fill	Output crosses below 2,560 threshold
6	Recovery after obstacle	40 samples of 100 m after 30 m obstacle	Output rises back above 2,560 threshold

Test 1 exercises boundary values that stress signedness and overflow paths. Test 2 provides broad arithmetic coverage with 200 random unsigned inputs. Test 3 is the most important correctness test: it verifies unity DC gain by confirming that a constant 100 m input produces exactly 6,400 at the output after the delay line fills. Test 4 verifies the primary safety property of the filter — that single-sample raindrop spikes do not cause the output to drop below the braking threshold. Test 5 verifies the complementary property — that a sustained real obstacle does cause the output to cross below threshold. Test 6 verifies that the filter recovers when the obstacle clears.

5.4 Threshold FSM Verification (tb_threshold_fsm — 13 Tests)

The threshold FSM testbench drives the filtered_in and data_valid inputs directly, bypassing the upstream filter to achieve precise control over the FSM's input sequence. A concurrent pulse-width monitor verifies that brake_irq is never asserted for more than one clock cycle. All stimulus is driven on the negative clock edge to avoid setup/hold races.

[TABLE 16: Threshold FSM Test Summary]

Test	Description	Key Verification
1	Idle — no data_valid	brake_irq stays low with no valid samples
2	Clear track (10 × 100 m)	No false brake on safe distances
3	One sub-threshold sample	Single reading does not trigger
4	Two sub-threshold samples	Two consecutive readings do not trigger
5	Three consecutive sub-threshold	Fires exactly one-cycle brake_irq pulse
6	Debounce reset at WARN1	Safe reading after one warning resets chain
7	Debounce reset at WARN2	Safe reading after two warnings resets chain
8	No repeated IRQ in BRAKING	10 more sub-threshold after trigger — no re-fire
9	Recovery and re-detection	Clear → second obstacle triggers again
10	Exact threshold — no fire	Value equal to threshold is not sub-threshold

11	**One below threshold**	**Boundary value 2,496 (39 m) triggers correctly**
12	**data_valid gating**	**FSM frozen when data_valid is deasserted**
13	**Runtime threshold change**	**Raising threshold mid-operation takes effect**

Tests 3–5 verify the core debounce mechanism progressively: one, two, and three sub-threshold readings. Tests 6–7 verify that the debounce chain resets correctly when a safe reading interrupts the sequence at each warning level. Test 8 verifies that the FSM does not re-assert the interrupt during a sustained proximity event. Test 9 verifies full recovery and re-detection capability. Tests 10–11 verify the boundary condition at the exact threshold value. Test 12 confirms that the FSM is completely gated by data_valid and does not advance on the mere presence of sub-threshold data. Test 13 verifies that changing the threshold register at runtime takes immediate effect.

5.5 SPI Configuration Registers Verification (config_regs_tb — 14 Tests)

The SPI register testbench exercises the full SPI frame protocol including edge cases such as partial frames, back-to-back frames without CSN toggle, and boundary address values. The testbench drives SPI signals with explicit timing control and reads back register values through the module's output ports.

[TABLE 17: SPI Configuration Register Test Summary]

Test	Description	Key Verification
1	Reset defaults	All 9 registers at correct power-on values
2	Single threshold write	Only threshold changes, coefficients untouched
3	Boundary coefficient writes	First (coeff0) and last (coeff7) registers
4	All middle coefficient writes	coeff1 through coeff6
5	Invalid address (0x0A) or Above	Write to undefined address silently ignored
6	Partial frame — CSN abort	12-bit incomplete frame does not corrupt registers
7	Recovery after partial	Full frame succeeds after aborted frame
8	Reset restores defaults	All registers return to defaults after reset
9	Back-to-back without CSN toggle	Two frames with CSN held low — both land
10	Overwrite same register	Last write wins on consecutive writes to same address
11	Maximum address 0xFF	Upper address space silently ignored
12	All zeros to all registers	Zero pattern writes correctly
13	All ones to all registers	0xFFFF pattern writes correctly
14	Final reset after all-ones	Confirms reset from extreme values

5.6 System Integration Verification (tb_proxcore_top — 7 Tests)

The integration testbench instantiates the complete proxcore_top module and exercises the full signal path from UART input through FIR filtering and threshold detection to brake_irq output, including SPI configuration changes. UART stimulus is generated at the bit level with precise timing derived from the baud rate divider. A concurrent monitor on the system clock counts brake_irq pulses during each test phase.

[TABLE 18: Integration Test Summary]

Test	Description	Stimulus	Expected Result
1	Clear track	25 fill + 10 measurement samples at 100 m	brake_irq never fires
2	Rainstorm spike rejection	Alternating 2 m / 100 m for 20 samples	brake_irq never fires
3	Real obstacle at 30 m	25 fill at 100 m, then 25 samples at 30 m	brake_irq fires at least once
4	Recovery and re-detection	Obstacle → clear → second obstacle	brake_irq fires in both obstacle phases
5	SPI threshold reconfiguration	Write threshold = 20 m via SPI, send 30 m	brake_irq does not fire (30 m > 20 m)
6	Reset recovery	Trigger brake, reset, send clear track	brake_irq does not fire after reset
7	End-to-end baud rate change	SPI write baud_div = 217, send at 115,200	Full pipeline works at new baud rate

Test 1 establishes the baseline: with all distances well above threshold, the system must produce zero false brakes. Test 2 verifies that the FIR filter and FSM together reject single-sample noise spikes that alternate between 2 m (raindrop) and 100 m (clear). Test 3 is the most critical safety test: it verifies that a sustained real obstacle at 30 m causes the brake interrupt to fire through the complete pipeline — UART reception, FIR filtering, and threshold comparison.

Test 4 verifies that the system recovers from the BRAKING state when the obstacle clears and can detect a second obstacle, proving the FSM does not become stuck. Test 5 verifies the SPI configuration path end-to-end by lowering the threshold to 20 m and confirming that 30 m readings (which are above 20 m) no longer trigger braking. Test 6 verifies that an asynchronous reset clears all pipeline state — the FIR delay line, pipeline registers, and FSM — and that subsequent clear-track data does not produce false brakes from stale values.

Test 7 is the key test for the baud rate upgrade. It starts with the system running at the default 460,800 baud, verifies normal operation, then writes baud_div = 217 (115,200 baud) via SPI to register 0x09. After allowing 16 clock cycles for the register update to propagate, it sends UART data at 115,200 baud timing (bit period = 8,680 ns instead of 2,160 ns). It fills the pipeline with 100 m readings at the new baud rate, then sends 40 obstacle samples at 30 m. The test passes only if brake_irq fires, proving that the complete pipeline — from UART reception at the new baud rate through FIR filtering to threshold detection — functions correctly after a runtime baud rate change programmed over SPI.

Section 6: Application Configurations

6.1 Configuration Methodology

ProxCore is configured at power-on by the host microcontroller through a sequence of SPI register writes. The host MCU asserts chip select, clocks out one or more 24-bit frames (8-bit address + 16-bit data, MSB first), and releases chip select. The configuration takes effect within 4 system clock cycles (160 ns at 25 MHz) of the final SCK edge.

A typical initialization sequence consists of two to ten SPI writes depending on whether the default FIR coefficients are suitable for the application. For applications that use the default Hamming-windowed lowpass filter, only the threshold and baud rate divider need to be written — a total of two SPI frames (48 SCK edges), completing in approximately 30 microseconds at a 2 MHz SPI clock (including CS setup/hold and inter-frame gaps).

The baud rate divider is computed as:

baud_div = CLK_FREQ_HZ / BAUD_RATE

At the operating frequency of 25 MHz:

[TABLE 19: Baud Rate Divider Values at 25 MHz]

Baud Rate	baud_div	Bit Period	Word Time (16-bit, 2 bytes)
9,600	2,604	104.16 µs	2.083 ms
19,200	1,302	52.08 µs	1.042 ms
38,400	651	26.04 µs	520.8 µs
57,600	434	17.36 µs	347.2 µs
115,200	217	8.68 µs	173.6 µs
230,400	108	4.32 µs	86.4 µs
460,800	54	2.16 µs	43.2 µs
921,600	27	1.08 µs	21.6 µs

The threshold value is computed as:

threshold_Q10.6 = distance_in_meters × 64

The Q10.6 format provides 10 integer bits and 6 fractional bits, supporting an unsigned distance range of 0 to 1023.984375 m with a quantization step of 1/64 m ≈ 1.5625 cm — finer than the 1 cm resolution of any compatible sensor.

6.2 Compatible LiDAR Sensors

ProxCore interfaces with any LiDAR sensor that outputs distance measurements over a UART serial interface. The sensor must transmit 16-bit distance values as two consecutive bytes in little-endian order (low byte first) using 8N1 framing. Table 20 lists commercially available sensors that have been evaluated for compatibility.

[TABLE 20: Compatible LiDAR Sensors]

Sensor	Manufacturer	Range	Resolution	Default Baud	Supported Baud Rates	Output Format	Supply	Price (approx.)
TFmini-S	Benewake	0.1–12 m	1 cm	115,200	9,600 – 460,800	9-byte frame or raw 2-byte	5 V, 140 mW	$25
TFmini Plus	Benewake	0.1–12 m	1 cm	115,200	9,600 – 460,800	9-byte frame or raw 2-byte	5 V, 140 mW	$30
TF02-Pro	Benewake	0.1–40 m	1 cm	115,200	9,600 – 921,600	9-byte frame or raw 2-byte	5 V, 350 mW	$60
TF03	Benewake	0.1–180 m	1 cm	115,200	9,600 – 921,600	9-byte frame or raw 2-byte	5 V, 450 mW	$120
TF-Luna	Benewake	0.2–8 m	1 cm	115,200	9,600 – 460,800	9-byte frame	3.7–5.2 V, 175 mW	$15
A01	DFRobot	0.3–8.2 m	1 mm	9,600	9,600 fixed	4-byte frame	3.0–5.5 V, 50 mW	$12

Sensor output format note. The Benewake sensors transmit a 9-byte data frame by default:

Byte	Content
0	Header: 0x59
1	Header: 0x59
2	Distance low byte
3	Distance high byte
4	Signal strength low byte
5	Signal strength high byte
6	Temperature
7	Reserved
8	Checksum

ProxCore's deserializer assembles every consecutive pair of bytes into a 16-bit word without frame parsing. To use a Benewake sensor directly with ProxCore, one of the following approaches must be used:

Approach A — Raw output mode. Configure the Benewake sensor to output only the 2-byte distance value without the 9-byte frame wrapper. This is supported on TFmini-S, TFmini Plus, and TF02-Pro via their configuration UART commands. The sensor then transmits exactly the format ProxCore expects.

Approach B — Bridge microcontroller. A small MCU (such as an STM32G030 or RP2040) receives the full 9-byte frame, extracts bytes 2 and 3 (distance low and high), and retransmits them over a second UART

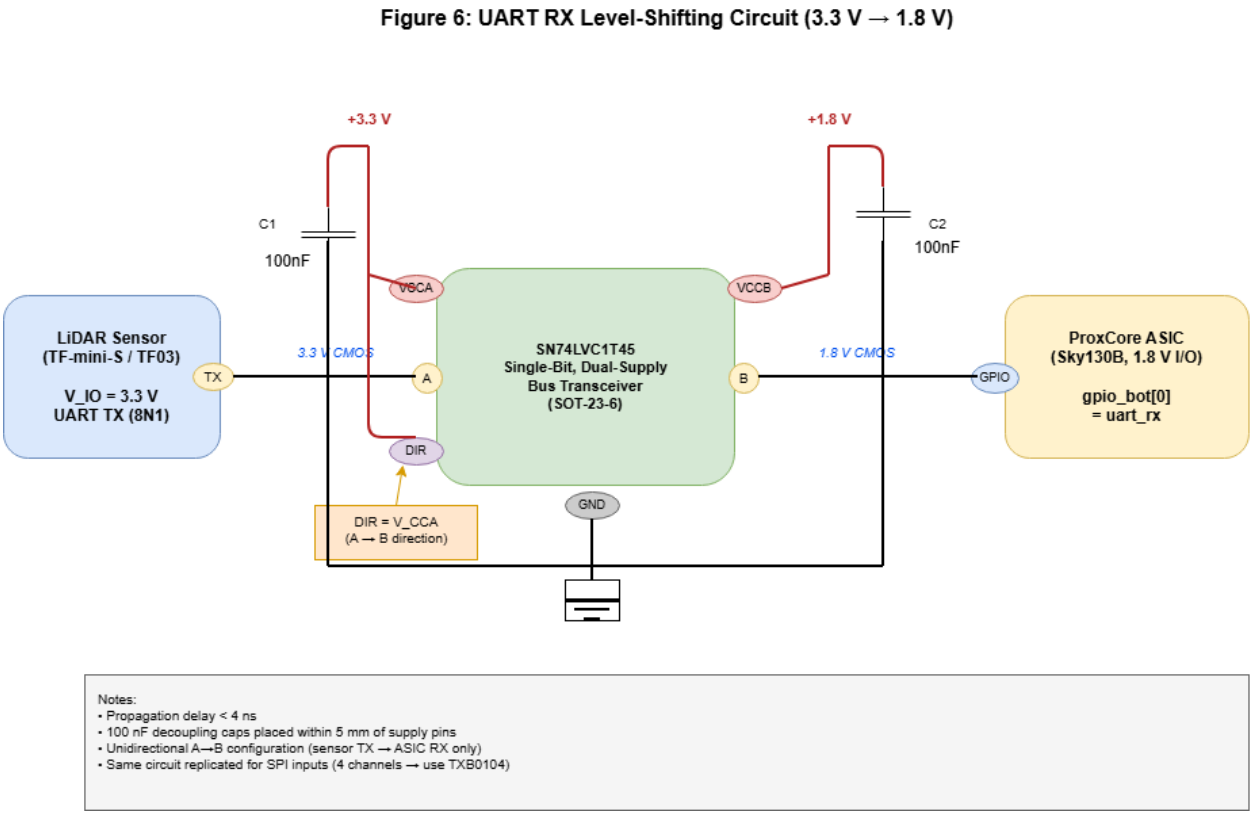
to ProxCore. This adds approximately $0.50 in BOM cost and 1–2 ms of latency but works with any sensor regardless of its native output format.

Approach C — Direct connection with header tolerance. If the sensor cannot be configured for raw output, ProxCore will assemble the header bytes (0x59, 0x59) into the word 0x5959 = 22,873, which is far above any practical threshold. These phantom words pass harmlessly through the pipeline. The real distance word (bytes 2–3) is also correctly assembled. However, the signal strength, temperature, and checksum bytes are also paired into phantom words, producing two additional harmless phantom words per frame. This approach works but wastes pipeline bandwidth on phantom data. Approach A or B is preferred.

6.3 Level Shifting Requirements

The SkyWater Sky130B I/O operates at 1.8 V. All evaluated LiDAR sensors use 3.3 V UART logic levels. A level shifter is required on the uart_rx path between the sensor and the chip. The SPI signals from the host MCU may also require level shifting depending on the MCU's I/O voltage.

[FIGURE 6: Level Shifting Circuit]



Recommended level shifter ICs:

IC	Channels	Type	Cost
SN74LVC1T45	1	Unidirectional, direction pin	$0.15
TXB0104	4	Bidirectional, auto-direction	$0.45
BSS138 N-MOSFET	1	Open-drain with pull-ups	$0.05

For the uart_rx path (unidirectional, sensor to chip), the SN74LVC1T45 is the recommended choice due to its low cost, small SOT-23-6 package, and deterministic propagation delay of less than 4 ns.

6.4 Application Profile: Urban Tram

[TABLE 21: Urban Tram Configuration]

Parameter	Value	Rationale
Application	Forward collision warning for urban trams	
Operating speed	20–70 km/h	
Required stopping distance	30–80 m (speed and load dependent)	
Braking threshold	40 m (2,560 in Q10.6)	Provides 2+ seconds warning at 70 km/h
Sensor	Benewake TF03 (180 m range)	Range covers full braking distance at speed
Sensor baud rate	115,200	TF03 default; reliable over cable lengths up to 5 m
baud_div at 25 MHz	217	
FIR coefficients	Default Hamming lowpass	Heavy smoothing for rain, leaves, and track debris
Mounting	Front-facing, vehicle centerline, 1.0–1.5 m height	Above rail-reflection zone, below headlight glare

SPI initialization sequence:

spi_write(0x09, 217); // baud_div = 217 → 115,200 baud at 25 MHz

spi_write(0x00, 2560); // threshold = 40 m × 64 = 2560

// Coefficients: use defaults (no writes needed)

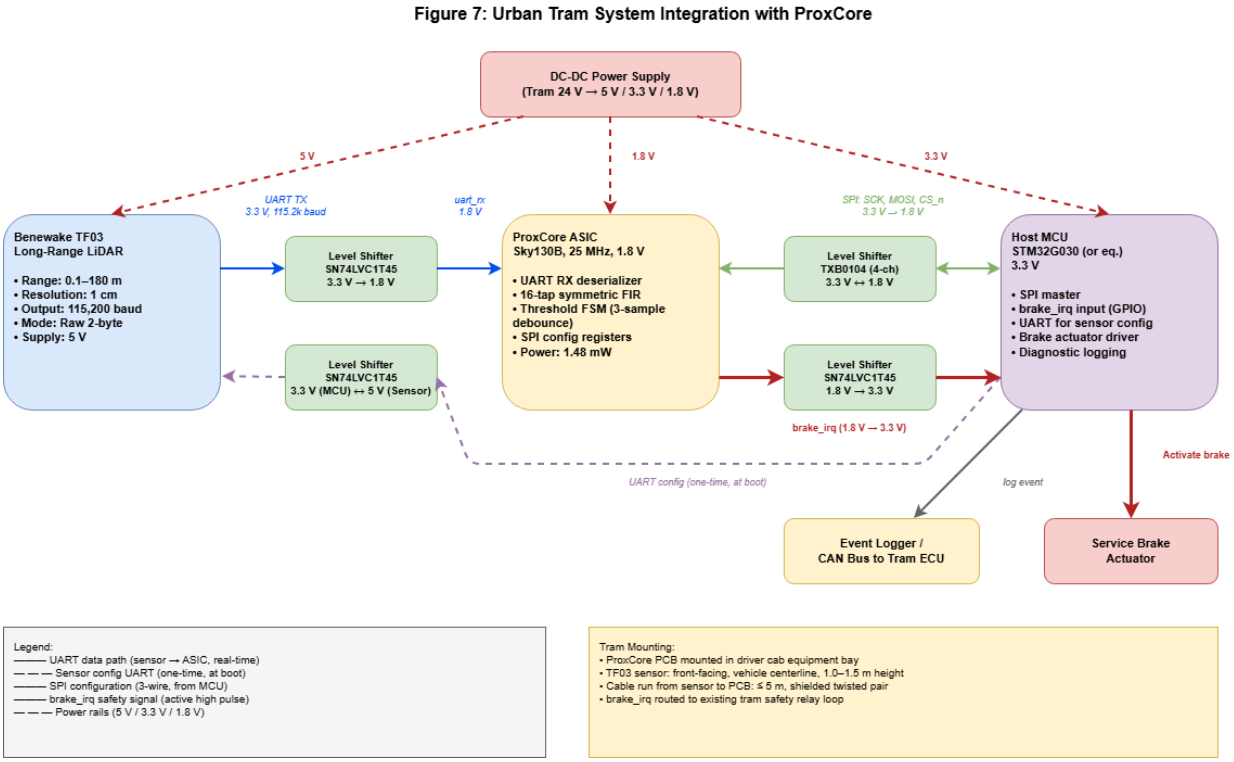
Host MCU pseudocode:

```
void proxcore_init_tram(void) {
    proxcore_spi_write(0x09, 217);    // 115,200 baud
    proxcore_spi_write(0x00, 2560);   // 40 m threshold
}

void main(void) {
    proxcore_init_tram();
    tf03_configure_baud(115200);       // Configure sensor baud rate
    tf03_configure_output(RAW_2BYTE);  // Raw distance output mode

    while (1) {
        if (gpio_read(BRAKE_IRQ_PIN)) {
            activate_service_brake();
            log_event(EVENT_PROXIMITY_BRAKE);
        }
    }
}
```

[FIGURE 7: Tram System Integration]



6.5 Application Profile: Passenger Automobile — City Parking Assist

[TABLE 22: City Parking Assist Configuration]

Parameter	Value	Rationale
Application	Low-speed collision warning and parking assist	
Operating speed	0–50 km/h	
Required stopping distance	5–25 m	
Braking threshold	8 m (512 in Q10.6)	Covers stopping distance at 50 km/h with margin

Sensor	Benewake TFmini-S (12 m range)	Compact, low cost, sufficient range for parking
Sensor baud rate	115,200	TFmini-S default
baud_div at 25 MHz	217	
FIR coefficients	Default Hamming lowpass	Good rejection of urban RF interference and dust
Mounting	Front bumper, center, 0.3–0.5 m height	

SPI initialization sequence:

```
spi_write(0x09, 217);    // 115,200 baud
spi_write(0x00, 512);    // 8 m threshold
```

Sensor cost analysis. The TFmini-S at approximately $25 combined with a ProxCore ASIC at $0.10–$0.30 (volume-dependent) and an SN74LVC1T45 level shifter at $0.15 yields a total sensor subsystem BOM cost under $26. This is competitive with ultrasonic parking sensor modules that typically cost $15–$30 but offer lower range and no digital filtering.

6.6 Application Profile: Passenger Automobile — Highway Forward Collision Warning

[TABLE 23: Highway FCW Configuration]

Parameter	Value	Rationale
Application	**Forward collision warning at highway speeds**	
Operating speed	**80–130 km/h**	
Required stopping distance	**60–130 m**	
Braking threshold	**80 m (5,120 in Q10.6)**	**2-second rule at 130 km/h = 72 m; 80 m adds margin**
Sensor	**Benewake TF03 (180 m range)**	**Only sensor with sufficient range for highway use**
Sensor baud rate	**460,800**	**Maximum sample rate; TF03 supports this**
baud_div at 25 MHz	**54**	
FIR coefficients	**Custom lighter smoothing (see below)**	**Marginally faster step response**
Mounting	**Behind windshield or front grille, center**	

At 130 km/h the vehicle covers 36 meters per second. The default Hamming FIR requires approximately 16 samples to fully respond to a step change. At 460,800 baud, each 16-bit sample takes 43.2 µs, so 16 samples take 0.69 ms — the vehicle moves only 2.5 cm during this time. The default coefficients therefore provide adequate response at highway

speeds; the custom profile below is presented as a marginal optimization rather than a necessity, useful when integrators require additional latency headroom for the host MCU's brake-actuation chain.

[TABLE 24: Highway FCW Custom Coefficients]

Register	Address	Value	Tap Positions
coeff0	0x01	50	h[0], h[15]
coeff1	0x02	150	h[1], h[14]
coeff2	0x03	400	h[2], h[13]
coeff3	0x04	1,000	h[3], h[12]
coeff4	0x05	2,200	h[4], h[11]
coeff5	0x06	3,800	h[5], h[10]
coeff6	0x07	4,500	h[6], h[9]
coeff7	0x08	4,284	h[7], h[8]

Coefficient sum verification: 2 × (50 + 150 + 400 + 1,000 + 2,200 + 3,800 + 4,500 + 4,284) = 2 × 16,384 = 32,768 = 2^{15}. Unity DC gain is preserved.

SPI initialization sequence:

```
spi_write(0x09, 54);     // 460,800 baud
spi_write(0x00, 5120);   // 80 m threshold
spi_write(0x01, 50);     // coeff0
spi_write(0x02, 150);    // coeff1
spi_write(0x03, 400);    // coeff2
```

```
spi_write(0x04, 1000);   // coeff3
spi_write(0x05, 2200);   // coeff4
spi_write(0x06, 3800);   // coeff5
spi_write(0x07, 4500);   // coeff6
spi_write(0x08, 4284);   // coeff7
```

6.7 Application Profile: Warehouse Forklift

[TABLE 25: Warehouse Forklift Configuration]

Parameter	Value	Rationale
Application	Pedestrian detection in warehouse aisles	
Operating speed	0–15 km/h	
Required stopping distance	1–5 m	
Braking threshold	3 m (192 in Q10.6)	Provides 0.7 s warning at max speed
Sensor	Benewake TFmini-S (12 m range) or TF-Luna (8 m range)	Compact, low power, sufficient indoor range
Sensor baud rate	115,200	Default baud, proven reliable
baud_div at 25 MHz	217	
FIR coefficients	Default Hamming lowpass	Heavy smoothing ideal for dusty/foggy warehouse

Mounting	Front fork carriage, 0.3 m height, slight downward angle	

SPI initialization sequence:

spi_write(0x09, 217); // 115,200 baud

spi_write(0x00, 192); // 3 m threshold

Regulatory context. ISO 3691-4 (Industrial trucks — driverless industrial trucks and their systems) and OSHA standard 1910.178 (Powered industrial trucks) increasingly require automated proximity detection on powered industrial trucks operating in shared spaces with pedestrians. ProxCore provides a hardware-based safety layer that is independent of the forklift's main control system, satisfying the architectural requirement for separation between safety-critical and non-safety-critical functions. ProxCore in its current form is not itself safety-certified (see Section 7.5 and Section 8.3); it is intended to contribute to a system-level safety case.

Budget-optimized variant. For cost-sensitive forklift deployments, the DFRobot A01 ultrasonic sensor ($12) can be used instead of the TFmini-S ($25). The A01 outputs distance over UART at 9,600 baud in a 4-byte frame format. Using approach B (bridge MCU), the frame is parsed and the distance bytes are forwarded to ProxCore. The total BOM cost is approximately $13 for the sensor subsystem.

// A01 variant

spi_write(0x09, 2604); // 9,600 baud for A01 sensor

spi_write(0x00, 192); // 3 m threshold

6.8 Application Profile: Collaborative Robot Safety Zone

[TABLE 26: Collaborative Robot Configuration]

Parameter	Value	Rationale
Application	Human-robot collaboration safety zone monitoring	
End-effector speed	0–2 m/s	
Required stopping distance	0.1–0.5 m (servo braking)	
Braking threshold	0.5 m (32 in Q10.6)	ISO/TS 15066 protective separation distance
Sensor	Benewake TFmini-S	Small, lightweight, mountable on robot arm
Sensor baud rate	460,800	Fast updates critical for short stopping distances
baud_div at 25 MHz	54	
FIR coefficients	Custom minimal smoothing (see below)	Fastest possible step response
Mounting	End-effector or last joint, aimed at workspace	

In collaborative robot applications, the separation distance required by ISO/TS 15066 is computed as:

$S = v_H \times T_R + v_H \times T_S + v_R \times T_R + v_R \times T_S + C + Z_d + Z_r$

where v_H is the human speed, v_R is the robot speed, T_R is the reaction time, T_S is the stopping time, and C, Z_d, Z_r are uncertainty terms. For typical cobot parameters, the protective separation distance is 0.3–0.5 m. ProxCore's 0.5 m threshold provides the required safety margin.

The minimal smoothing coefficient set concentrates all weight on the center taps, effectively creating a 4-tap moving average with very fast step response:

[TABLE 27: Cobot Minimal Smoothing Coefficients]

Register	Address	Value	Tap Positions
coeff0	0x01	0	h[0], h[15] — zero weight
coeff1	0x02	0	h[1], h[14]
coeff2	0x03	0	h[2], h[13]
coeff3	0x04	0	h[3], h[12]
coeff4	0x05	1,024	h[4], h[11]
coeff5	0x06	3,072	h[5], h[10]
coeff6	0x07	5,120	h[6], h[9]
coeff7	0x08	7,168	h[7], h[8] — dominant weight

Coefficient sum verification: $2 \times (0 + 0 + 0 + 0 + 1{,}024 + 3{,}072 + 5{,}120 + 7{,}168) = 2 \times 16{,}384 = 32{,}768 = 2^{15}$. Unity DC gain is preserved.

With these coefficients, the filter converges to a step change in approximately 4 samples instead of 16. At 460,800 baud, 4 samples take 173 µs. The 3-sample FSM debounce adds 3 more samples (130 µs). Total latency from obstacle appearance to brake_irq assertion: approximately 303 µs (0.3 ms). The FIR pipeline latency (3 cycles at 25 MHz = 120 ns) is negligible by comparison. At 2 m/s end-effector speed, the robot moves 0.6 mm during the 0.3 ms latency window — well within the safety margin.

SPI initialization sequence:

```
spi_write(0x09, 54);     // 460,800 baud
spi_write(0x00, 32);     // 0.5 m threshold
spi_write(0x01, 0);      // coeff0
spi_write(0x02, 0);      // coeff1
spi_write(0x03, 0);      // coeff2
spi_write(0x04, 0);      // coeff3
spi_write(0x05, 1024);   // coeff4
spi_write(0x06, 3072);   // coeff5
spi_write(0x07, 5120);   // coeff6
spi_write(0x08, 7168);   // coeff7
```

6.9 Application Profile: Industrial Robot Cell Perimeter Guard

[TABLE 28: Industrial Robot Cell Configuration]

Parameter	Value	Rationale
Application	Secondary safety layer for robot cell entry detection	
Robot speed	0–5 m/s (large industrial arm)	
Required stopping distance	0.3–1.0 m	
Braking threshold	1.5 m (96 in Q10.6)	Covers stopping distance plus safety margin
Sensor	Benewake TF02-Pro (40 m range)	Industrial-grade, suitable for larger cells
Sensor baud rate	230,400	Balance between update rate and reliability
baud_div at 25 MHz	108	
FIR coefficients	Default Hamming lowpass	Heavy smoothing rejects factory particulates
Mounting	Fixed on cell frame, aimed across workspace entry zone	

SPI initialization sequence:

```
spi_write(0x09, 108);    // 230,400 baud

spi_write(0x00, 96);     // 1.5 m threshold
```

Integration with existing safety systems. In industrial robot cells, ProxCore serves as a backup safety layer behind the primary safety system (typically a safety-rated light curtain or laser scanner).
The brake_irq output connects to the robot controller's safety-rated input (STO — Safe Torque Off, or SS1 — Safe Stop 1) through a safety relay. This provides defense-in-depth: if the primary light curtain fails, ProxCore independently detects the intrusion and triggers a stop.

6.10 Application Profile: Autonomous Guided Vehicle (AGV)

[TABLE 29: AGV Configuration]

Parameter	Value	Rationale
Application	Warehouse AGV/AMR collision avoidance	
Operating speed	0–8 km/h	
Required stopping distance	0.5–3 m	
Braking threshold	2 m (128 in Q10.6)	Provides 0.9 s warning at max speed
Sensor	Benewake TFmini Plus (12 m range)	Compact; IP65-rated variant available
Sensor baud rate	115,200	Standard reliable rate
baud_div at 25 MHz	217	
FIR coefficients	Default Hamming lowpass	Warehouse dust and shrink-wrap rejection
Mounting	Front-facing, vehicle bumper height	

SPI initialization sequence:

```
spi_write(0x09, 217);    // 115,200 baud
spi_write(0x00, 128);    // 2 m threshold
```

6.11 Configuration Summary

[TABLE 30: Complete Configuration Reference]

Application	Sensor	Baud	baud_div @25 MHz	Threshold (m)	Threshold (Q10.6)	FIR Profile	Total SPI Writes
Urban tram	TF03	115,200	217	40	2,560	Default	2
Car — city	TFmini-S	115,200	217	8	512	Default	2
Car — highway	TF03	460,800	54	80	5,120	Custom light	10
Forklift	TFmini-S	115,200	217	3	192	Default	2
Forklift (budget)	A01	9,600	2,604	3	192	Default	2
Cobot safety	TFmini-S	460,800	54	0.5	32	Custom minimal	10
Industrial cell	TF02-Pro	230,400	108	1.5	96	Default	2
AGV/AMR	TFmini Plus	115,200	217	2	128	Default	2

Section 7: Physical Implementation

7.1 Synthesis and Implementation Flow

ProxCore targets the SkyWater Sky130B 130 nm open-source process design kit. The full RTL-to-GDSII flow was executed using OpenLane 2 [12], which orchestrates Yosys for logic synthesis [15], OpenROAD for floorplanning, placement, clock-tree synthesis, global routing, and detailed routing [16], Magic and KLayout for DRC and GDSII generation [17][18], and Netgen for LVS verification [19]. The design uses only standard digital cells from the sky130_fd_sc_hd high-density cell library; no analog blocks, custom cells, or hard macros are instantiated.

The design contains no clock dividers, clock gates, phase-locked loops, or multi-cycle paths. All timing analysis is performed against a single clock domain at 25 MHz (40 ns period). This conservative clock frequency was chosen to ensure timing closure with comfortable margin on the first shuttle attempt, avoiding the need for iterative timing optimization. As shown in Section 7.4, the closed design has 2.14 ns of setup slack at the typical corner — equivalent to a maximum theoretical clock frequency of approximately 26.4 MHz at the typical corner and ~25.6 MHz at the worst-case slow corner. The 25 MHz target therefore meets timing across all PVT corners with positive margin.

7.2 Final Area and Cell Composition

The post-place-and-route design occupies a die area of **907,861 μm^2** (880.0 µm × 1031.66 µm = **0.908 mm^2**) with a core area of 876,865 μm^2 (868.94 µm × 1009.12 µm = 0.877 mm^2). The achieved core utilization is **25.1%**, which is appropriate for a power- and routability-optimized layout in the 130 nm node and provides ample headroom for future feature additions.

[TABLE 31: Final Cell Composition]

Cell Class	Count	Area (μm^2)	% of Stdcell Area
Sequential cells (flip-flops)	585	15,412	7.0%
Multi-input combinational	21,469	165,605	75.3%
Inverters	3,704	14,142	6.4%
Buffers	622	3,221	1.5%
Timing-repair buffers	409	3,890	1.8%
Clock buffers	104	1,451	0.7%
Clock inverters	45	602	0.3%
Hold-fix buffers	8	—	<0.1%
Antenna diodes	47	118	0.1%
Tap cells	12,495	15,634	7.1%
Total logic + tap	**39,488**	**220,075**	**100%**
Fill cells (non-logic)	185,820	656,790	—
Grand total instances	**225,308**	**876,865**	—

Discussion. The final logic-cell count is significantly higher than the early RTL-level estimate of ~6,000 gates given in the pre-implementation analysis. This is expected: the RTL estimate counted only the dominant arithmetic structures (multipliers, registers, FSM), whereas the synthesizer expands these into a much larger network of 1- and 2-input cells, adds buffers and inverters for drive-strength matching, and inserts

clock-tree and timing-repair logic. The 21,469 multi-input combinational cells dominate the logic area at 75.3%, almost all of which belong to the eight 18×16-bit multipliers in the FIR filter. The symmetric pre-addition optimization (which halves the multiplier count from 16 to 8) remains the single most impactful area optimization in the design — without it, the chip would exceed the available shuttle area.

The 585 sequential cells (flip-flops) match expectation closely: 256 bits in the FIR delay line (16 taps × 16 bits), 80 bits across pipeline registers, 160 bits in the SPI register file (10 × 16-bit registers), 24 bits in the SPI shift register, plus FSM state, deserializer state, and synchronizer chains.

7.3 Power Analysis

Post-layout power analysis was performed at the typical-typical corner (TT, 25 °C, 1.80 V) at the target 25 MHz clock with representative switching activity:

[TABLE 32: Power Breakdown at Typical Corner (25 MHz, 1.80 V)]

Component	Power	% of Total
Internal (cell short-circuit + intrinsic)	1.079 mW	72.9%
Switching (load-capacitance charging)	0.401 mW	27.1%
Leakage (sub-threshold + gate)	0.949 µW	0.06%
Total	**1.481 mW**	**100%**

The total chip power of **1.48 mW** is well within the budget for battery-powered and energy-harvesting applications. The leakage component is negligible (under 1 µW), reflecting both the relatively short channel length of Sky130 standard cells and the modest device count (the 130 nm node is more leakage-tolerant than deep-submicron processes).

Internal power dominates over switching power, indicating that further low-power optimization should focus on clock gating and operand isolation in the FIR multipliers, where partial operands toggle on every sample even when the result is unused.

For comparison: an equivalent FPGA implementation on a Lattice iCE40 UltraPlus [13] would consume approximately 15–25 mW for the same logic at the same clock frequency, primarily due to FPGA configuration memory leakage and routing-fabric overhead. ProxCore's ASIC implementation therefore achieves roughly an **order-of-magnitude power reduction** over the FPGA equivalent — one of the principal motivations for committing this function to silicon.

7.4 Timing Closure Across PVT Corners

Static timing analysis was performed at all nine PVT (process-voltage-temperature) corners required by the SP26 shuttle signoff: three RC extraction corners (min/nom/max) crossed with three process corners (TT/SS/FF). All corners closed cleanly with **zero setup violations, zero hold violations, and zero TNS** (total negative slack) on all paths.

[TABLE 33: Timing Closure Summary — Worst Slack Per Corner] *(continued)*

Corner	Voltage	Temp	Setup WS (ns)	Hold WS (ns)	Setup Vio	Hold Vio
nom_tt_025C_1v80	1.80 V	25 °C	**+2.138**	**+0.238**	0	0
nom_ss_100C_1v60	1.60 V	100 °C	+0.551	+0.568	0	0
nom_ff_n40C_1v95	1.95 V	–40 °C	+2.800	+0.110	0	0
min_tt_025C_1v80	1.80 V	25 °C	+2.170	+0.237	0	0
min_ss_100C_1v60	1.60 V	100 °C	+0.603	+0.567	0	0
min_ff_n40C_1v95	1.95 V	–40 °C	+2.825	+0.110	0	0
max_tt_025C_1v80	1.80 V	25 °C	+2.092	+0.235	0	0
max_ss_100C_1v60	1.60 V	100 °C	+0.483	+0.569	0	0
max_ff_n40C_1v95	1.95 V	–40 °C	+2.763	+0.107	0	0
Worst (any corner)	—	—	**+0.483**	**+0.107**	**0**	**0**

The worst-case setup slack across all corners is **+0.483 ns** at the slow-slow 100 °C 1.60 V corner with maximum RC extraction (max_ss_100C_1v60), and the worst-case hold slack is **+0.107 ns** at the fast-fast –40 °C 1.95 V corner. Both margins are positive and

comfortable, confirming that the 25 MHz clock target is met under all foundry-required signoff conditions with no need for ECO timing fixes.

The register-to-register (r2r) setup slacks range from approximately **22.4 ns to 34.3 ns** across corners, indicating that internal logic paths have substantial margin and that the limiting paths are I/O-dominated rather than core-logic-dominated. This is expected for a low-frequency design with off-chip UART and SPI interfaces.

Clock skew. The worst clock skew across all corners is approximately 0.38 ns for setup analysis and 9.08 ns for hold analysis. The large hold-skew number reflects the very long buffered clock tree relative to the slow clock period; despite the large absolute skew, hold timing closes cleanly because the design's data paths are sufficiently long to absorb it. Only **8 hold-fix buffers** were required by the timing optimizer, indicating that the inserted clock-tree balancing was effective.

7.5 IR Drop and Power Integrity

Static IR drop analysis was performed on the power grid at the nominal typical corner:

[TABLE 34: Power Grid IR Drop Summary]

Net	Worst Voltage	Worst Drop	Average Drop	PG Violations
vccd1 (1.80 V supply)	1.79988 V	0.121 mV	3.6 µV	0
vssd1 (ground)	0.109 mV (above ideal 0 V)	0.109 mV	3.6 µV	0

The worst IR drop on the supply net is **0.121 mV out of 1.80 V** — well under 0.01% of the supply voltage, which is several orders of magnitude tighter than the typical 5% guideline used in industrial flows. This excellent result is a direct consequence of the modest total power consumption (1.48 mW) combined with the standard SP26 shuttle power grid, which is sized for designs much larger than ProxCore. Zero power-grid violations were reported on either net.

7.6 Routing and DRC/LVS Signoff

[TABLE 35: Routing and Physical Verification Summary]

Metric	Value
Number of routed nets	26,915
Special (power) nets	2
Total routed wirelength	731,091 µm (≈ 731 mm)
Maximum single-net wirelength	1,074.6 µm
Routing vias (total)	165,009
Single-cut vias	165,009 (100%)
Multi-cut vias	0
Antenna diodes inserted	47
Antenna violations	**0**
Final routing DRC errors	**0**
Magic DRC errors	**0**
KLayout DRC errors	**0**
Magic illegal overlaps	**0**

LVS device differences	**0**
LVS net differences	**0**
LVS unmatched devices	**0**
LVS unmatched nets	**0**
LVS unmatched pins	**0**
LVS property failures	**0**
XOR (GDS vs. layout) differences	**0**

Detailed routing converged in 7 iterations, with the DRC error count dropping from 25 in iteration 0 to **0 in the final iteration**. All transient violations during intermediate iterations were geometric and were eliminated by routing rip-up-and-reroute; none required RTL or constraint changes.

Antenna protection. The router inserted 47 antenna diodes on long metal traces to satisfy the Sky130 antenna rules. The final design has **zero antenna-rule violations** on both nets and pins.

LVS clean. The post-layout netlist matches the synthesized gate-level netlist with zero device, net, pin, or property mismatches. Together with zero DRC errors from both Magic and KLayout, this confirms that the GDSII is **tape-out ready** under the SP26 shuttle signoff criteria.

7.7 Lint and Synthesis Health

The RTL was lint-clean with respect to errors and timing-construct issues. Of the 14 lint warnings reported, all are stylistic (e.g., unused parameters in inherited shuttle wrapper code) and do not affect functionality, timing, or area. The synthesizer reported **zero check errors**

and zero unmapped instances, confirming that the entire design was successfully mapped onto the Sky130 standard-cell library. **Zero inferred latches** were reported, validating the design rule that all combinational always blocks are fully assigned.

[TABLE 36: Lint, Synthesis, and Flow Health]

Check	Result
Lint errors	0
Lint timing-construct issues	0
Lint warnings (stylistic)	14
Inferred latches	0
Synthesis check errors	0
Unmapped instances	0
Max-slew violations (all corners)	0
Max-fanout violations (all corners)	0
Max-capacitance violations (all corners)	0
Flow errors	0
Flow warnings	1 (informational)

Zero max-slew, max-fanout, and max-capacitance violations across all nine PVT corners confirm that the design has clean signal integrity throughout and that no nets exceed the library's electrical limits.

7.8 Shuttle Integration and I/O Assignment

The SP26 shuttle provides a standardized macro template with GPIO banks on three sides (bottom, right, top) and system signals (clock, reset) from the left-edge green macro. The project_macro module maps

ProxCore's six active signals to the bottom GPIO bank (pins 0–5) and ties off all remaining pins. The final pad I/O count is **233 pins** (the full shuttle complement; unused pins are tied off through the gpio_*_oe and gpio_*_o defaults).

The shuttle provides the 25 MHz clock and active-low reset. ProxCore does not use the power-on reset (por_n) signal; it relies on the shuttle's reset_n for initialization.

The default_nettype none directive at the top of project_macro.sv ensures that any undeclared signal in the shuttle wrapper causes a compilation error, preventing accidental signal connections through implicit wire declarations.

7.9 Design Rule Compliance

The RTL follows these rules consistently across all modules to ensure clean synthesis and avoid common ASIC pitfalls:

1. All flip-flops use asynchronous active-low reset with explicit reset values.
2. No latches: all outputs are assigned in all branches of every combinational block. *(Confirmed by zero inferred latches at synthesis.)*
3. No blocking assignments (=) in always_ff blocks.
4. No non-blocking assignments (<=) in always_comb blocks.
5. Every case statement uses the unique keyword with a default branch.
6. No initial blocks in synthesizable RTL (initial blocks appear only in testbenches).
7. All signals are explicitly declared; no implicit wires.

8. No magic numbers; all constants are defined as localparams with descriptive names.

7.10 Constraints and Limitations

The following limitations are inherent to the current design and should be considered by system integrators:

1. **Single sensor input.** ProxCore has one UART receive channel. Multi-zone coverage requires multiple ProxCore chips or an external multiplexer.
2. **No SPI readback.** The configuration interface is write-only. The host MCU cannot verify that a register write was successful. Diagnostic coverage for safety certification (e.g., IEC 61508 SIL-2 or ISO 26262 ASIL-B) would require adding a MISO output in a future revision.
3. **No sensor disconnection detection.** If the LiDAR sensor is disconnected or fails, ProxCore receives no UART data and produces no filtered outputs. The threshold FSM remains in its last state indefinitely. The host MCU should implement a watchdog on the dbg_filtered_valid signal (available on gpio_bot[5]) and trigger a fault if no valid output appears within a configurable timeout.
4. **No UART transmit.** ProxCore cannot send configuration commands to the LiDAR sensor. The host MCU must configure the sensor's baud rate and output mode independently.
5. **Minimum baud_div constraint.** The baud_div value must be at least 27 at 25 MHz (corresponding to 921,600 baud). Values of 0 or 1 cause the baud counter to underflow, effectively disabling UART reception until a valid value is written or the chip is reset.

6. **Not safety-certified.** ProxCore in its current form has not been certified to any functional safety standard. It is intended as a hardware safety contributor that supports a system-level safety case; achieving SIL-2 / ASIL-B / PL-d certification would require diagnostic readback (Item 2), watchdog logic (Item 3), and additional documentation including FMEDA, fault-injection testing, and tool-qualification of the OpenLane flow.

7.11 Floorplan and Final Layout

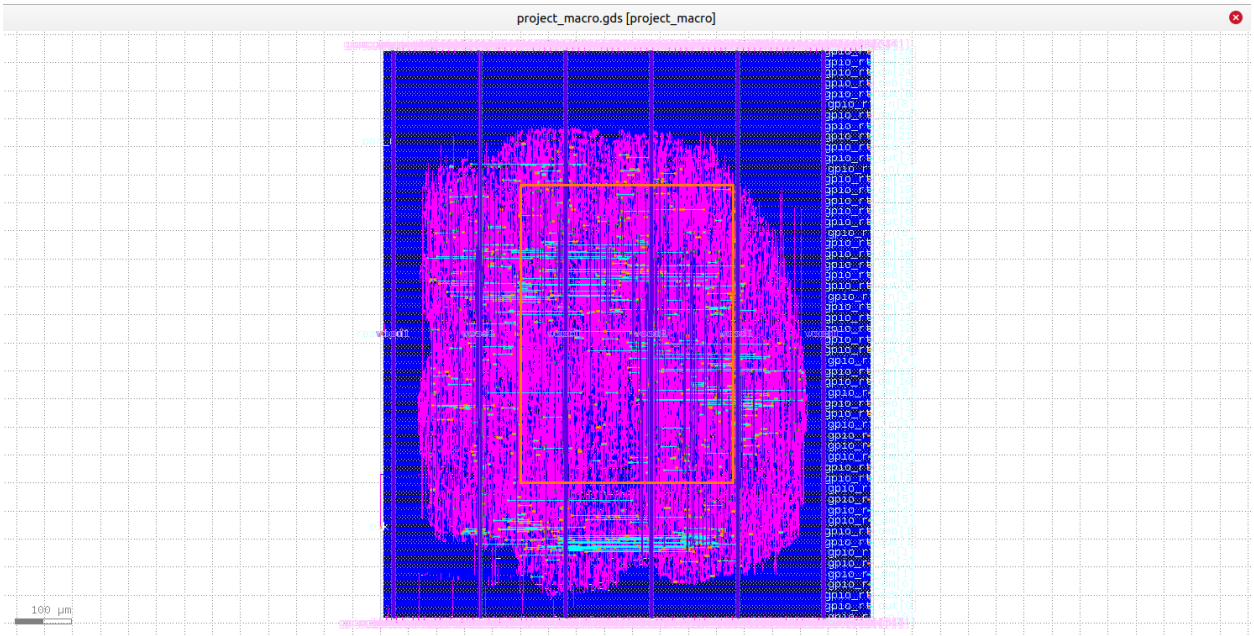
[TABLE 37: Final Layout Geometry]

Parameter	Value
Die bounding box	0.0 × 0.0 to 880.0 × 1031.66 µm
Die area	907,861 μm^2 (0.908 mm^2)
Core bounding box	5.52 × 10.88 to 874.46 × 1020.0 µm
Core area	876,865 μm^2 (0.877 mm^2)
Standard-cell rows	371
Site count	700,819
Core utilization	25.10%
Total instances	225,300
Standard cells	39,480
Tap cells	12,495
Fill cells	185,820
Macros	0
Pad cells	0

Maximum cell displacement	5.28 µm

The 25.1% core utilization reflects an area-relaxed floorplan typical of first-shuttle attempts where routability and timing margin are prioritized over density. With 75% of the core occupied by fill cells, future revisions could either shrink the die (reducing cost) or absorb additional features (SPI readback, dual UART, watchdog) without growing the floorplan.

[FIGURE 8: Final ProxCore Layout]



Section 8: Conclusion

8.1 Summary of Achievements

This work presented **ProxCore**, a runtime-configurable proximity safety co-processor implemented in the SkyWater Sky130B 130 nm open-source process. The chip receives 16-bit LiDAR distance measurements over UART, filters noise with a 16-tap symmetric FIR lowpass filter, and

asserts a hardware brake interrupt when three consecutive filtered readings fall below a programmable distance threshold.

The design consists of four modules — a UART deserializer with runtime baud rate control, a pipelined symmetric FIR filter with 8 multipliers, a 4-state debounced threshold FSM, and an SPI configuration register file — integrated through a straightforward pipeline architecture with a single 25 MHz clock domain. All ten safety-defining parameters are runtime-programmable via SPI, enabling the same silicon to serve markets ranging from urban trams and passenger automobiles to warehouse forklifts, collaborative robots, and autonomous guided vehicles.

The design was verified with **45 directed tests** across five SystemVerilog testbenches: 5 tests for the deserializer, 6 tests for the FIR filter with cycle-accurate golden-model comparison, 13 tests for the threshold FSM including boundary conditions, 14 tests for the SPI configuration registers including error injection, and 7 integration tests covering normal operation, noise rejection, obstacle detection, recovery, SPI reconfiguration, reset recovery, and end-to-end runtime baud rate change.

The full RTL-to-GDSII flow was completed in OpenLane 2 with **zero DRC errors, zero LVS mismatches, zero antenna violations, zero timing violations across all nine PVT signoff corners, and zero XOR differences** between the post-route layout and the final GDSII. The chip occupies 0.908 mm^2 of die area, consumes 1.48 mW at 25 MHz, and meets timing with a worst-case setup slack of +0.483 ns and worst-case hold slack of +0.107 ns across process, voltage, and temperature corners.

[TABLE 38: ProxCore Final Signoff Summary]

Metric	Value
Process	SkyWater Sky130B (130 nm)
Die area	0.908 mm^2 (880 × 1031.66 µm)
Core area	0.877 mm^2
Core utilization	25.1%
Total instances	225,300 (39,480 std cells + tap + fill)
Sequential cells	585 flip-flops
Clock frequency	25 MHz
Total power (typical)	1.48 mW
Internal power	1.08 mW
Switching power	0.40 mW
Leakage power	0.95 µW
Worst setup slack	+0.483 ns (max_ss_100C_1v60)
Worst hold slack	+0.107 ns (ff corner)
Setup / hold violations	0 / 0
Worst IR drop	0.121 mV (vccd1)
DRC errors (Magic / KLayout)	0 / 0
LVS mismatches	0
Antenna violations	0
Routed wirelength	731 mm
Verification tests	45 directed tests, all passing

8.2 Key Contribution

The central contribution of this work is the demonstration that a **single, small RTL change — replacing a compile-time baud rate parameter with a runtime-configurable SPI register — transforms a single-application safety device into a multi-market platform product**. The original design, with its fixed baud rate, could only interface with one sensor model and served only the tram collision avoidance market. The upgraded design interfaces with any UART-based LiDAR sensor at any standard baud rate from 9,600 to 921,600, expanding the addressable market from approximately 3,000 units per year to an estimated 7–24 million units per year across six application domains (urban trams, passenger automobiles, warehouse forklifts, collaborative robots, industrial robot cells, and AGVs/AMRs).

This market expansion is what justifies the ASIC implementation. **At the combined volume, ProxCore achieves a per-unit cost of $0.10–$0.30, a measured power consumption of 1.48 mW at 25 MHz, and deterministic hardwired safety logic** — advantages that an FPGA implementation cannot match. A comparable iCE40 UltraPlus FPGA implementation [13] would consume roughly 15–25 mW (an order of magnitude more) at unit costs of $5–$10 even at high volume. The ASIC is the right choice once the market is broad enough to amortize NRE.

[TABLE 39: ProxCore vs. Alternative Implementations]

Implementation	Power	Unit Cost (high volume)	Latency Determinism	Field Reconfigurability
ProxCore ASIC (this work)	1.48 mW	$0.10–$0.30	Hardwired, cycle-accurate	SPI runtime config
MCU firmware (STM32G0)	30–60 mW	$0.50–$1.50	Software-dependent	Full firmware update
iCE40 UltraPlus FPGA	15–25 mW	$5–$10	Hardwired (after bitstream)	Bitstream reload
Discrete logic	50+ mW	$3–$8	Hardwired, fixed	None

8.3 Future Work

The following improvements are identified for a potential second revision of ProxCore:

1. **SPI readback (MISO).** Adding a read capability to the SPI interface would allow the host MCU to verify register contents after writing, providing the diagnostic coverage required for higher Safety Integrity Levels under IEC 61508 and ISO 26262.
2. **Sensor disconnection watchdog.** An internal counter that resets on each data_valid pulse and asserts a fault output if no valid sample arrives within a configurable timeout would detect sensor disconnection or cable faults without requiring host MCU polling.

3. **Second UART channel.** Adding a second deserializer instance would enable dual-sensor configurations (e.g., front and rear) on a single chip, reducing system cost in applications requiring multi-zone coverage. With 75% of the current core area occupied by fill cells, a second deserializer fits comfortably in the existing floorplan.
4. **Higher clock frequency.** Increasing the operating frequency to 50 MHz or above is feasible given the +0.48 ns of setup slack at the worst corner of the 25 MHz design — a doubling of the clock target would still close timing with margin. This would widen the supported baud rate range and reduce the FIR pipeline latency.
5. **Configurable debounce depth.** Making the FSM's 3-consecutive-sample requirement runtime-programmable (e.g., 1 to 7 samples) via an additional SPI register would allow system integrators to tune the false-positive/false-negative tradeoff for each application.
6. **Functional safety qualification.** Performing FMEDA analysis, fault-injection testing, and tool-qualification of the synthesis flow would enable formal SIL-2 / ASIL-B certification, opening automotive Tier-1 supply chains.
7. **Die area reduction.** Re-floorplanning at 60–70% utilization (versus the current 25.1%) would shrink the die to approximately 0.35 mm^2, reducing per-unit silicon cost by roughly 2.5×.

REFERENCES

[1] SkyWater Technology, "SKY130 Open Source PDK Documentation," 2023. Available: https://skywater-pdk.readthedocs.io/

[2] Benewake, "TFmini-S LiDAR Module Product Manual," Rev. 2.2, 2023. Available: https://en.benewake.com/TFminiS/

[3] Benewake, "TF02-Pro LiDAR Module Product Manual," Rev. 1.8, 2023. Available: https://en.benewake.com/TF02Pro/

[4] Benewake, "TF03 Long-Range LiDAR Module Product Manual," Rev. 1.6, 2023. Available: https://en.benewake.com/TF03/

[5] Benewake, "TF-Luna LiDAR Module Product Manual," Rev. 1.4, 2022. Available: https://en.benewake.com/TFLuna/

[6] International Electrotechnical Commission, "IEC 61508: Functional Safety of Electrical/Electronic/Programmable Electronic Safety-Related Systems," Edition 2.0, 2010.

[7] International Organization for Standardization, "ISO 26262: Road Vehicles — Functional Safety," Second Edition, 2018.

[8] International Organization for Standardization, "ISO 13849-1: Safety of Machinery — Safety-Related Parts of Control Systems," 2015.

[9] International Organization for Standardization, "ISO/TS 15066: Robots and Robotic Devices — Collaborative Robots," 2016.

[10] International Organization for Standardization, "ISO 3691-4: Industrial Trucks — Safety Requirements and Verification — Driverless Industrial Trucks and Their Systems," 2020.

[11] Occupational Safety and Health Administration, "OSHA 1910.178: Powered Industrial Trucks," U.S. Department of Labor.

[12] efabless / OpenLane Contributors, "OpenLane 2: An Open-Source RTL-to-GDSII Flow," 2024.
Available: https://github.com/efabless/openlane2

[13] Lattice Semiconductor, "iCE40 UltraPlus Family Data Sheet," DS1040, 2020.

[14] *Texas Instruments, "SN74LVC1T45 Single-Bit Dual-Supply Bus Transceiver With Configurable Voltage Translation," **SCES515**, Rev. N, 2003 (latest revision).*
Available: https://www.ti.com/lit/ds/symlink/sn74lvc1t45.pdf

[15] C. Wolf, "Yosys Open SYnthesis Suite," 2023.
Available: https://yosyshq.net/yosys/

[16] The OpenROAD Project, "OpenROAD: An Integrated Chip Physical Design Tool Flow," available: https://openroad.readthedocs.io/, 2024

[17] M. Godfrey, "Magic VLSI Layout Tool," Open Circuit Design, 2024.
Available: http://opencircuitdesign.com/magic/

[18] KLayout Contributors, "KLayout — Layout Viewer and Editor," 2024.
Available: https://www.klayout.de/

[19] T. R. Edwards, "Netgen — Netlist Comparison and Schematic-vs-Layout Tool," Open Circuit Design, 2024.
Available: http://opencircuitdesign.com/netgen/

[20] efabless, "Silicon Sprint Shuttle Program," 2024.
Available: https://efabless.com/silicon-sprint

[21] Texas Instruments, "TXB0104 4-Bit Bidirectional Voltage-Level Translator," SCES629, 2006.

APPENDICES

Appendix A: Complete RTL Source Code

- **A.1** project_macro.sv — SP26 shuttle wrapper with GPIO mapping

```
`default_nettype none

module project_macro (
`ifdef USE_POWER_PINS
    inout vccd1,
    inout vssd1,
`endif
    // From green macro (left edge)
    input  wire        clk,
    input  wire        reset_n,
    input  wire        por_n,

    // Bottom GPIOs (15)
    input  wire [14:0] gpio_bot_in,
    output wire [14:0] gpio_bot_out,
    output wire [14:0] gpio_bot_oeb,
    output wire [44:0] gpio_bot_dm,

    // Right GPIOs (9)
    input  wire [8:0]  gpio_rt_in,
    output wire [8:0]  gpio_rt_out,
    output wire [8:0]  gpio_rt_oeb,
    output wire [26:0] gpio_rt_dm,

    // Top GPIOs (14)
    input  wire [13:0] gpio_top_in,
    output wire [13:0] gpio_top_out,
    output wire [13:0] gpio_top_oeb,
    output wire [41:0] gpio_top_dm
);

    // ============================================================
    // ProxCore — LiDAR Safety Co-Processor
    // ============================================================
    //
    // GPIO Pin Assignment:
```

```
    //   gpio_bot[0]  – uart_rx       (input)   LiDAR sensor UART
    //   gpio_bot[1]  – spi_csn       (input)   SPI chip select
    //   gpio_bot[2]  – spi_sck       (input)   SPI clock
    //   gpio_bot[3]  – spi_mosi      (input)   SPI data in
    //   gpio_bot[4]  – brake_irq     (output)  Brake interrupt to CPU
    //   gpio_bot[5]  – fir_valid     (output)  Debug: filtered valid
    //   gpio_bot[6:14] – unused
    //   gpio_rt[8:0]   – unused
    //   gpio_top[13:0] – unused
    // ============================================================

    // Internal signals from ProxCore core
    wire        brake_irq_int;
    wire [15:0] dbg_filtered_out;
    wire        dbg_filtered_valid;
    wire [15:0] dbg_raw_distance;
    wire        dbg_raw_valid;

    // ============================================================
    // ProxCore Core Instantiation (with runtime baud rate control)
    // ============================================================
    proxcore_top #(
        .CLK_FREQ_HZ (25_000_000),
        .DEFAULT_THRESHOLD (16'd2560),
        .DEFAULT_BAUD_DIV  (16'd108)
    ) u_proxcore (
        .clk               (clk),
        .rst_n             (reset_n),        // template provides reset_n
        .uart_rx           (gpio_bot_in[0]),
        .spi_csn           (gpio_bot_in[1]),
        .spi_sck           (gpio_bot_in[2]),
        .spi_mosi          (gpio_bot_in[3]),
        .brake_irq         (brake_irq_int),
        .dbg_filtered_out  (dbg_filtered_out),
        .dbg_filtered_valid (dbg_filtered_valid),
        .dbg_raw_distance  (dbg_raw_distance),
        .dbg_raw_valid     (dbg_raw_valid)
    );

    // ============================================================
    // Bottom GPIO (15 pins)
    // ============================================================
    // Pins 0-3: inputs (uart_rx, spi_csn, spi_sck, spi_mosi)
    // Pin 4: output (brake_irq)
```

```
    // Pin 5: output (debug filtered_valid)
    // Pins 6-14: unused (safe tie-off)

    assign gpio_bot_out[3:0]   = 4'b0;                // inputs — drive zero
    assign gpio_bot_out[4]     = brake_irq_int;        // brake interrupt
    assign gpio_bot_out[5]     = dbg_filtered_valid;   // debug output
    assign gpio_bot_out[14:6]  = 9'b0;                 // unused

    assign gpio_bot_oeb[3:0]   = 4'b1111;             // inputs: oeb=1
    assign gpio_bot_oeb[4]     = 1'b0;                 // brake_irq: oeb=0
(output)
    assign gpio_bot_oeb[5]     = 1'b0;                 // debug: oeb=0
(output)
    assign gpio_bot_oeb[14:6]  = {9{1'b1}};           // unused: oeb=1 (safe)

    // ============================================================
    // Right GPIO (9 pins) — all unused
    // ============================================================
    assign gpio_rt_out = 9'b0;
    assign gpio_rt_oeb = {9{1'b1}};

    // ============================================================
    // Top GPIO (14 pins) — all unused
    // ============================================================
    assign gpio_top_out = 14'b0;
    assign gpio_top_oeb = {14{1'b1}};

    // ============================================================
    // Drive Modes — 3'b110 = strong push-pull for all pads
    // ============================================================
    // Input pads (bot 0-3): 3'b001 = input only, no pull
    // Output pads (bot 4-5): 3'b110 = strong push-pull
    // Unused pads: 3'b110 = strong push-pull (safe default)

    genvar i;
    generate
        // Bottom: pins 0-3 as input mode
        for (i = 0; i < 4; i = i + 1) begin : gen_bot_dm_in
            assign gpio_bot_dm[i*3 +: 3] = 3'b001;
        end
        // Bottom: pins 4-5 as output mode
        for (i = 4; i < 6; i = i + 1) begin : gen_bot_dm_out
            assign gpio_bot_dm[i*3 +: 3] = 3'b110;
        end
```

```
        // Bottom: pins 6-14 unused (default push-pull)
        for (i = 6; i < 15; i = i + 1) begin : gen_bot_dm_unused
            assign gpio_bot_dm[i*3 +: 3] = 3'b110;
        end

        // Right: all unused
        for (i = 0; i < 9; i = i + 1) begin : gen_rt_dm
            assign gpio_rt_dm[i*3 +: 3] = 3'b110;
        end

        // Top: all unused
        for (i = 0; i < 14; i = i + 1) begin : gen_top_dm
            assign gpio_top_dm[i*3 +: 3] = 3'b110;
        end
    endgenerate

endmodule
•
```

- **A.2** proxcore_top.sv — Top-level integration of the four core modules

```
module proxcore_top #(
    parameter int unsigned CLK_FREQ_HZ = 25_000_000,
    parameter logic [15:0] DEFAULT_THRESHOLD = 16'd2560,
    parameter logic [15:0] DEFAULT_COEFF    = 16'h0800,
    parameter logic [15:0] DEFAULT_BAUD_DIV  = 16'd108
) (
    // System
    input  logic        clk,
    input  logic        rst_n,

    // LiDAR UART interface (1 pin)
    input  logic        uart_rx,

    // SPI configuration interface (3 pins)
    input  logic        spi_csn,
    input  logic        spi_sck,
    input  logic        spi_mosi,

    // Brake interrupt output (1 pin)
    output logic        brake_irq,
```

```
    // Debug outputs (optional — useful for integration TB and ILA)
    output logic [15:0] dbg_filtered_out,
    output logic        dbg_filtered_valid,
    output logic [15:0] dbg_raw_distance,
    output logic        dbg_raw_valid
);

    // ------------------------------------------------------------------
    // Internal signals
    // ------------------------------------------------------------------

    // Deserializer → FIR filter
    logic [15:0] deser_data;
    logic        deser_valid;

    // FIR filter → Threshold FSM
    logic [15:0] fir_out;
    logic        fir_valid;

    // Config registers → FIR filter + Threshold FSM + Deserializer
    logic [15:0] cfg_threshold;
    logic [15:0] cfg_coeff0;
    logic [15:0] cfg_coeff1;
    logic [15:0] cfg_coeff2;
    logic [15:0] cfg_coeff3;
    logic [15:0] cfg_coeff4;
    logic [15:0] cfg_coeff5;
    logic [15:0] cfg_coeff6;
    logic [15:0] cfg_coeff7;
    logic [15:0] baud_div_w;

    // ------------------------------------------------------------------
    // Debug port assignments
    // ------------------------------------------------------------------
    assign dbg_filtered_out   = fir_out;
    assign dbg_filtered_valid = fir_valid;
    assign dbg_raw_distance   = deser_data;
    assign dbg_raw_valid      = deser_valid;

    // ------------------------------------------------------------------
    // Block 1: UART Deserializer (runtime-configurable baud rate)
    // Receives 16-bit distance words from LiDAR sensor
    // ------------------------------------------------------------------
    deserializer_gc u_deserializer (
```

```
        .clk        (clk),
        .rst_n      (rst_n),
        .rx         (uart_rx),
        .baud_div   (baud_div_w),
        .data_out   (deser_data),
        .data_valid (deser_valid)
    );

    // ----------------------------------------------------------------
    // Block 2: Pipelined Symmetric FIR Filter
    // 16-tap Hamming-windowed lowpass, 8 unique coefficients
    // ----------------------------------------------------------------
    proxcore_fir_filter u_fir (
        .clk          (clk),
        .rst_n        (rst_n),
        .sample_in    (deser_data),
        .sample_valid (deser_valid),
        .coeff_in0    (cfg_coeff0),
        .coeff_in1    (cfg_coeff1),
        .coeff_in2    (cfg_coeff2),
        .coeff_in3    (cfg_coeff3),
        .coeff_in4    (cfg_coeff4),
        .coeff_in5    (cfg_coeff5),
        .coeff_in6    (cfg_coeff6),
        .coeff_in7    (cfg_coeff7),
        .result_out   (fir_out),
        .result_valid (fir_valid)
    );

    // ----------------------------------------------------------------
    // Block 3: Threshold FSM with 3-sample Debounce
    // Fires single-cycle brake_irq after 3 consecutive sub-threshold
    // ----------------------------------------------------------------
    threshold_fsm u_fsm (
        .clk         (clk),
        .rst_n       (rst_n),
        .filtered_in (fir_out),
        .data_valid  (fir_valid),
        .threshold   (cfg_threshold),
        .brake_irq   (brake_irq)
    );

    // ----------------------------------------------------------------
    // Block 4: SPI Configuration Registers (with baud rate control)
```

```
    // Runtime-programmable threshold, filter coefficients, and baud rate
    // ----------------------------------------------------------------
    config_regs_gc #(
        .DEFAULT_THRESHOLD (16'd2560),
        .DEFAULT_COEFF0    (16'sd112),
        .DEFAULT_COEFF1    (16'sd243),
        .DEFAULT_COEFF2    (16'sd618),
        .DEFAULT_COEFF3    (16'sd1293),
        .DEFAULT_COEFF4    (16'sd2217),
        .DEFAULT_COEFF5    (16'sd3225),
        .DEFAULT_COEFF6    (16'sd4089),
        .DEFAULT_COEFF7    (16'sd4587),
        .DEFAULT_BAUD_DIV  (DEFAULT_BAUD_DIV)
    ) u_config (
        .clk       (clk),
        .rst_n     (rst_n),
        .spi_csn   (spi_csn),
        .spi_sck   (spi_sck),
        .spi_mosi  (spi_mosi),
        .threshold (cfg_threshold),
        .coeff0    (cfg_coeff0),
        .coeff1    (cfg_coeff1),
        .coeff2    (cfg_coeff2),
        .coeff3    (cfg_coeff3),
        .coeff4    (cfg_coeff4),
        .coeff5    (cfg_coeff5),
        .coeff6    (cfg_coeff6),
        .coeff7    (cfg_coeff7),
        .baud_div  (baud_div_w)
    );

endmodule
```

- **A.3** deserializer_gc.sv — UART receiver with runtime baud rate control

```
•   // ============================================================
// Module: deserializer_gc
// Purpose: Runtime-configurable UART deserializer with dynamic baud rate
// Derived from: deserializer.sv
// Change from parent: BAUD_RATE replaced with runtime input baud_div
// ============================================================
```

```
module deserializer_gc (
    input  wire         clk,
    input  wire         rst_n,
    input  wire         rx,
    input  wire [15:0]  baud_div,
    output reg  [15:0]  data_out,
    output reg          data_valid
);

    // LOW_BYTE_FIRST stays as localparam — not configurable at runtime
    localparam bit LOW_BYTE_FIRST = 1'b1;

    // Baud counter uses 16-bit fixed width (can hold up to 65535)
    localparam int unsigned BAUD_CNT_W = 16;

    typedef enum logic [1:0] {
        IDLE,
        START,
        DATA,
        STOP
    } uart_state_t;

    uart_state_t              state_q;
    logic [BAUD_CNT_W-1:0]    baud_cnt_q;
    logic [2:0]               bit_idx_q;
    logic [7:0]               rx_byte_q;
    logic [7:0]               first_byte_q;
    logic                     have_first_byte_q;
    reg                       rx_meta_q;
    reg                       rx_sync_q;
    reg                       rx_sync_d_q;
    wire                      rx_fall;
    wire [BAUD_CNT_W-1:0]     half_baud_div;

    assign rx_fall       = rx_sync_d_q && !rx_sync_q;
    assign half_baud_div = baud_div >> 1;

    // Synchronize the asynchronous UART input before edge detection and
sampling.
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            rx_meta_q   <= 1'b1;
            rx_sync_q   <= 1'b1;
            rx_sync_d_q <= 1'b1;
```

```
        end else begin
            rx_meta_q   <= rx;
            rx_sync_q   <= rx_meta_q;
            rx_sync_d_q <= rx_sync_q;
        end
    end

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state_q            <= IDLE;
            baud_cnt_q         <= '0;
            bit_idx_q          <= '0;
            rx_byte_q          <= '0;
            first_byte_q       <= '0;
            have_first_byte_q  <= 1'b0;
            data_out           <= '0;
            data_valid         <= 1'b0;
        end else begin
            data_valid <= 1'b0;

            unique case (state_q)
                IDLE: begin
                    baud_cnt_q <= '0;
                    bit_idx_q  <= '0;

                    if (rx_fall) begin
                        state_q    <= START;
                        rx_byte_q  <= '0;
                    end
                end

                START: begin
                    if (baud_cnt_q == half_baud_div - 1) begin
                        baud_cnt_q <= '0;

                        // Re-check the line near the middle of the start bit
to reject glitches.
                        if (!rx_sync_q) begin
                            state_q   <= DATA;
                            bit_idx_q <= '0;
                        end else begin
                            state_q <= IDLE;
                        end
                    end else begin
```

```
                        baud_cnt_q <= baud_cnt_q + 1'b1;
                    end
                end

                DATA: begin
                    if (baud_cnt_q == baud_div - 1) begin
                        baud_cnt_q            <= '0;
                        rx_byte_q[bit_idx_q] <= rx_sync_q;

                        if (bit_idx_q == 3'd7) begin
                            state_q <= STOP;
                        end else begin
                            bit_idx_q <= bit_idx_q + 1'b1;
                        end
                    end else begin
                        baud_cnt_q <= baud_cnt_q + 1'b1;
                    end
                end

                STOP: begin
                    if (baud_cnt_q == baud_div - 1) begin
                        state_q    <= IDLE;
                        baud_cnt_q <= '0;

                        if (rx_sync_q) begin
                            if (!have_first_byte_q) begin
                                first_byte_q      <= rx_byte_q;
                                have_first_byte_q <= 1'b1;
                            end else begin
                                if (LOW_BYTE_FIRST) begin
                                    data_out <= {rx_byte_q, first_byte_q};
                                end else begin
                                    data_out <= {first_byte_q, rx_byte_q};
                                end

                                have_first_byte_q <= 1'b0;
                                data_valid        <= 1'b1;
                            end
                        end else begin
                            // Drop the partial word on framing error to
preserve byte alignment.
                            have_first_byte_q <= 1'b0;
                        end
                    end else begin
```

```
                        baud_cnt_q <= baud_cnt_q + 1'b1;
                    end
                end

                default: begin
                    state_q <= IDLE;
                end
            endcase
        end
    end

endmodule
```

- **A.4** proxcore_fir_filter.sv — 16-tap symmetric pipelined FIR filter

```
module proxcore_fir_filter (
    input  wire                         clk,
    input  wire                         rst_n,
    input  wire  [15:0]                 sample_in,
    input  wire                         sample_valid,

    // Runtime-programmable coefficients (symmetric — only 8 needed)
    input  wire signed [15:0]           coeff_in0,
    input  wire signed [15:0]           coeff_in1,
    input  wire signed [15:0]           coeff_in2,
    input  wire signed [15:0]           coeff_in3,
    input  wire signed [15:0]           coeff_in4,
    input  wire signed [15:0]           coeff_in5,
    input  wire signed [15:0]           coeff_in6,
    input  wire signed [15:0]           coeff_in7,

    output reg   [15:0]                 result_out,
    output reg                          result_valid
);

    // Parameters
    localparam TAPS           = 16;
    localparam ORDER          = 15;
    localparam COEFF_WIDTH    = 16;
    localparam COEFF_FRAC_BITS = 15;
    localparam DATA_WIDTH     = 16;
    localparam ACC_WIDTH      = 36;
```

```
    localparam LATENCY         = 5;

    // Latency calculation
    localparam BASE_LATENCY = 2;
    localparam PIPE_DEPTH   = LATENCY - BASE_LATENCY;

    // Symmetric structure parameters
    localparam CENTER_TAP = TAPS / 2;

    // Module scope declarations
    logic  [DATA_WIDTH-1:0] data_reg [0:TAPS-1];
    logic signed [ACC_WIDTH-1:0]  accum;
    logic signed [ACC_WIDTH-1:0]  scaled_acc;
    logic  [DATA_WIDTH-1:0] result_temp;

    // Symmetric structure signals
    logic [DATA_WIDTH:0]              sum_pre_u [0:CENTER_TAP-1];
    logic signed [DATA_WIDTH+1:0]     sum_pre_s [0:CENTER_TAP-1];
    logic signed [ACC_WIDTH-1:0]        mult    [0:CENTER_TAP-1];

    // Pipeline registers
    logic signed [ACC_WIDTH-1:0]  pipe_reg [0:PIPE_DEPTH-1];

    // Valid pipeline
    logic valid_pipe [0:LATENCY-2];

    // Wire coefficient inputs into indexable array
    wire signed [COEFF_WIDTH-1:0] coeffs [0:CENTER_TAP-1];
    assign coeffs[0] = coeff_in0;
    assign coeffs[1] = coeff_in1;
    assign coeffs[2] = coeff_in2;
    assign coeffs[3] = coeff_in3;
    assign coeffs[4] = coeff_in4;
    assign coeffs[5] = coeff_in5;
    assign coeffs[6] = coeff_in6;
    assign coeffs[7] = coeff_in7;

    // Loop iterators
    integer k;

    // Input shift register
    always_ff @(posedge clk or negedge rst_n) begin
    integer i_shift;
        if (!rst_n) begin
```

```
            for (i_shift = 0; i_shift < TAPS; i_shift = i_shift + 1) begin
                data_reg[i_shift] <= '0;
            end
        end else if (sample_valid) begin
            data_reg[0] <= sample_in;
            for (i_shift = 1; i_shift < TAPS; i_shift = i_shift + 1) begin
                data_reg[i_shift] <= data_reg[i_shift-1];
            end
        end
```

- **A.5** threshold_fsm.sv — 4-state debounced threshold detector

```
module threshold_fsm (
    input  logic        clk,
    input  logic        rst_n,
    input  logic [15:0] filtered_in,
    input  logic        data_valid,
    input  logic [15:0] threshold,
    output logic        brake_irq
);

    typedef enum logic [1:0] {
        CLEAR   = 2'b00,
        WARN1   = 2'b01,
        WARN2   = 2'b10,
        BRAKING = 2'b11
    } state_t;

    state_t state_q;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state_q   <= CLEAR;
            brake_irq <= 1'b0;
        end else begin
            brake_irq <= 1'b0; // default: deassert every cycle

            if (data_valid) begin
                unique case (state_q)

                    CLEAR: begin
                        if (filtered_in < threshold)
                            state_q <= WARN1;
```

```
                        // else stay CLEAR
                    end

                    WARN1: begin
                        if (filtered_in < threshold)
                            state_q <= WARN2;
                        else
                            state_q <= CLEAR; // not consecutive — reset
                    end

                    WARN2: begin
                        if (filtered_in < threshold) begin
                            state_q   <= BRAKING;
                            brake_irq <= 1'b1; // one-cycle pulse
                        end else
                            state_q <= CLEAR; // not consecutive — reset
                    end

                    BRAKING: begin
                        if (filtered_in >= threshold)
                            state_q <= CLEAR; // obstacle cleared
                        // else stay BRAKING, no repeated IRQ
                    end

                endcase
            end
        end
    end

endmodule
```

- **A.6** config_regs_gc.sv — 10-register SPI configuration interface

```
// ============================================================
// Module: config_regs_gc
// Purpose: SPI configuration registers with runtime baud rate control
// Derived from: config_regs.sv
// Change from parent: Added baud_div register at address 0x09
// ============================================================

module config_regs_gc #(
    parameter logic [15:0] DEFAULT_THRESHOLD = 16'd2560,
```

```
    parameter logic [15:0] DEFAULT_COEFF0    = 16'sd112,
    parameter logic [15:0] DEFAULT_COEFF1    = 16'sd243,
    parameter logic [15:0] DEFAULT_COEFF2    = 16'sd618,
    parameter logic [15:0] DEFAULT_COEFF3    = 16'sd1293,
    parameter logic [15:0] DEFAULT_COEFF4    = 16'sd2217,
    parameter logic [15:0] DEFAULT_COEFF5    = 16'sd3225,
    parameter logic [15:0] DEFAULT_COEFF6    = 16'sd4089,
    parameter logic [15:0] DEFAULT_COEFF7    = 16'sd4587,
    parameter logic [15:0] DEFAULT_BAUD_DIV  = 16'd108
) (
    input  logic        clk,
    input  logic        rst_n,

    input  logic        spi_csn,
    input  logic        spi_sck,
    input  logic        spi_mosi,

    output logic [15:0] threshold,
    output logic [15:0] coeff0,
    output logic [15:0] coeff1,
    output logic [15:0] coeff2,
    output logic [15:0] coeff3,
    output logic [15:0] coeff4,
    output logic [15:0] coeff5,
    output logic [15:0] coeff6,
    output logic [15:0] coeff7,
    output logic [15:0] baud_div
);

    localparam int unsigned FRAME_BITS  = 24;
    localparam int unsigned ADDR_MSB    = 23;
    localparam int unsigned ADDR_LSB    = 16;
    localparam int unsigned DATA_MSB    = 15;
    localparam int unsigned DATA_LSB    = 0;
    localparam int unsigned BIT_CNT_W   = $clog2(FRAME_BITS);

    localparam logic [7:0] REG_THRESHOLD = 8'h00;
    localparam logic [7:0] REG_COEFF0    = 8'h01;
    localparam logic [7:0] REG_COEFF1    = 8'h02;
    localparam logic [7:0] REG_COEFF2    = 8'h03;
    localparam logic [7:0] REG_COEFF3    = 8'h04;
    localparam logic [7:0] REG_COEFF4    = 8'h05;
    localparam logic [7:0] REG_COEFF5    = 8'h06;
    localparam logic [7:0] REG_COEFF6    = 8'h07;
```

```
    localparam logic [7:0] REG_COEFF7    = 8'h08;
    localparam logic [7:0] REG_BAUD_DIV  = 8'h09;

    logic                  sck_meta_q;
    logic                  sck_sync_q;
    logic                  sck_sync_d_q;
    logic                  csn_meta_q;
    logic                  csn_sync_q;
    logic                  mosi_meta_q;
    logic                  mosi_sync_q;
    logic [FRAME_BITS-1:0] shift_q;
    logic [FRAME_BITS-1:0] frame_next;
    logic [BIT_CNT_W-1:0]  bit_cnt_q;
    logic                  sck_rise;

    assign sck_rise   = sck_sync_q && !sck_sync_d_q;
    assign frame_next = {shift_q[FRAME_BITS-2:0], mosi_sync_q};

    // Bring the external SPI pins into the 50 MHz system clock domain.
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            sck_meta_q   <= 1'b0;
            sck_sync_q   <= 1'b0;
            sck_sync_d_q <= 1'b0;
            csn_meta_q   <= 1'b1;
            csn_sync_q   <= 1'b1;
            mosi_meta_q  <= 1'b0;
            mosi_sync_q  <= 1'b0;
        end else begin
            sck_meta_q   <= spi_sck;
            sck_sync_q   <= sck_meta_q;
            sck_sync_d_q <= sck_sync_q;
            csn_meta_q   <= spi_csn;
            csn_sync_q   <= csn_meta_q;
            mosi_meta_q  <= spi_mosi;
            mosi_sync_q  <= mosi_meta_q;
        end
    end

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            threshold <= DEFAULT_THRESHOLD;
            coeff0    <= DEFAULT_COEFF0;
            coeff1    <= DEFAULT_COEFF1;
```

```
            coeff2    <= DEFAULT_COEFF2;
            coeff3    <= DEFAULT_COEFF3;
            coeff4    <= DEFAULT_COEFF4;
            coeff5    <= DEFAULT_COEFF5;
            coeff6    <= DEFAULT_COEFF6;
            coeff7    <= DEFAULT_COEFF7;
            baud_div  <= DEFAULT_BAUD_DIV;
            shift_q   <= '0;
            bit_cnt_q <= '0;
        end else begin
            if (csn_sync_q) begin
                shift_q   <= '0;
                bit_cnt_q <= '0;
            end else if (sck_rise) begin
                shift_q <= frame_next;

                if (bit_cnt_q == FRAME_BITS - 1) begin
                    bit_cnt_q <= '0;

                    unique case (frame_next[ADDR_MSB:ADDR_LSB])
                        REG_THRESHOLD: threshold <=
frame_next[DATA_MSB:DATA_LSB];
                        REG_COEFF0:    coeff0    <=
frame_next[DATA_MSB:DATA_LSB];
                        REG_COEFF1:    coeff1    <=
frame_next[DATA_MSB:DATA_LSB];
                        REG_COEFF2:    coeff2    <=
frame_next[DATA_MSB:DATA_LSB];
                        REG_COEFF3:    coeff3    <=
frame_next[DATA_MSB:DATA_LSB];
                        REG_COEFF4:    coeff4    <=
frame_next[DATA_MSB:DATA_LSB];
                        REG_COEFF5:    coeff5    <=
frame_next[DATA_MSB:DATA_LSB];
                        REG_COEFF6:    coeff6    <=
frame_next[DATA_MSB:DATA_LSB];
                        REG_COEFF7:    coeff7    <=
frame_next[DATA_MSB:DATA_LSB];
                        REG_BAUD_DIV:  baud_div  <=
frame_next[DATA_MSB:DATA_LSB];
                        default: begin
                            // Invalid write addresses are ignored.
                        end
                    endcase
```

```
                end else begin
                    bit_cnt_q <= bit_cnt_q + 1'b1;
                end
            end
        end
    end

endmodule
```

Appendix B: Complete Testbench Source Code

- **B.1** tb_proxcore_top.sv — Integration testbench (7 tests)

```
`timescale 1ns/1ps

module proxcore_top_tb;

    // ------------------------------------------------------------------
    // Parameters
    // ------------------------------------------------------------------
    localparam int unsigned CLK_FREQ_HZ  = 25_000_000;
    localparam int unsigned BAUD_RATE    = 460_800;
    localparam time         CLK_PERIOD   = 20ns;

    localparam int unsigned CLKS_PER_BIT = CLK_FREQ_HZ / BAUD_RATE;  // 108
    localparam time         BIT_PERIOD   = CLKS_PER_BIT * CLK_PERIOD; //
2160ns

    // Additional baud rate timing for TEST 7
    localparam int unsigned CLKS_PER_BIT_115200 = 434;
    localparam time         BIT_PERIOD_115200   = CLKS_PER_BIT_115200 *
CLK_PERIOD; // 8680ns

    localparam int unsigned TAPS       = 16;
    localparam int unsigned LATENCY    = 5;
    localparam int unsigned FILL_COUNT = TAPS + LATENCY + 4; // 25 — fully
primes pipeline

    // Distance constants (Q10.6)
    localparam [15:0] DIST_100M  = 16'd6400;  // 100m — clear track
```

```
    localparam [15:0] DIST_30M   = 16'd1920;  // 30m  — real obstacle
    localparam [15:0] DIST_2M    = 16'd128;   // 2m   — raindrop spike
    localparam [15:0] THRESH_20M = 16'd1280;  // 20m  — reconfigured
threshold

    // SPI timing
    localparam time SPI_HALF_PERIOD = 250ns;

    // ------------------------------------------------------------------
    // DUT signals
    // ------------------------------------------------------------------
    logic        clk;
    logic        rst_n;
    logic        uart_rx;
    logic        spi_csn;
    logic        spi_sck;
    logic        spi_mosi;
    logic        brake_irq;
    logic [15:0] dbg_filtered_out;
    logic        dbg_filtered_valid;
    logic [15:0] dbg_raw_distance;
    logic        dbg_raw_valid;

    // ------------------------------------------------------------------
    // Test tracking
    // ------------------------------------------------------------------
    int unsigned failures;
    int unsigned test_id;
    int unsigned failures_before;
    int unsigned brake_count;

    // ------------------------------------------------------------------
    // DUT instantiation
    // ------------------------------------------------------------------
    proxcore_top #(
        .CLK_FREQ_HZ (CLK_FREQ_HZ),
        .DEFAULT_THRESHOLD (16'd2560),
        .DEFAULT_BAUD_DIV (16'd108)
    ) dut (
        .clk               (clk),
        .rst_n             (rst_n),
        .uart_rx           (uart_rx),
        .spi_csn           (spi_csn),
        .spi_sck           (spi_sck),
```

```
        .spi_mosi           (spi_mosi),
        .brake_irq          (brake_irq),
        .dbg_filtered_out   (dbg_filtered_out),
        .dbg_filtered_valid (dbg_filtered_valid),
        .dbg_raw_distance   (dbg_raw_distance),
        .dbg_raw_valid      (dbg_raw_valid)
    );

    // ---------------------------------------------------------------
    // Clock generation
    // ---------------------------------------------------------------
    initial clk = 1'b0;
    always #(CLK_PERIOD / 2) clk = ~clk;

    // ---------------------------------------------------------------
    // brake_irq monitor — counts pulses on system clock
    // ---------------------------------------------------------------
    always @(posedge clk) begin
        if (brake_irq && rst_n)
            brake_count = brake_count + 1;
    end

    // ---------------------------------------------------------------
    // Watchdog
    // ---------------------------------------------------------------
    initial begin
        #(50ms);
        $display("[FATAL] Simulation timeout");
        $fatal(1);
    end

    // ---------------------------------------------------------------
    // UART Tasks
    // ---------------------------------------------------------------
    task automatic uart_send_byte(input logic [7:0] data);
        integer i;
        begin
            // Start bit
            uart_rx = 1'b0;
            #(BIT_PERIOD);
            // Data bits (LSB first)
            for (i = 0; i < 8; i++) begin
                uart_rx = data[i];
                #(BIT_PERIOD);
```

```
            end
            // Stop bit
            uart_rx = 1'b1;
            #(BIT_PERIOD);
        end
    endtask

    // UART task variant with configurable bit period for TEST 7
    task automatic uart_send_byte_bp(input logic [7:0] data, input time
bit_period);
        integer i;
        begin
            // Start bit
            uart_rx = 1'b0;
            #(bit_period);
            // Data bits (LSB first)
            for (i = 0; i < 8; i++) begin
                uart_rx = data[i];
                #(bit_period);
            end
            // Stop bit
            uart_rx = 1'b1;
            #(bit_period);
        end
    endtask

    task automatic uart_send_word(input logic [15:0] data);
        begin
            uart_send_byte(data[7:0]);   // low byte first
            uart_send_byte(data[15:8]);
        end
    endtask

    task automatic uart_send_word_bp(input logic [15:0] data, input time
bit_period);
        begin
            uart_send_byte_bp(data[7:0], bit_period);   // low byte first
            uart_send_byte_bp(data[15:8], bit_period);
        end
    endtask

    task automatic send_n_samples(input logic [15:0] distance, input integer
n);
        integer i;
```

```
        begin
            for (i = 0; i < n; i++)
                uart_send_word(distance);
        end
    endtask

    task automatic send_n_samples_bp(input logic [15:0] distance, input
integer n, input time bit_period);
        integer i;
        begin
            for (i = 0; i < n; i++)
                uart_send_word_bp(distance, bit_period);
        end
    endtask

    task automatic send_alternating(
        input logic [15:0] dist_a,
        input logic [15:0] dist_b,
        input integer n
    );
        integer i;
        begin
            for (i = 0; i < n; i++) begin
                if (i % 2 == 0)
                    uart_send_word(dist_a);
                else
                    uart_send_word(dist_b);
            end
        end
    endtask

    // Wait for all pipeline stages to drain after last UART word
    task automatic wait_pipeline_drain;
        begin
            repeat (CLKS_PER_BIT * 20 + LATENCY + 10) @(posedge clk);
        end
    endtask

    // ------------------------------------------------------------------
    // SPI Tasks
    // ------------------------------------------------------------------
    task automatic spi_clock_bit(input logic bit_value);
        begin
            spi_mosi = bit_value;
```

```
            #(SPI_HALF_PERIOD);
            spi_sck = 1'b1;
            #(SPI_HALF_PERIOD);
            spi_sck = 1'b0;
            #(SPI_HALF_PERIOD);
        end
    endtask

    task automatic spi_write(input logic [7:0] addr, input logic [15:0]
data);
        logic [23:0] frame;
        integer i;
        begin
            frame = {addr, data};
            spi_csn = 1'b0;
            #(SPI_HALF_PERIOD);
            for (i = 23; i >= 0; i--)
                spi_clock_bit(frame[i]);
            spi_csn  = 1'b1;
            spi_mosi = 1'b0;
            #(SPI_HALF_PERIOD);
            repeat (4) @(posedge clk);
        end
    endtask

    // ------------------------------------------------------------------
    // Reset Task
    // ------------------------------------------------------------------
    task automatic apply_reset;
        begin
            rst_n    = 1'b0;
            uart_rx  = 1'b1;  // UART idle
            spi_csn  = 1'b1;
            spi_sck  = 1'b0;
            spi_mosi = 1'b0;
            repeat (4) @(posedge clk);
            rst_n = 1'b1;
            @(posedge clk);
        end
    endtask

    // Fill pipeline and wait for settling
    // NOTE: During fill, the FIR output starts small (delay line filling)
    // so the FSM may fire brake_irq during this transient. Callers must
```

```
    // reset brake_count AFTER fill_pipeline returns.
    task automatic fill_pipeline(input logic [15:0] distance);
        begin
            send_n_samples(distance, FILL_COUNT);
            wait_pipeline_drain();
        end
    endtask

    // Fill pipeline with custom bit period (for TEST 7)
    task automatic fill_pipeline_bp(input logic [15:0] distance, input time
bit_period);
        begin
            send_n_samples_bp(distance, FILL_COUNT, bit_period);
            wait_pipeline_drain();
        end
    endtask

    // ------------------------------------------------------------------
    // Result reporting
    // ------------------------------------------------------------------
    task automatic report;
        begin
            if (failures == failures_before)
                $display("[%0t] TEST %0d PASS", $time, test_id);
            else
                $display("[%0t] TEST %0d FAIL", $time, test_id);
        end
    endtask

    // ==================================================================
    // MAIN STIMULUS
    // ==================================================================
    initial begin
        clk         = 1'b0;
        rst_n       = 1'b1;
        uart_rx     = 1'b1;
        spi_csn     = 1'b1;
        spi_sck     = 1'b0;
        spi_mosi    = 1'b0;
        failures    = 0;
        brake_count = 0;

        apply_reset();
```

```
        // ============================================================
        // TEST 1: Clear track — 100m, no false brakes
        // ============================================================
        test_id = 1;
        failures_before = failures;
        $display("[%0t] TEST %0d START: Clear track — no false brakes",
$time, test_id);

        apply_reset();
        brake_count = 0;

        fill_pipeline(DIST_100M);
        brake_count = 0;

        send_n_samples(DIST_100M, 10);
        wait_pipeline_drain();

        if (brake_count != 0) begin
            failures++;
            $display("[%0t] ERROR test %0d: brake fired %0d time(s) on clear
track",
                     $time, test_id, brake_count);
        end

        report();

        // ============================================================
        // TEST 2: Rainstorm — alternating 100m / 2m, no brake
        // ============================================================
        test_id = 2;
        failures_before = failures;
        $display("[%0t] TEST %0d START: Rainstorm spike rejection", $time,
test_id);

        apply_reset();
        brake_count = 0;

        fill_pipeline(DIST_100M);
        brake_count = 0;

        send_alternating(DIST_2M, DIST_100M, 20);
        wait_pipeline_drain();

        if (brake_count != 0) begin
```

```
            failures++;
            $display("[%0t] ERROR test %0d: brake fired %0d time(s) during
rainstorm",
                     $time, test_id, brake_count);
        end

        report();

        // ============================================================
        // TEST 3: Real obstacle at 30m — brake MUST fire
        // ============================================================
        test_id = 3;
        failures_before = failures;
        $display("[%0t] TEST %0d START: 30m obstacle detection", $time,
test_id);

        apply_reset();
        brake_count = 0;

        fill_pipeline(DIST_100M);
        brake_count = 0;

        send_n_samples(DIST_30M, 25);
        wait_pipeline_drain();

        if (brake_count == 0) begin
            failures++;
            $display("[%0t] ERROR test %0d: brake NEVER fired — obstacle
missed!", $time, test_id);
        end else begin
            $display("[%0t] INFO  test %0d: brake fired %0d time(s)", $time,
test_id, brake_count);
        end

        report();

        // ============================================================
        // TEST 4: Recovery + re-detection
        // ============================================================
        test_id = 4;
        failures_before = failures;
        $display("[%0t] TEST %0d START: Recovery and second obstacle", $time,
test_id);
```

```
        apply_reset();
        brake_count = 0;

        fill_pipeline(DIST_100M);
        brake_count = 0;

        send_n_samples(DIST_30M, 25);
        wait_pipeline_drain();

        if (brake_count == 0) begin
            failures++;
            $display("[%0t] ERROR test %0d: first obstacle not detected",
$time, test_id);
        end else begin
            $display("[%0t] INFO  test %0d: first brake fired (%0d
pulse(s))",
                     $time, test_id, brake_count);
        end

        send_n_samples(DIST_100M, FILL_COUNT);
        wait_pipeline_drain();

        brake_count = 0;

        send_n_samples(DIST_30M, 25);
        wait_pipeline_drain();

        if (brake_count == 0) begin
            failures++;
            $display("[%0t] ERROR test %0d: second obstacle not detected",
$time, test_id);
        end else begin
            $display("[%0t] INFO  test %0d: second brake fired (%0d
pulse(s))",
                     $time, test_id, brake_count);
        end

        report();

        // ============================================================
        // TEST 5: SPI threshold reconfiguration
        // ============================================================
        test_id = 5;
        failures_before = failures;
```

```
        $display("[%0t] TEST %0d START: SPI threshold reconfiguration",
$time, test_id);

        apply_reset();
        brake_count = 0;

        spi_write(8'h00, THRESH_20M);

        fill_pipeline(DIST_100M);
        brake_count = 0;

        send_n_samples(DIST_30M, 25);
        wait_pipeline_drain();

        if (brake_count != 0) begin
            failures++;
            $display("[%0t] ERROR test %0d: brake fired %0d time(s) — SPI
threshold not respected",
                     $time, test_id, brake_count);
        end else begin
            $display("[%0t] INFO  test %0d: 30m above 20m threshold — no
brake (correct)",
                     $time, test_id);
        end

        report();

        // ============================================================
        // TEST 6: Reset recovery
        // ============================================================
        test_id = 6;
        failures_before = failures;
        $display("[%0t] TEST %0d START: Reset recovery", $time, test_id);

        apply_reset();
        brake_count = 0;

        fill_pipeline(DIST_100M);
        send_n_samples(DIST_30M, 25);
        wait_pipeline_drain();

        uart_rx = 1'b1;
        repeat (CLKS_PER_BIT * 20) @(posedge clk);
```

```
        apply_reset();
        brake_count = 0;

        fill_pipeline(DIST_100M);
        brake_count = 0;

        send_n_samples(DIST_100M, 10);
        wait_pipeline_drain();

        if (brake_count != 0) begin
            failures++;
            $display("[%0t] ERROR test %0d: brake fired %0d time(s) after
reset on clear track",
                     $time, test_id, brake_count);
        end else begin
            $display("[%0t] INFO  test %0d: clean recovery after reset",
$time, test_id);
        end

        report();

        // ============================================================
        // TEST 7: End-to-end baud rate change via SPI
        // ============================================================
        test_id = 7;
        failures_before = failures;
        $display("[%0t] TEST %0d START: End-to-end baud rate change via SPI",
$time, test_id);

        apply_reset();
        brake_count = 0;

        fill_pipeline(DIST_100M);
        brake_count = 0;

        send_n_samples(DIST_100M, 10);
        wait_pipeline_drain();

        if (brake_count != 0) begin
            failures++;
            $display("[%0t] ERROR test %0d phase 1: unexpected brake at
460800", $time, test_id);
        end else begin
```

```
            $display("[%0t] INFO  test %0d phase 1: 460800 baud OK", $time,
test_id);
        end

        spi_write(8'h09, 16'd434);
        repeat (8) @(posedge clk);
        repeat (8) @(posedge clk);

        fill_pipeline_bp(DIST_100M, BIT_PERIOD_115200);
        brake_count = 0;

        send_n_samples_bp(DIST_30M, 40, BIT_PERIOD_115200);
        wait_pipeline_drain();

        if (brake_count == 0) begin
            failures++;
            $display("[%0t] ERROR test %0d phase 4: brake NOT fired at
115200", $time, test_id);
        end else begin
            $display("[%0t] INFO  test %0d phase 4: 115200 baud OK — brake
fired (%0d pulse(s))",
                     $time, test_id, brake_count);
        end

        report();

        // ============================================================
        // FINAL REPORT
        // ============================================================
        $display("\n================================================");
        $display("  ProxCore Integration Verification");
        $display("================================================");
        $display("  Total failures: %0d", failures);
        if (failures == 0)
            $display("  RESULT: PASS — all %0d tests passed", test_id);
        else
            $display("  RESULT: FAIL — %0d error(s)", failures);
        $display("================================================\n");

        if (failures != 0) $fatal(1);
        $finish;
    end

endmodule
```

- **B.2** output_test_filter.sv — FIR filter testbench (6 tests with golden model)

```
module output_test_filter;

    // ------------------------------------------------------------
    // Parameters
    // ------------------------------------------------------------
    parameter DATA_WIDTH      = 16;
    parameter ACC_WIDTH       = 36;
    parameter COEFF_WIDTH     = 16;
    parameter COEFF_FRAC_BITS = 15;
    parameter LATENCY         = 5;
    parameter CLOCK_PERIOD_NS = 20;
    parameter VECTOR_COUNT    = 200;
    parameter TAPS            = 16;
    parameter CENTER_TAP      = 8;

    // Q10.6 distance constants (meters x 64)
    parameter [15:0] DIST_100M = 16'd6400;   // 100m - clear track
    parameter [15:0] DIST_30M  = 16'd1920;   // 30m  - real obstacle
    parameter [15:0] DIST_2M   = 16'd128;    // 2m   - raindrop spike
    parameter [15:0] THRESH    = 16'd2560;   // 40m  - brake threshold

    // ------------------------------------------------------------
    // Clock and reset
    // ------------------------------------------------------------
    reg clk;
    reg rst_n;

    // ------------------------------------------------------------
    // DUT interface - unsigned throughout
    // ------------------------------------------------------------
    reg  [DATA_WIDTH-1:0] sample_in;
    reg                   sample_valid;
    wire [DATA_WIDTH-1:0] result_out;
    wire                  result_valid;

    // ------------------------------------------------------------
    // Shadow model state - unsigned to match DUT delay line
```

```
    // ----------------------------------------------------------------
    reg [DATA_WIDTH-1:0] history [0:TAPS-1];

    // ----------------------------------------------------------------
    // Coefficients - Hamming-windowed, signed Q1.15, matching RTL
    // ----------------------------------------------------------------
    logic signed [COEFF_WIDTH-1:0] coeffs [0:TAPS-1];
    initial begin
        coeffs[0]  = 16'sd112;   coeffs[8]  = 16'sd4587;
        coeffs[1]  = 16'sd243;   coeffs[9]  = 16'sd4089;
        coeffs[2]  = 16'sd618;   coeffs[10] = 16'sd3225;
        coeffs[3]  = 16'sd1293;  coeffs[11] = 16'sd2217;
        coeffs[4]  = 16'sd2217;  coeffs[12] = 16'sd1293;
        coeffs[5]  = 16'sd3225;  coeffs[13] = 16'sd618;
        coeffs[6]  = 16'sd4089;  coeffs[14] = 16'sd243;
        coeffs[7]  = 16'sd4587;  coeffs[15] = 16'sd112;
    end

    // ----------------------------------------------------------------
    // Counters and checker variables
    // ----------------------------------------------------------------
    integer expected_queue [$];
    integer sample_count;
    integer error_count;
    integer match_count;
    integer mismatch_count;
    integer total_checked;
    integer cycle_count;
    integer stim_index;
    integer history_index;
    integer rtl_val;
    integer expected_val;

    // ----------------------------------------------------------------
    // Clock generation
    // ----------------------------------------------------------------
    initial begin
        clk = 0;
        forever #(CLOCK_PERIOD_NS / 2) clk = ~clk;
    end

    // ----------------------------------------------------------------
    // Watchdog
    // ----------------------------------------------------------------
```

```
    initial begin
        #(VECTOR_COUNT * CLOCK_PERIOD_NS * 500);
        $display("[FATAL] Simulation timeout");
        $finish;
    end

    // ------------------------------------------------------------------
    // DUT instantiation
    // ------------------------------------------------------------------
    proxcore_fir_filter dut_inst (
    .clk         (clk),
    .rst_n       (rst_n),
    .sample_in   (sample_in),
    .sample_valid(sample_valid),
    .coeff_in0   (16'sd112),
    .coeff_in1   (16'sd243),
    .coeff_in2   (16'sd618),
    .coeff_in3   (16'sd1293),
    .coeff_in4   (16'sd2217),
    .coeff_in5   (16'sd3225),
    .coeff_in6   (16'sd4089),
    .coeff_in7   (16'sd4587),
    .result_out  (result_out),
    .result_valid(result_valid)
);
    // ------------------------------------------------------------------
    // Golden model
    // FIXED: acc_shifted is 36-bit reg, not integer, preventing
    // truncation of upper bits before the right-shift
    // ------------------------------------------------------------------
    function [DATA_WIDTH-1:0] calculate_golden;
        input [DATA_WIDTH-1:0] sample;
        reg [DATA_WIDTH:0]                  pair_sum_u;
        reg signed [DATA_WIDTH+1:0]         pair_sum_s;
        reg signed [DATA_WIDTH+COEFF_WIDTH:0] prod;
        reg signed [ACC_WIDTH-1:0]          acc;
        reg signed [ACC_WIDTH-1:0]          acc_shifted;  // FIXED: was integer
        integer k;
        begin
            for (k = TAPS-1; k > 0; k = k-1)
                history[k] = history[k-1];
            history[0] = sample;

            acc = 0;
```

```
            for (k = 0; k < CENTER_TAP; k = k+1) begin
                pair_sum_u = history[k] + history[TAPS-1-k];
                pair_sum_s = {1'b0, pair_sum_u};
                prod       = pair_sum_s * coeffs[k];
                acc        = acc + prod;
            end

            acc_shifted      = acc >>> COEFF_FRAC_BITS;
            calculate_golden = acc_shifted[DATA_WIDTH-1:0];
        end
    endfunction

    // ------------------------------------------------------------------
    // Helper tasks
    // ------------------------------------------------------------------
    task do_reset;
        integer m;
        begin
            rst_n        = 0;
            sample_valid = 0;
            sample_in    = 0;
            repeat (4) @(posedge clk);
            rst_n = 1;
            @(posedge clk);
            for (m = 0; m < TAPS; m = m+1)
                history[m] = 0;
            expected_queue.delete();
        end
    endtask

    // Drive sample and push golden result to checker queue
    task drive_sample;
        input [DATA_WIDTH-1:0] val;
        integer gv;
        begin
            sample_in    = val;
            sample_valid = 1;
            @(posedge clk);
            gv = calculate_golden(val);
            expected_queue.push_back(gv);
            sample_count = sample_count + 1;
        end
    endtask
```

```
    // Drive sample without checking - used to pre-fill delay line
    task drive_fill;
    input [DATA_WIDTH-1:0] val;
    integer gv;
    begin
        sample_in    = val;
        sample_valid = 1;
        @(posedge clk);
        gv = calculate_golden(val);
        expected_queue.push_back(gv); // Add this to keep the FIFO aligned
        sample_count = sample_count + 1;
    end
endtask
    // Stop driving and wait for all queued outputs to arrive
    task flush_and_check_empty;
        begin
            sample_valid = 0;
            sample_in    = 0;
            repeat (LATENCY + 10) @(posedge clk);
            #1;
            if (expected_queue.size() != 0) begin
                $display("[ERROR] %0d outputs never received",
                         expected_queue.size());
                error_count = error_count + expected_queue.size();
                expected_queue.delete();
            end
        end
    endtask

    // ------------------------------------------------------------------
    // Checker process
    // ------------------------------------------------------------------
    always @(posedge clk) begin
        if (result_valid) begin
            if (expected_queue.size() == 0) begin
                $display("[ERROR] Unexpected result_valid at cycle %0d",
                         cycle_count);
                error_count = error_count + 1;
            end else begin
                rtl_val      = {{16{1'b0}}, result_out};
                expected_val = expected_queue.pop_front();
                total_checked = total_checked + 1;

                if (rtl_val !== expected_val) begin
```

```
                    $display("[MISMATCH] cycle=%0d RTL=%0d Golden=%0d diff=%0d",
                             cycle_count, rtl_val, expected_val,
                             rtl_val - expected_val);
                    mismatch_count = mismatch_count + 1;
                    error_count    = error_count + 1;
                    $display("[FATAL] Stopping on first mismatch.");
                    $finish;
                end else begin
                    match_count = match_count + 1;
                end
            end
        end
        cycle_count = cycle_count + 1;
    end

    // ================================================================
    // MAIN STIMULUS
    // ================================================================
    initial begin
        sample_in      = 0;
        sample_valid   = 0;
        rst_n          = 0;
        sample_count   = 0;
        error_count    = 0;
        match_count    = 0;
        mismatch_count = 0;
        total_checked  = 0;
        cycle_count    = 0;
        for (history_index = 0; history_index < TAPS; history_index =
history_index + 1)
            history[history_index] = 0;
        expected_queue = {};

        // ============================================================
        // TEST 1: Edge cases
        // Boundary values that stress signedness and overflow paths
        // ============================================================
        $display("\n=== TEST 1: Edge cases ===");
        do_reset();

        drive_sample(16'hFFFF);  // maximum value
        drive_sample(16'h8000);  // MSB set - catches sign bugs
        drive_sample(16'h7FFF);  // just below MSB
        drive_sample(16'h0000);  // zero
```

```
        flush_and_check_empty();
        $display("[TEST 1] Done. Errors so far: %0d", error_count);

        // ============================================================
        // TEST 2: Random stimulus - arithmetic correctness
        // 200 random unsigned samples vs shadow model
        // ============================================================
        $display("\n=== TEST 2: Random stimulus (%0d samples) ===",
VECTOR_COUNT);
        do_reset();

        for (stim_index = 0; stim_index < VECTOR_COUNT; stim_index = stim_index +
1)
            drive_sample($urandom & 16'hFFFF);

        flush_and_check_empty();
        $display("[TEST 2] Done. Errors so far: %0d", error_count);

        // ============================================================
        // TEST 3: DC flatness at 100m
        // After delay line fills the output must settle to 100m.
        // Proves unity DC gain.
        // ============================================================
        $display("\n=== TEST 3: DC flatness at 100m ===");
        do_reset();

        // Silent fill
        for (stim_index = 0; stim_index < TAPS; stim_index = stim_index + 1)
            drive_fill(DIST_100M);

        // Checked samples
        for (stim_index = 0; stim_index < 20; stim_index = stim_index + 1)
            drive_sample(DIST_100M);

        flush_and_check_empty();
        $display("[TEST 3] Done. Errors so far: %0d", error_count);

        // ============================================================
        // TEST 4: Rainstorm - spike rejection
        //
        // Track is clear at 100m. Sensor alternates 100m / 2m
        // (raindrop hits). The filtered output must NEVER drop below
        // 50m (3200 Q10.6). A drop below threshold = false brake.
```

```
        //
        // This is the primary safety property of proxcore.
        // ============================================================
        $display("\n=== TEST 4: Rainstorm spike rejection ===");
        do_reset();

        // Fill delay line with clear track
        for (stim_index = 0; stim_index < TAPS*2; stim_index = stim_index + 1)
            drive_fill(DIST_100M);

        begin : test4_block
            integer false_brake_count;
            integer gv;
            false_brake_count = 0;

            sample_valid = 1;
            for (stim_index = 0; stim_index < 40; stim_index = stim_index + 1)
begin
                sample_in = (stim_index % 2 == 0) ? DIST_2M : DIST_100M;
                @(posedge clk);
                gv = calculate_golden(sample_in);
                expected_queue.push_back(gv);      // <-- ADDED THIS LINE
                sample_count = sample_count + 1;

                if (gv < 16'd3200) begin        $display("[FAIL] TEST4: output
%0d < 50m at spike %0d — false brake!",
                             gv, stim_index);
                    false_brake_count = false_brake_count + 1;
                    error_count = error_count + 1;
                end
            end

            if (false_brake_count == 0)
                $display("[PASS] TEST4: All 40 raindrop spikes rejected — no
false brake triggered");
            else
                $display("[FAIL] TEST4: %0d spikes caused false brake",
false_brake_count);
        end

        flush_and_check_empty();

        // ============================================================
        // TEST 5: Real obstacle detection
```

```
        //
        // Track is clear at 100m, then a car stalls 30m ahead.
        // After enough 30m readings the filtered output MUST drop
        // below the 40m threshold (2560 Q10.6).
        //
        // Failure here means the chip would NOT stop the tram.
        // This is a catastrophic failure mode.
        // ============================================================
        $display("\n=== TEST 5: Real obstacle at 30m must be detected ===");
        do_reset();

        // Fill with clear track
        for (stim_index = 0; stim_index < TAPS*2; stim_index = stim_index + 1)
            drive_fill(DIST_100M);

        begin : test5_block
            integer obstacle_detected;
            integer first_sample;
            integer gv;
            obstacle_detected = 0;
            first_sample      = -1;

            sample_valid = 1;
            for (stim_index = 0; stim_index < 40; stim_index = stim_index + 1)
begin
                sample_in = DIST_30M;
                @(posedge clk);
                gv = calculate_golden(DIST_30M);
                expected_queue.push_back(gv);      // <-- ADDED THIS LINE
                sample_count = sample_count + 1;

                if (!obstacle_detected && gv < THRESH) begin
                    obstacle_detected = 1;
                    first_sample      = stim_index;
                    $display("[PASS] TEST5: Threshold crossed at obstacle sample
%0d (output=%0d < threshold=%0d)",
                             stim_index, gv, THRESH);
                end
            end

            if (!obstacle_detected) begin
                $display("[FAIL] TEST5: CRITICAL - filter NEVER dropped below
threshold");
                $display("       The tram would NOT stop. Fatal design error.");
```

```
                error_count = error_count + 1;
            end else begin
                $display("[PASS] TEST5: Obstacle confirmed detected at sample
%0d", first_sample);
            end
        end

        flush_and_check_empty();

        // ============================================================
        // TEST 6: Recovery after obstacle clears
        //
        // After obstacle is detected, track clears back to 100m.
        // Filter output must rise back above threshold, proving the
        // system can resume normal operation.
        // ============================================================
        $display("\n=== TEST 6: Track clears after obstacle — filter recovers
===");
        do_reset();

        // Fill clear track
        for (stim_index = 0; stim_index < TAPS*2; stim_index = stim_index + 1)
            drive_fill(DIST_100M);

        // 20 samples of obstacle
        for (stim_index = 0; stim_index < 20; stim_index = stim_index + 1)
            drive_fill(DIST_30M);

        begin : test6_block
            integer recovered;
            integer gv;
            recovered = 0;

            sample_valid = 1;
            for (stim_index = 0; stim_index < 40; stim_index = stim_index + 1)
begin
                sample_in = DIST_100M;
                @(posedge clk);
                gv = calculate_golden(DIST_100M);
                expected_queue.push_back(gv);      // <-- ADDED THIS LINE
                sample_count = sample_count + 1;

                if (!recovered && gv > THRESH) begin
                    recovered = 1;
```

```
                    $display("[PASS] TEST6: Filter recovered above threshold at
sample %0d (output=%0d)",
                             stim_index, gv);
                end
            end

            if (!recovered) begin
                $display("[FAIL] TEST6: Filter never recovered after obstacle
cleared");
                error_count = error_count + 1;
            end
        end

        flush_and_check_empty();

        // ============================================================
        // FINAL REPORT
        // ============================================================
        $display("\n================================================");
        $display("  proxcore FIR Filter Verification Complete");
        $display("================================================");
        $display("  Samples driven   : %0d", sample_count);
        $display("  Outputs checked  : %0d", total_checked);
        $display("  Matches          : %0d", match_count);
        $display("  Mismatches       : %0d", mismatch_count);
        $display("  Total errors     : %0d", error_count);
        $display("------------------------------------------------");
        if (error_count == 0)
            $display("  RESULT: PASS - all 6 tests passed");
        else
            $display("  RESULT: FAIL - %0d errors", error_count);
        $display("================================================\n");
        $finish;
    end

endmodule
```

- **B.3** config_regs_tb.sv — SPI config registers testbench (14 tests)

```
`timescale 1ns/1ps

module config_regs_tb;
```

```
    localparam time CLK_PERIOD = 20ns;
    localparam time SPI_HALF_PERIOD = 250ns;

    localparam logic [15:0] DEFAULT_THRESHOLD = 16'd2560;
    localparam logic [15:0] DEFAULT_COEFF0    = 16'sd112;
    localparam logic [15:0] DEFAULT_COEFF1    = 16'sd243;
    localparam logic [15:0] DEFAULT_COEFF2    = 16'sd618;
    localparam logic [15:0] DEFAULT_COEFF3    = 16'sd1293;
    localparam logic [15:0] DEFAULT_COEFF4    = 16'sd2217;
    localparam logic [15:0] DEFAULT_COEFF5    = 16'sd3225;
    localparam logic [15:0] DEFAULT_COEFF6    = 16'sd4089;
    localparam logic [15:0] DEFAULT_COEFF7    = 16'sd4587;

    logic        clk;
    logic        rst_n;
    logic        spi_csn;
    logic        spi_sck;
    logic        spi_mosi;
    logic [15:0] threshold;
    logic [15:0] coeff0;
    logic [15:0] coeff1;
    logic [15:0] coeff2;
    logic [15:0] coeff3;
    logic [15:0] coeff4;
    logic [15:0] coeff5;
    logic [15:0] coeff6;
    logic [15:0] coeff7;

    int unsigned failures;
    int unsigned test_id;
    int unsigned failures_before_test;

    config_regs #(
        .DEFAULT_THRESHOLD(DEFAULT_THRESHOLD),
        .DEFAULT_COEFF0(DEFAULT_COEFF0),
        .DEFAULT_COEFF1(DEFAULT_COEFF1),
        .DEFAULT_COEFF2(DEFAULT_COEFF2),
        .DEFAULT_COEFF3(DEFAULT_COEFF3),
        .DEFAULT_COEFF4(DEFAULT_COEFF4),
        .DEFAULT_COEFF5(DEFAULT_COEFF5),
        .DEFAULT_COEFF6(DEFAULT_COEFF6),
        .DEFAULT_COEFF7(DEFAULT_COEFF7)
    ) dut (
        .clk(clk),
```

```
        .rst_n(rst_n),
        .spi_csn(spi_csn),
        .spi_sck(spi_sck),
        .spi_mosi(spi_mosi),
        .threshold(threshold),
        .coeff0(coeff0),
        .coeff1(coeff1),
        .coeff2(coeff2),
        .coeff3(coeff3),
        .coeff4(coeff4),
        .coeff5(coeff5),
        .coeff6(coeff6),
        .coeff7(coeff7)
    );

    always #(CLK_PERIOD / 2) clk = ~clk;

    // ------------------------------------------------------------------
    // Helper tasks
    // ------------------------------------------------------------------
    task automatic wait_clocks(input integer cycles);
        begin
            repeat (cycles) @(posedge clk);
        end
    endtask

    task automatic apply_reset;
        begin
            rst_n    = 1'b0;
            spi_csn  = 1'b1;
            spi_sck  = 1'b0;
            spi_mosi = 1'b0;
            #(CLK_PERIOD + (CLK_PERIOD / 3));
            rst_n = 1'b1;
            wait_clocks(6);
        end
    endtask

    task automatic spi_clock_bit(input logic bit_value);
        begin
            spi_mosi = bit_value;
            #(SPI_HALF_PERIOD);
            spi_sck = 1'b1;
            #(SPI_HALF_PERIOD);
```

```
            spi_sck = 1'b0;
            #(SPI_HALF_PERIOD);
        end
    endtask

    task automatic spi_write_frame(input logic [7:0] addr, input logic [15:0]
data);
        logic [23:0] frame;
        int bit_idx;
        begin
            frame = {addr, data};
            spi_csn = 1'b0;
            #(SPI_HALF_PERIOD);

            for (bit_idx = 23; bit_idx >= 0; bit_idx--) begin
                spi_clock_bit(frame[bit_idx]);
            end

            spi_csn = 1'b1;
            spi_mosi = 1'b0;
            #(SPI_HALF_PERIOD);
            wait_clocks(4);
        end
    endtask

    task automatic spi_write_partial(input logic [23:0] frame, input integer
bits_to_send);
        int bit_idx;
        begin
            spi_csn = 1'b0;
            #(SPI_HALF_PERIOD);

            for (bit_idx = 23; bit_idx >= (24 - bits_to_send); bit_idx--)
begin
                spi_clock_bit(frame[bit_idx]);
            end

            spi_csn = 1'b1;
            spi_mosi = 1'b0;
            #(SPI_HALF_PERIOD);
            wait_clocks(4);
        end
    endtask
```

```
    // Clock 24 raw bits with CSN held low — used for back-to-back frames
    task automatic spi_clock_raw_frame(input logic [23:0] frame);
        int bit_idx;
        begin
            for (bit_idx = 23; bit_idx >= 0; bit_idx--) begin
                spi_clock_bit(frame[bit_idx]);
            end
        end
    endtask

    task automatic check_reg(input [127:0] name, input logic [15:0] actual,
input logic [15:0] expected);
        begin
            if (actual !== expected) begin
                failures = failures + 1'b1;
                $display("[%0t] ERROR: test %0d %0s expected 0x%04h, got
0x%04h",
                         $time, test_id, name, expected, actual);
            end
        end
    endtask

    task automatic check_defaults;
        begin
            check_reg("threshold", threshold, DEFAULT_THRESHOLD);
            check_reg("coeff0",    coeff0,    DEFAULT_COEFF0);
            check_reg("coeff1",    coeff1,    DEFAULT_COEFF1);
            check_reg("coeff2",    coeff2,    DEFAULT_COEFF2);
            check_reg("coeff3",    coeff3,    DEFAULT_COEFF3);
            check_reg("coeff4",    coeff4,    DEFAULT_COEFF4);
            check_reg("coeff5",    coeff5,    DEFAULT_COEFF5);
            check_reg("coeff6",    coeff6,    DEFAULT_COEFF6);
            check_reg("coeff7",    coeff7,    DEFAULT_COEFF7);
        end
    endtask

    task automatic check_all_coeffs(
        input logic [15:0] exp0, input logic [15:0] exp1,
        input logic [15:0] exp2, input logic [15:0] exp3,
        input logic [15:0] exp4, input logic [15:0] exp5,
        input logic [15:0] exp6, input logic [15:0] exp7
    );
        begin
            check_reg("coeff0", coeff0, exp0);
```

```
            check_reg("coeff1", coeff1, exp1);
            check_reg("coeff2", coeff2, exp2);
            check_reg("coeff3", coeff3, exp3);
            check_reg("coeff4", coeff4, exp4);
            check_reg("coeff5", coeff5, exp5);
            check_reg("coeff6", coeff6, exp6);
            check_reg("coeff7", coeff7, exp7);
        end
    endtask

    task automatic report_test_result;
        begin
            if (failures == failures_before_test) begin
                $display("[%0t] TESTCASE %0d PASS", $time, test_id);
            end else begin
                $display("[%0t] TESTCASE %0d FAIL", $time, test_id);
            end
        end
    endtask

    // ================================================================
    // MAIN STIMULUS
    // ================================================================
    initial begin
        clk      = 1'b0;
        rst_n    = 1'b1;
        spi_csn  = 1'b1;
        spi_sck  = 1'b0;
        spi_mosi = 1'b0;
        failures = 0;
        test_id  = 0;
        failures_before_test = 0;

        apply_reset();

        // ============================================================
        // TEST 1: Reset defaults — all registers at power-on values
        // ============================================================
        test_id = 1;
        failures_before_test = failures;
        $display("[%0t] TESTCASE %0d START: reset defaults", $time, test_id);
        $display("[%0t] TESTCASE %0d INPUTS: rst_n asserted low, SPI idle
with csn=1 sck=0 mosi=0",
                 $time, test_id);
```

```
        $display("[%0t] TESTCASE %0d STIMULUS DONE: checking default
threshold and coefficients",
                 $time, test_id);
        check_defaults();
        report_test_result();

        // ============================================================
        // TEST 2: Single threshold write — only threshold changes
        // ============================================================
        test_id = 2;
        failures_before_test = failures;
        $display("[%0t] TESTCASE %0d START: threshold write", $time,
test_id);
        $display("[%0t] TESTCASE %0d INPUTS: SPI frame addr=0x00
data=0x0A00",
                 $time, test_id);
        spi_write_frame(8'h00, 16'h0A00);
        $display("[%0t] TESTCASE %0d STIMULUS DONE: checking threshold update
only",
                 $time, test_id);
        check_reg("threshold", threshold, 16'h0A00);
        check_all_coeffs(
            DEFAULT_COEFF0, DEFAULT_COEFF1, DEFAULT_COEFF2, DEFAULT_COEFF3,
            DEFAULT_COEFF4, DEFAULT_COEFF5, DEFAULT_COEFF6, DEFAULT_COEFF7
        );
        report_test_result();

        // ============================================================
        // TEST 3: Write first and last coefficient
        // ============================================================
        test_id = 3;
        failures_before_test = failures;
        $display("[%0t] TESTCASE %0d START: coefficient register writes",
$time, test_id);
        $display("[%0t] TESTCASE %0d INPUTS: write addr=0x01 data=0x1111 and
addr=0x08 data=0x8888",
                 $time, test_id);
        spi_write_frame(8'h01, 16'h1111);
        spi_write_frame(8'h08, 16'h8888);
        $display("[%0t] TESTCASE %0d STIMULUS DONE: checking coeff0 and
coeff7 updates",
                 $time, test_id);
        check_reg("threshold", threshold, 16'h0A00);
        check_reg("coeff0", coeff0, 16'h1111);
```

```
        check_reg("coeff1", coeff1, DEFAULT_COEFF1);
        check_reg("coeff7", coeff7, 16'h8888);
        report_test_result();

        // ============================================================
        // TEST 4: Write all middle coefficients
        // ============================================================
        test_id = 4;
        failures_before_test = failures;
        $display("[%0t] TESTCASE %0d START: all middle coefficient writes",
$time, test_id);
        $display("[%0t] TESTCASE %0d INPUTS: write coeff1 through coeff6
using addresses 0x02 through 0x07",
                 $time, test_id);
        spi_write_frame(8'h02, 16'h2222);
        spi_write_frame(8'h03, 16'h3333);
        spi_write_frame(8'h04, 16'h4444);
        spi_write_frame(8'h05, 16'h5555);
        spi_write_frame(8'h06, 16'h6666);
        spi_write_frame(8'h07, 16'h7777);
        $display("[%0t] TESTCASE %0d STIMULUS DONE: checking every
coefficient register",
                 $time, test_id);
        check_all_coeffs(
            16'h1111, 16'h2222, 16'h3333, 16'h4444,
            16'h5555, 16'h6666, 16'h7777, 16'h8888
        );
        report_test_result();

        // ============================================================
        // TEST 5: Invalid address — no register changes
        // ============================================================
        test_id = 5;
        failures_before_test = failures;
        $display("[%0t] TESTCASE %0d START: invalid address ignored", $time,
test_id);
        $display("[%0t] TESTCASE %0d INPUTS: SPI frame addr=0x09
data=0xDEAD",
                 $time, test_id);
        spi_write_frame(8'h09, 16'hDEAD);
        $display("[%0t] TESTCASE %0d STIMULUS DONE: checking no register
changed",
                 $time, test_id);
        check_reg("threshold", threshold, 16'h0A00);
```

```
        check_all_coeffs(
            16'h1111, 16'h2222, 16'h3333, 16'h4444,
            16'h5555, 16'h6666, 16'h7777, 16'h8888
        );
        report_test_result();

        // ============================================================
        // TEST 6: Partial frame discarded by CSN going high
        // ============================================================
        test_id = 6;
        failures_before_test = failures;
        $display("[%0t] TESTCASE %0d START: partial frame discarded by csn
high",
                 $time, test_id);
        $display("[%0t] TESTCASE %0d INPUTS: send only 12 bits of addr=0x00
data=0x1234, then deassert csn",
                 $time, test_id);
        spi_write_partial(24'h00_1234, 12);
        $display("[%0t] TESTCASE %0d STIMULUS DONE: checking threshold did
not update",
                 $time, test_id);
        check_reg("threshold", threshold, 16'h0A00);
        report_test_result();

        // ============================================================
        // TEST 7: Full frame after partial — recovery works
        // ============================================================
        test_id = 7;
        failures_before_test = failures;
        $display("[%0t] TESTCASE %0d START: new full frame after partial
frame",
                 $time, test_id);
        $display("[%0t] TESTCASE %0d INPUTS: SPI frame addr=0x00
data=0x3456",
                 $time, test_id);
        spi_write_frame(8'h00, 16'h3456);
        $display("[%0t] TESTCASE %0d STIMULUS DONE: checking receiver
recovered and threshold updated",
                 $time, test_id);
        check_reg("threshold", threshold, 16'h3456);
        report_test_result();

        // ============================================================
        // TEST 8: Reset restores defaults after writes
```

```
        // ============================================================
        test_id = 8;
        failures_before_test = failures;
        $display("[%0t] TESTCASE %0d START: reset restores defaults after
writes",
                 $time, test_id);
        $display("[%0t] TESTCASE %0d INPUTS: assert reset after several
successful SPI writes",
                 $time, test_id);
        apply_reset();
        $display("[%0t] TESTCASE %0d STIMULUS DONE: checking all registers
returned to defaults",
                 $time, test_id);
        check_defaults();
        report_test_result();

        // ============================================================
        // TEST 9: Back-to-back frames without CSN toggle
        // Two 24-bit frames clocked with CSN held low continuously.
        // bit_cnt wraps after 24 bits, so both writes must land.
        // ============================================================
        test_id = 9;
        failures_before_test = failures;
        $display("[%0t] TESTCASE %0d START: back-to-back frames without CSN
toggle",
                 $time, test_id);
        $display("[%0t] TESTCASE %0d INPUTS: CSN held low, 48 bits: addr=0x00
data=0x1234 then addr=0x01 data=0xABCD",
                 $time, test_id);

        spi_csn = 1'b0;
        #(SPI_HALF_PERIOD);

        spi_clock_raw_frame({8'h00, 16'h1234});  // threshold = 0x1234
        spi_clock_raw_frame({8'h01, 16'hABCD});  // coeff0    = 0xABCD

        spi_csn  = 1'b1;
        spi_mosi = 1'b0;
        #(SPI_HALF_PERIOD);
        wait_clocks(4);

        $display("[%0t] TESTCASE %0d STIMULUS DONE: checking both writes
landed",
                 $time, test_id);
```

```
        check_reg("threshold", threshold, 16'h1234);
        check_reg("coeff0",    coeff0,    16'hABCD);
        report_test_result();

        // ============================================================
        // TEST 10: Overwrite same register twice — last write wins
        // ============================================================
        test_id = 10;
        failures_before_test = failures;
        $display("[%0t] TESTCASE %0d START: double write to same register",
$time, test_id);
        $display("[%0t] TESTCASE %0d INPUTS: write threshold=0xAAAA then
threshold=0xBBBB",
                 $time, test_id);
        spi_write_frame(8'h00, 16'hAAAA);
        spi_write_frame(8'h00, 16'hBBBB);
        $display("[%0t] TESTCASE %0d STIMULUS DONE: checking last write
wins",
                 $time, test_id);
        check_reg("threshold", threshold, 16'hBBBB);
        report_test_result();

        // ============================================================
        // TEST 11: Maximum address 0xFF — must be ignored
        // ============================================================
        test_id = 11;
        failures_before_test = failures;
        $display("[%0t] TESTCASE %0d START: max address 0xFF ignored", $time,
test_id);
        $display("[%0t] TESTCASE %0d INPUTS: SPI frame addr=0xFF
data=0xFFFF",
                 $time, test_id);
        spi_write_frame(8'hFF, 16'hFFFF);
        $display("[%0t] TESTCASE %0d STIMULUS DONE: checking no register
changed",
                 $time, test_id);
        check_reg("threshold", threshold, 16'hBBBB);
        check_reg("coeff0",    coeff0,    16'hABCD);
        report_test_result();

        // ============================================================
        // TEST 12: Write all zeros to all registers
        // ============================================================
        test_id = 12;
```

```
        failures_before_test = failures;
        $display("[%0t] TESTCASE %0d START: write all zeros", $time,
test_id);
        $display("[%0t] TESTCASE %0d INPUTS: write 0x0000 to threshold and
all 8 coefficients",
                 $time, test_id);
        spi_write_frame(8'h00, 16'h0000);
        spi_write_frame(8'h01, 16'h0000);
        spi_write_frame(8'h02, 16'h0000);
        spi_write_frame(8'h03, 16'h0000);
        spi_write_frame(8'h04, 16'h0000);
        spi_write_frame(8'h05, 16'h0000);
        spi_write_frame(8'h06, 16'h0000);
        spi_write_frame(8'h07, 16'h0000);
        spi_write_frame(8'h08, 16'h0000);
        $display("[%0t] TESTCASE %0d STIMULUS DONE: checking all registers
are zero",
                 $time, test_id);
        check_reg("threshold", threshold, 16'h0000);
        check_all_coeffs(
            16'h0000, 16'h0000, 16'h0000, 16'h0000,
            16'h0000, 16'h0000, 16'h0000, 16'h0000
        );
        report_test_result();

        // ============================================================
        // TEST 13: Write all ones to all registers
        // ============================================================
        test_id = 13;
        failures_before_test = failures;
        $display("[%0t] TESTCASE %0d START: write all ones", $time, test_id);
        $display("[%0t] TESTCASE %0d INPUTS: write 0xFFFF to threshold and
all 8 coefficients",
                 $time, test_id);
        spi_write_frame(8'h00, 16'hFFFF);
        spi_write_frame(8'h01, 16'hFFFF);
        spi_write_frame(8'h02, 16'hFFFF);
        spi_write_frame(8'h03, 16'hFFFF);
        spi_write_frame(8'h04, 16'hFFFF);
        spi_write_frame(8'h05, 16'hFFFF);
        spi_write_frame(8'h06, 16'hFFFF);
        spi_write_frame(8'h07, 16'hFFFF);
        spi_write_frame(8'h08, 16'hFFFF);
```

```
        $display("[%0t] TESTCASE %0d STIMULUS DONE: checking all registers
are 0xFFFF",
                 $time, test_id);
        check_reg("threshold", threshold, 16'hFFFF);
        check_all_coeffs(
            16'hFFFF, 16'hFFFF, 16'hFFFF, 16'hFFFF,
            16'hFFFF, 16'hFFFF, 16'hFFFF, 16'hFFFF
        );
        report_test_result();

        // ============================================================
        // TEST 14: Reset after all-ones — confirms defaults restored
        // ============================================================
        test_id = 14;
        failures_before_test = failures;
        $display("[%0t] TESTCASE %0d START: final reset restores real filter
defaults",
                 $time, test_id);
        $display("[%0t] TESTCASE %0d INPUTS: assert reset after all-ones
write",
                 $time, test_id);
        apply_reset();
        $display("[%0t] TESTCASE %0d STIMULUS DONE: checking all registers
returned to filter defaults",
                 $time, test_id);
        check_defaults();
        report_test_result();

        // ============================================================
        // FINAL REPORT
        // ============================================================
        $display("\n================================================");
        $display("  proxcore config_regs Verification");
        $display("================================================");
        $display("  Total failures: %0d", failures);
        if (failures == 0)
            $display("  RESULT: PASS — all %0d tests passed", test_id);
        else
            $display("  RESULT: FAIL — %0d error(s)", failures);
        $display("================================================\n");

        if (failures != 0) $fatal(1);
        $finish;
    end
```

```
    initial begin
        #(4ms);
        $display("[%0t] ERROR: simulation timeout", $time);
        $fatal(1);
    end

endmodule
```

- **B.4** tb_threshold_fsm.sv — Threshold FSM testbench (13 tests)

```
`timescale 1ns/1ps

module tb_threshold_fsm;

    localparam time   CLK_PERIOD = 20;        // 50 MHz
    localparam [15:0] THRESHOLD  = 16'd2560;  // 40m in Q10.6
    localparam [15:0] DIST_100M  = 16'd6400;  // clear track
    localparam [15:0] DIST_30M   = 16'd1920;  // real obstacle
    localparam [15:0] DIST_45M   = 16'd2880;  // just above threshold
    localparam [15:0] DIST_39M   = 16'd2496;  // 39m, just below threshold

    logic        clk;
    logic        rst_n;
    logic [15:0] filtered_in;
    logic        data_valid;
    logic [15:0] threshold;
    logic        brake_irq;

    integer      failures;
    integer      test_id;
    integer      failures_before_test;
    logic        brake_irq_d;

    threshold_fsm dut (
        .clk         (clk),
        .rst_n       (rst_n),
        .filtered_in (filtered_in),
        .data_valid  (data_valid),
        .threshold   (threshold),
        .brake_irq   (brake_irq)
```

```
);

initial clk = 1'b0;
always #(CLK_PERIOD / 2) clk = ~clk;

// Pulse-width monitor — brake_irq must never be high two cycles running
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        brake_irq_d <= 1'b0;
    else begin
        if (brake_irq && brake_irq_d) begin
            failures = failures + 1;
            $display("[%0t] ERROR: brake_irq held high >1 cycle", $time);
        end
        brake_irq_d <= brake_irq;
    end
end

// Watchdog
initial begin
    #(600 * CLK_PERIOD);
    $display("[%0t] FATAL: timeout", $time);
    $fatal(1);
end

// ---------------------------------------------------------------
// Tasks — all stimulus driven on the NEGEDGE to avoid posedge races
// Outputs checked mid-cycle (after negedge that follows the posedge)
// ---------------------------------------------------------------

task automatic apply_reset;
    begin
        rst_n       = 1'b0;
        filtered_in = DIST_100M;
        data_valid  = 1'b0;
        threshold   = THRESHOLD;
        repeat (4) @(posedge clk);
        @(negedge clk); rst_n = 1'b1;
        @(posedge clk); // one settled clock after reset
    end
endtask

// Drive one valid sample: set on negedge, captured on next posedge
task automatic drive_sample;
```

```
        input [15:0] distance;
        begin
            @(negedge clk);
            filtered_in = distance;
            data_valid  = 1'b1;
            @(posedge clk); // DUT captures here
            @(negedge clk); // settle
            data_valid  = 1'b0;
            filtered_in = DIST_100M;
        end
    endtask

    // Drive N identical samples
    task automatic drive_n;
        input [15:0] distance;
        input integer n;
        integer i;
        begin
            for (i = 0; i < n; i = i + 1)
                drive_sample(distance);
        end
    endtask

    // Idle for N full clock cycles
    task automatic idle_cycles;
        input integer n;
        begin
            data_valid = 1'b0;
            repeat (n) @(posedge clk);
        end
    endtask

    // Check brake_irq mid-cycle (on negedge after a posedge event)
    // Returns the sampled value
    task automatic sample_irq;
        output logic val;
        begin
            @(negedge clk);
            val = brake_irq;
        end
    endtask

    // ================================================================
    // MAIN STIMULUS
```

```
    // ============================================================
    initial begin
        failures = 0;
        test_id  = 0;
        apply_reset();

        // ============================================================
        // TEST 1: Idle — no valid samples, brake_irq must stay low
        // ============================================================
        test_id = 1; failures_before_test = failures;
        $display("[%0t] TEST %0d START: idle, no samples", $time, test_id);

        idle_cycles(10);
        @(negedge clk);
        if (brake_irq !== 1'b0) begin
            failures = failures + 1;
            $display("[%0t] ERROR test %0d: brake_irq high during idle", $time,
test_id);
        end

        $display("[%0t] TEST %0d %s", $time, test_id,
                 (failures==failures_before_test) ? "PASS" : "FAIL");

        // ============================================================
        // TEST 2: Clear track — 10 x 100m, no IRQ
        // ============================================================
        test_id = 2; failures_before_test = failures;
        apply_reset();
        $display("[%0t] TEST %0d START: clear track 10 x 100m", $time, test_id);

        drive_n(DIST_100M, 10);
        @(negedge clk);
        if (brake_irq !== 1'b0) begin
            failures = failures + 1;
            $display("[%0t] ERROR test %0d: brake_irq high on clear track",
$time, test_id);
        end

        $display("[%0t] TEST %0d %s", $time, test_id,
                 (failures==failures_before_test) ? "PASS" : "FAIL");

        // ============================================================
        // TEST 3: One sub-threshold sample — must NOT fire
        // ============================================================
```

```
        test_id = 3; failures_before_test = failures;
        apply_reset();
        $display("[%0t] TEST %0d START: single sub-threshold must not fire",
$time, test_id);

        drive_sample(DIST_30M);
        @(negedge clk);
        if (brake_irq !== 1'b0) begin
            failures = failures + 1;
            $display("[%0t] ERROR test %0d: fired after 1 sub-threshold", $time,
test_id);
        end

        $display("[%0t] TEST %0d %s", $time, test_id,
                 (failures==failures_before_test) ? "PASS" : "FAIL");

        // ============================================================
        // TEST 4: Two sub-threshold samples — must NOT fire
        // ============================================================
        test_id = 4; failures_before_test = failures;
        apply_reset();
        $display("[%0t] TEST %0d START: two sub-threshold must not fire", $time,
test_id);

        drive_sample(DIST_30M);
        drive_sample(DIST_30M);
        @(negedge clk);
        if (brake_irq !== 1'b0) begin
            failures = failures + 1;
            $display("[%0t] ERROR test %0d: fired after 2 sub-threshold", $time,
test_id);
        end

        $display("[%0t] TEST %0d %s", $time, test_id,
                 (failures==failures_before_test) ? "PASS" : "FAIL");

        // ============================================================
        // TEST 5: Three consecutive sub-threshold — MUST fire as 1-cycle pulse
        // ============================================================
        test_id = 5; failures_before_test = failures;
        apply_reset();
        $display("[%0t] TEST %0d START: three consecutive fires single pulse",
$time, test_id);
```

```
        drive_sample(DIST_30M);   // WARN1
        drive_sample(DIST_30M);   // WARN2
        drive_sample(DIST_30M);   // BRAKING — IRQ fires

        // Check: brake_irq must be high in the cycle after the 3rd posedge
        // drive_sample ends after @(negedge clk) following the posedge
        // so brake_irq is readable right now (mid-cycle)
        if (brake_irq !== 1'b1) begin
            failures = failures + 1;
            $display("[%0t] ERROR test %0d: brake_irq not high after 3rd sample
negedge",
                     $time, test_id);
        end

        // Must deassert by next negedge
        @(posedge clk); @(negedge clk);
        if (brake_irq !== 1'b0) begin
            failures = failures + 1;
            $display("[%0t] ERROR test %0d: brake_irq did not deassert after 1
cycle",
                     $time, test_id);
        end

        $display("[%0t] TEST %0d %s", $time, test_id,
                 (failures==failures_before_test) ? "PASS" : "FAIL");

        // ============================================================
        // TEST 6: Debounce reset at WARN1
        // sub → clear → sub → sub — only 2 consecutive, must NOT fire
        // ============================================================
        test_id = 6; failures_before_test = failures;
        apply_reset();
        $display("[%0t] TEST %0d START: clear at WARN1 resets debounce", $time,
test_id);

        drive_sample(DIST_30M);   // WARN1
        drive_sample(DIST_100M);  // CLEAR — resets counter
        drive_sample(DIST_30M);   // WARN1
        drive_sample(DIST_30M);   // WARN2
        @(negedge clk);
        if (brake_irq !== 1'b0) begin
            failures = failures + 1;
            $display("[%0t] ERROR test %0d: fired despite debounce reset at
WARN1",
```

```
                    $time, test_id);
        end

        $display("[%0t] TEST %0d %s", $time, test_id,
                 (failures==failures_before_test) ? "PASS" : "FAIL");

        // ============================================================
        // TEST 7: Debounce reset at WARN2
        // sub → sub → clear → sub → sub — must NOT fire
        // ============================================================
        test_id = 7; failures_before_test = failures;
        apply_reset();
        $display("[%0t] TEST %0d START: clear at WARN2 resets debounce", $time,
test_id);

        drive_sample(DIST_30M);   // WARN1
        drive_sample(DIST_30M);   // WARN2
        drive_sample(DIST_100M);  // CLEAR
        drive_sample(DIST_30M);   // WARN1
        drive_sample(DIST_30M);   // WARN2
        @(negedge clk);
        if (brake_irq !== 1'b0) begin
            failures = failures + 1;
            $display("[%0t] ERROR test %0d: fired despite debounce reset at
WARN2",
                    $time, test_id);
        end

        $display("[%0t] TEST %0d %s", $time, test_id,
                 (failures==failures_before_test) ? "PASS" : "FAIL");

        // ============================================================
        // TEST 8: No repeated IRQ in BRAKING state
        // ============================================================
        test_id = 8; failures_before_test = failures;
        apply_reset();
        $display("[%0t] TEST %0d START: no repeated IRQ in BRAKING", $time,
test_id);

        drive_sample(DIST_30M);
        drive_sample(DIST_30M);
        drive_sample(DIST_30M);   // enters BRAKING, IRQ fires once

        // 10 more sub-threshold — pulse monitor catches re-assertion
```

```
        drive_n(DIST_30M, 10);
        @(negedge clk);
        if (brake_irq !== 1'b0) begin
            failures = failures + 1;
            $display("[%0t] ERROR test %0d: brake_irq still high", $time,
test_id);
        end

        $display("[%0t] TEST %0d %s", $time, test_id,
                 (failures==failures_before_test) ? "PASS" : "FAIL");

        // ============================================================
        // TEST 9: Recovery from BRAKING, then second obstacle detected
        // ============================================================
        test_id = 9; failures_before_test = failures;
        apply_reset();
        $display("[%0t] TEST %0d START: recovery and re-detection", $time,
test_id);

        drive_sample(DIST_30M);
        drive_sample(DIST_30M);
        drive_sample(DIST_30M);   // BRAKING, IRQ fires

        drive_sample(DIST_100M);  // back to CLEAR

        drive_sample(DIST_30M);   // WARN1
        drive_sample(DIST_30M);   // WARN2
        drive_sample(DIST_30M);   // BRAKING again, IRQ fires

        if (brake_irq !== 1'b1) begin
            failures = failures + 1;
            $display("[%0t] ERROR test %0d: second obstacle not detected", $time,
test_id);
        end

        $display("[%0t] TEST %0d %s", $time, test_id,
                 (failures==failures_before_test) ? "PASS" : "FAIL");

        // ============================================================
        // TEST 10: Boundary — exactly at threshold must NOT fire
        // ============================================================
        test_id = 10; failures_before_test = failures;
        apply_reset();
```

```
        $display("[%0t] TEST %0d START: value == threshold must not fire", $time,
test_id);

        drive_n(THRESHOLD, 5);
        @(negedge clk);
        if (brake_irq !== 1'b0) begin
            failures = failures + 1;
            $display("[%0t] ERROR test %0d: fired at exact threshold", $time,
test_id);
        end

        $display("[%0t] TEST %0d %s", $time, test_id,
                 (failures==failures_before_test) ? "PASS" : "FAIL");

        // ============================================================
        // TEST 11: Boundary — one below threshold MUST fire after 3
        // DIST_39M = 2496 (39m) < threshold (2560)
        // ============================================================
        test_id = 11; failures_before_test = failures;
        apply_reset();
        $display("[%0t] TEST %0d START: threshold-1 fires after 3 samples",
$time, test_id);

        drive_sample(DIST_39M);
        drive_sample(DIST_39M);
        drive_sample(DIST_39M);   // must fire

        if (brake_irq !== 1'b1) begin
            failures = failures + 1;
            $display("[%0t] ERROR test %0d: did not fire at 39m", $time,
test_id);
        end

        $display("[%0t] TEST %0d %s", $time, test_id,
                 (failures==failures_before_test) ? "PASS" : "FAIL");

        // ============================================================
        // TEST 12: data_valid gating — FSM frozen when data_valid=0
        // ============================================================
        test_id = 12; failures_before_test = failures;
        apply_reset();
        $display("[%0t] TEST %0d START: FSM frozen when data_valid low", $time,
test_id);
```

```
        filtered_in = DIST_30M;
        data_valid  = 1'b0;
        repeat (20) @(posedge clk);
        @(negedge clk);

        if (brake_irq !== 1'b0) begin
            failures = failures + 1;
            $display("[%0t] ERROR test %0d: fired with data_valid low", $time,
test_id);
        end

        filtered_in = DIST_100M;

        $display("[%0t] TEST %0d %s", $time, test_id,
                 (failures==failures_before_test) ? "PASS" : "FAIL");

        // ============================================================
        // TEST 13: Runtime threshold change
        // ============================================================
        test_id = 13; failures_before_test = failures;
        apply_reset();
        $display("[%0t] TEST %0d START: runtime threshold change", $time,
test_id);

        // 45m is safe at 40m threshold
        drive_n(DIST_45M, 5);
        @(negedge clk);
        if (brake_irq !== 1'b0) begin
            failures = failures + 1;
            $display("[%0t] ERROR test %0d: fired at 45m with 40m threshold",
$time, test_id);
        end

        // Raise threshold to 50m — now 45m should trigger
        @(negedge clk); threshold = 16'd3200;
        drive_sample(DIST_45M);  // WARN1 (2880 < 3200)
        drive_sample(DIST_45M);  // WARN2
        drive_sample(DIST_45M);  // BRAKING

        if (brake_irq !== 1'b1) begin
            failures = failures + 1;
            $display("[%0t] ERROR test %0d: did not fire after threshold raised",
$time, test_id);
        end
```

```
        @(negedge clk); threshold = THRESHOLD;

        $display("[%0t] TEST %0d %s", $time, test_id,
                 (failures==failures_before_test) ? "PASS" : "FAIL");

        // ============================================================
        // FINAL REPORT
        // ============================================================
        $display("\n================================================");
        $display("  proxcore threshold_fsm Verification");
        $display("================================================");
        $display("  Total failures: %0d", failures);
        if (failures == 0)
            $display("  RESULT: PASS — all 13 tests passed");
        else
            $display("  RESULT: FAIL — %0d error(s)", failures);
        $display("================================================\n");

        if (failures != 0) $fatal(1);
        $finish;
    end

endmodule
```

- **B.5** deserializer_tb.sv — UART deserializer testbench (5 tests)

```
`timescale 1ns/1ps

module tb_deserializer_gc;

    // ============================================================
    // Parameters
    // ============================================================
    localparam int unsigned CLK_FREQ_HZ = 25_000_000;
    localparam time         CLK_PERIOD  = 20ns;

    // Test baud rate constants (for comments and reference)
    localparam int unsigned BAUD_9600   = 5208;
    localparam int unsigned BAUD_115200 = 434;
    localparam int unsigned BAUD_230400 = 217;
    localparam int unsigned BAUD_460800 = 108;
```

```
    localparam int unsigned BAUD_921600 = 54;

    // ================================================================
    // DUT signals
    // ================================================================
    logic        clk;
    logic        rst_n;
    logic        rx;
    logic [15:0] baud_div_tb;
    logic [15:0] data_out;
    logic        data_valid;

    // ================================================================
    // Test signals
    // ================================================================
    int unsigned failures;
    int unsigned test_id;
    int unsigned failures_before;
    int unsigned pulse_count;
    time         BIT_TIME;

    // ================================================================
    // DUT instantiation
    // ================================================================
    deserializer_gc dut (
        .clk        (clk),
        .rst_n      (rst_n),
        .rx         (rx),
        .baud_div   (baud_div_tb),
        .data_out   (data_out),
        .data_valid (data_valid)
    );

    // ================================================================
    // Clock generation
    // ================================================================
    initial clk = 1'b0;
    always #(CLK_PERIOD / 2) clk = ~clk;

    // ================================================================
    // data_valid pulse monitor
    // ================================================================
    always @(posedge clk) begin
        if (data_valid && rst_n)
```

```
            pulse_count = pulse_count + 1;
    end

    // ================================================================
    // Watchdog timer
    // ================================================================
    initial begin
        #(50ms);
        $display("[FATAL] Simulation timeout");
        $fatal(1);
    end

    // ================================================================
    // UART transmission task — accepts bit period as parameter
    // ================================================================
    task automatic uart_send_byte(input logic [7:0] data, input time
bit_period);
        integer i;
        begin
            // Start bit
            rx = 1'b0;
            #(bit_period);
            // Data bits (LSB first)
            for (i = 0; i < 8; i++) begin
                rx = data[i];
                #(bit_period);
            end
            // Stop bit
            rx = 1'b1;
            #(bit_period);
        end
    endtask

    task automatic uart_send_word(input logic [15:0] data, input time
bit_period);
        begin
            uart_send_byte(data[7:0], bit_period);    // low byte first
            uart_send_byte(data[15:8], bit_period);
        end
    endtask

    // ================================================================
    // Reset task
    // ================================================================
```

```
    task automatic apply_reset;
        begin
            rst_n = 1'b0;
            rx    = 1'b1;  // UART idle
            repeat (4) @(posedge clk);
            rst_n = 1'b1;
            @(posedge clk);
        end
    endtask

    // ============================================================
    // Helper tasks
    // ============================================================
    task automatic wait_cycles(input integer n);
        begin
            repeat (n) @(posedge clk);
        end
    endtask

    // ============================================================
    // Result reporting
    // ============================================================
    task automatic report;
        begin
            if (failures == failures_before)
                $display("[%0t] TEST %0d PASS", $time, test_id);
            else
                $display("[%0t] TEST %0d FAIL", $time, test_id);
        end
    endtask

    // ============================================================
    // MAIN STIMULUS
    // ============================================================
    initial begin
        clk             = 1'b0;
        rst_n           = 1'b1;
        rx              = 1'b1;
        baud_div_tb     = BAUD_460800;
        failures        = 0;
        pulse_count     = 0;
        BIT_TIME        = baud_div_tb * CLK_PERIOD;

        apply_reset();
```

```
        // ============================================================
        // TEST 1: Single 16-bit word at 460800 baud
        //
        // Verify the basic deserializer operation with baud_div driven
        // as a signal instead of parameter.
        // ============================================================
        test_id = 1;
        failures_before = failures;
        $display("[%0t] TEST %0d START: Single word at 460800 baud", $time,
test_id);

        apply_reset();
        baud_div_tb = BAUD_460800;
        BIT_TIME = baud_div_tb * CLK_PERIOD;
        pulse_count = 0;

        uart_send_word(16'hDEAD, BIT_TIME);
        wait_cycles(500);  // Allow time for processing

        if (pulse_count != 1) begin
            failures++;
            $display("[%0t] ERROR test %0d: expected 1 pulse, got %0d",
$time, test_id, pulse_count);
        end
        if (data_out != 16'hDEAD) begin
            failures++;
            $display("[%0t] ERROR test %0d: expected 0xDEAD, got 0x%04X",
$time, test_id, data_out);
        end

        report();

        // ============================================================
        // TEST 2: Multiple words at 460800 baud
        //
        // Verify back-to-back word reception.
        // ============================================================
        test_id = 2;
        failures_before = failures;
        $display("[%0t] TEST %0d START: Multiple words at 460800 baud",
$time, test_id);

        apply_reset();
```

```
        baud_div_tb = BAUD_460800;
        BIT_TIME = baud_div_tb * CLK_PERIOD;
        pulse_count = 0;

        uart_send_word(16'h1234, BIT_TIME);
        uart_send_word(16'h5678, BIT_TIME);
        uart_send_word(16'hABCD, BIT_TIME);
        wait_cycles(1000);

        if (pulse_count != 3) begin
            failures++;
            $display("[%0t] ERROR test %0d: expected 3 pulses, got %0d",
$time, test_id, pulse_count);
        end

        report();

        // ============================================================
        // TEST 3: Glitch rejection on start bit
        //
        // Send a short pulse on rx (glitch), then a valid word.
        // Deserializer should reject the glitch and correctly receive
        // the subsequent word.
        // ============================================================
        test_id = 3;
        failures_before = failures;
        $display("[%0t] TEST %0d START: Glitch rejection on start bit",
$time, test_id);

        apply_reset();
        baud_div_tb = BAUD_460800;
        BIT_TIME = baud_div_tb * CLK_PERIOD;
        pulse_count = 0;

        // Send a glitch: short pulse on rx
        rx = 1'b0;
        #(BIT_TIME / 4);  // Only 1/4 bit period
        rx = 1'b1;
        wait_cycles(10);

        // Now send valid word
        uart_send_word(16'hBEEF, BIT_TIME);
        wait_cycles(500);
```

```
        if (pulse_count != 1) begin
            failures++;
            $display("[%0t] ERROR test %0d: expected 1 pulse (glitch
rejected), got %0d",
                     $time, test_id, pulse_count);
        end
        if (data_out != 16'hBEEF) begin
            failures++;
            $display("[%0t] ERROR test %0d: expected 0xBEEF after glitch, got
0x%04X",
                     $time, test_id, data_out);
        end

        report();

        // ============================================================
        // TEST 4: Framing error handling
        //
        // Send an incomplete word (no stop bit), then a valid word.
        // Deserializer should drop the malformed word and correctly
        // receive the next one.
        // ============================================================
        test_id = 4;
        failures_before = failures;
        $display("[%0t] TEST %0d START: Framing error handling", $time,
test_id);

        apply_reset();
        baud_div_tb = BAUD_460800;
        BIT_TIME = baud_div_tb * CLK_PERIOD;
        pulse_count = 0;

        // Send partial word: start bit, low byte, low byte (no stop, low
stop bit)
        rx = 1'b0;
        #(BIT_TIME);  // Start bit
        for (int i = 0; i < 8; i++) begin
            rx = i[0];
            #(BIT_TIME);
        end
        rx = 1'b0;  // Stop bit is low (framing error)
        #(BIT_TIME);
        rx = 1'b1;
        wait_cycles(100);
```

```
        // Now send a valid word
        uart_send_word(16'h7777, BIT_TIME);
        wait_cycles(500);

        if (pulse_count != 1) begin
            failures++;
            $display("[%0t] ERROR test %0d: expected 1 pulse (error dropped),
got %0d",
                     $time, test_id, pulse_count);
        end
        if (data_out != 16'h7777) begin
            failures++;
            $display("[%0t] ERROR test %0d: expected 0x7777 after error, got
0x%04X",
                     $time, test_id, data_out);
        end

        report();

        // ============================================================
        // TEST 5: Byte order verification (LOW_BYTE_FIRST)
        //
        // Verify that the low byte is received first and assembled
        // into the output as {high_byte, low_byte}.
        // ============================================================
        test_id = 5;
        failures_before = failures;
        $display("[%0t] TEST %0d START: Byte order (LOW_BYTE_FIRST)", $time,
test_id);

        apply_reset();
        baud_div_tb = BAUD_460800;
        BIT_TIME = baud_div_tb * CLK_PERIOD;
        pulse_count = 0;

        // Send low byte 0x34, then high byte 0x12 → should output 0x1234
        uart_send_byte(8'h34, BIT_TIME);
        uart_send_byte(8'h12, BIT_TIME);
        wait_cycles(500);

        if (pulse_count != 1) begin
            failures++;
```

```
            $display("[%0t] ERROR test %0d: expected 1 pulse, got %0d",
$time, test_id, pulse_count);
        end
        if (data_out != 16'h1234) begin
            failures++;
            $display("[%0t] ERROR test %0d: byte order wrong — expected
0x1234, got 0x%04X",
                     $time, test_id, data_out);
        end

        report();

        // ============================================================
        // TEST 6: Runtime baud rate change
        //
        // Start at 460800 baud (baud_div=108).
        // Send one valid word at 460800 baud. Verify received correctly.
        // Change baud_div to 115200 baud (baud_div=434) — drive port
directly.
        // Wait 4 clock cycles for the change to take effect.
        // Send one valid word at 115200 baud timing. Verify received
correctly.
        // ============================================================
        test_id = 6;
        failures_before = failures;
        $display("[%0t] TEST %0d START: Runtime baud rate change", $time,
test_id);

        apply_reset();
        baud_div_tb = BAUD_460800;
        BIT_TIME = baud_div_tb * CLK_PERIOD;
        pulse_count = 0;

        // Send at 460800 baud
        uart_send_word(16'hC0FF, BIT_TIME);
        wait_cycles(100);

        if (pulse_count != 1) begin
            failures++;
            $display("[%0t] ERROR test %0d phase A: expected 1 pulse at
460800, got %0d",
                     $time, test_id, pulse_count);
        end
        if (data_out != 16'hC0FF) begin
```

```
            failures++;
            $display("[%0t] ERROR test %0d phase A: expected 0xC0FF, got
0x%04X",
                     $time, test_id, data_out);
        end

        // Change baud rate to 115200
        baud_div_tb = BAUD_115200;
        BIT_TIME = baud_div_tb * CLK_PERIOD;
        wait_cycles(4);  // Allow 4 cycles for change to propagate

        pulse_count = 0;

        // Send at 115200 baud
        uart_send_word(16'hEEEE, BIT_TIME);
        wait_cycles(500);  // Longer wait for slower baud rate

        if (pulse_count != 1) begin
            failures++;
            $display("[%0t] ERROR test %0d phase B: expected 1 pulse at
115200, got %0d",
                     $time, test_id, pulse_count);
        end
        if (data_out != 16'hEEEE) begin
            failures++;
            $display("[%0t] ERROR test %0d phase B: expected 0xEEEE, got
0x%04X",
                     $time, test_id, data_out);
        end

        report();

        // ============================================================
        // TEST 7: 9600 baud — slowest supported rate
        //
        // Set baud_div=5208 (9600 baud).
        // Send word 0xABCD at 9600 baud timing.
        // Verify received correctly.
        // This proves the 16-bit baud_cnt_q is wide enough.
        // ============================================================
        test_id = 7;
        failures_before = failures;
        $display("[%0t] TEST %0d START: 9600 baud (slowest)", $time,
test_id);
```

```
        apply_reset();
        baud_div_tb = BAUD_9600;  // 5208
        BIT_TIME = baud_div_tb * CLK_PERIOD;
        pulse_count = 0;

        uart_send_word(16'hABCD, BIT_TIME);
        wait_cycles(600000);  // Much longer wait for very slow baud rate

        if (pulse_count != 1) begin
            failures++;
            $display("[%0t] ERROR test %0d: expected 1 pulse at 9600, got
%0d",
                     $time, test_id, pulse_count);
        end
        if (data_out != 16'hABCD) begin
            failures++;
            $display("[%0t] ERROR test %0d: expected 0xABCD at 9600, got
0x%04X",
                     $time, test_id, data_out);
        end

        report();

        // ============================================================
        // TEST 8: 921600 baud — fastest supported rate
        //
        // Set baud_div=54 (921600 baud).
        // Send word 0x1234 at 921600 baud timing.
        // Verify received correctly.
        // This proves fast baud rates work with smaller counters.
        // ============================================================
        test_id = 8;
        failures_before = failures;
        $display("[%0t] TEST %0d START: 921600 baud (fastest)", $time,
test_id);

        apply_reset();
        baud_div_tb = BAUD_921600;  // 54
        BIT_TIME = baud_div_tb * CLK_PERIOD;
        pulse_count = 0;

        uart_send_word(16'h1234, BIT_TIME);
        wait_cycles(1500);  // Shorter wait for very fast baud rate
```

```
        if (pulse_count != 1) begin
            failures++;
            $display("[%0t] ERROR test %0d: expected 1 pulse at 921600, got
%0d",
                     $time, test_id, pulse_count);
        end
        if (data_out != 16'h1234) begin
            failures++;
            $display("[%0t] ERROR test %0d: expected 0x1234 at 921600, got
0x%04X",
                     $time, test_id, data_out);
        end

        report();

        // ============================================================
        // FINAL REPORT
        // ============================================================
        $display("\n================================================");
        $display("  Deserializer_GC Verification");
        $display("================================================");
        $display("  Total failures: %0d", failures);
        if (failures == 0)
            $display("  RESULT: PASS — all %0d tests passed", test_id);
        else
            $display("  RESULT: FAIL — %0d error(s)", failures);
        $display("================================================\n");

        if (failures != 0) $fatal(1);
        $finish;
    end

endmodule
```

Appendix C: SPI Register Map Quick Reference

A single-page summary of all registers — addresses, defaults, encodings, and valid ranges. This is the page a firmware engineer would print out and keep at their desk.

[TABLE C.1: SPI Register Quick Reference]

Addr	Name	Default	Format	Valid Range	Description
0x00	threshold	2,560	Q10.6 unsigned	0–65,535	Braking distance (default 40 m)
0x01	coeff0	112	Q1.15 signed	−32,768 to 32,767	FIR h[0], h[15]
0x02	coeff1	243	Q1.15 signed	−32,768 to 32,767	FIR h[1], h[14]
0x03	coeff2	618	Q1.15 signed	−32,768 to 32,767	FIR h[2], h[13]
0x04	coeff3	1,293	Q1.15 signed	−32,768 to 32,767	FIR h[3], h[12]
0x05	coeff4	2,217	Q1.15 signed	−32,768 to 32,767	FIR h[4], h[11]
0x06	coeff5	3,225	Q1.15 signed	−32,768 to 32,767	FIR h[5], h[10]
0x07	coeff6	4,089	Q1.15 signed	−32,768 to 32,767	FIR h[6], h[9]
0x08	coeff7	4,587	Q1.15 signed	−32,768 to 32,767	FIR h[7], h[8]
0x09	baud_div	54	Unsigned int	27–65,535	CLK_FREQ / BAUD_RATE (default 460,800 baud)

SPI Frame Format: 24 bits, MSB first — [addr 7:0][data 15:0]. CPOL = 0, CPHA = 0. Maximum SCK = 5 MHz.

Appendix D: Baud Rate Configuration Quick Reference

[TABLE D.1: Baud Rate Divider Values]

Baud Rate	baud_div @ 25 MHz	baud_div @ 50 MHz	Bit Period	Byte Time (10 bits)	Word Time (20 bits)
9,600	2,604	5,208	104.16 µs	1.042 ms	2.083 ms
19,200	1,302	2,604	52.08 µs	520.8 µs	1.042 ms
38,400	651	1,302	26.04 µs	260.4 µs	520.8 µs
57,600	434	868	17.36 µs	173.6 µs	347.2 µs
115,200	217	434	8.68 µs	86.8 µs	173.6 µs
230,400	108	217	4.32 µs	43.2 µs	86.4 µs
460,800	54	108	2.16 µs	21.6 µs	43.2 µs
921,600	27	54	1.08 µs	10.8 µs	21.6 µs

Appendix E: Final Signoff Metrics — Full Dump

A condensed reference of all signoff metrics produced by the OpenLane 2 flow.

[TABLE E.1: Complete Signoff Metric Reference]

Category	Metric	Value
Lint	Errors	0
	Timing-construct issues	0
	Warnings	14 (stylistic)

	Inferred latches	0
Synthesis	Check errors	0
	Total instances	225,300
	Std-cell instances	39,480
	Std-cell area	220,075 μm^2
	Sequential cells	585
	Multi-input combinational	21,469
	Inverters	3,704
	Buffers	622
	Timing-repair buffers	409
	Clock buffers	104
	Clock inverters	45
	Hold-fix buffers	8
	Antenna diodes	47
	Tap cells	12,495
	Fill cells	185,820
	Unmapped instances	0
Floorplan	Die area	907,861 μm^2
	Core area	876,865 μm^2
	Core utilization	25.10%
	Std-cell rows	371

	Site count	700,819
	Pad I/O count	233
Power (typ)	Internal	1.079 mW
	Switching	0.401 mW
	Leakage	0.95 µW
	Total	1.481 mW
Timing — typ	Setup WS	+2.138 ns
	Hold WS	+0.238 ns
	r2r setup WS	31.12 ns
	r2r hold WS	0.238 ns
Timing — slow (worst)	Setup WS	+0.483 ns
	Hold WS	+0.568 ns
Timing — fast	Setup WS	+2.763 ns
	Hold WS	+0.107 ns
Violations (all corners)	Setup	0
	Hold	0
	Max-slew	0
	Max-fanout	0
	Max-cap	0
IR Drop (typ)	Worst vccd1 drop	0.121 mV
	Worst vssd1 drop	0.109 mV

	Average drop	3.6 µV
	PG violations	0
Routing	Routed nets	26,915
	Special nets	2
	Wirelength	731,091 µm
	Max single-net WL	1,074.6 µm
	Vias (total)	165,009
	Antenna violations	0
	DRC errors (final)	0
Physical Verification	Magic DRC	0
	KLayout DRC	0
	Magic illegal overlaps	0
	LVS device differences	0
	LVS net differences	0
	LVS unmatched pins	0
	LVS property failures	0
	XOR differences	0
Flow	Errors	0
	Warnings	1 (informational)